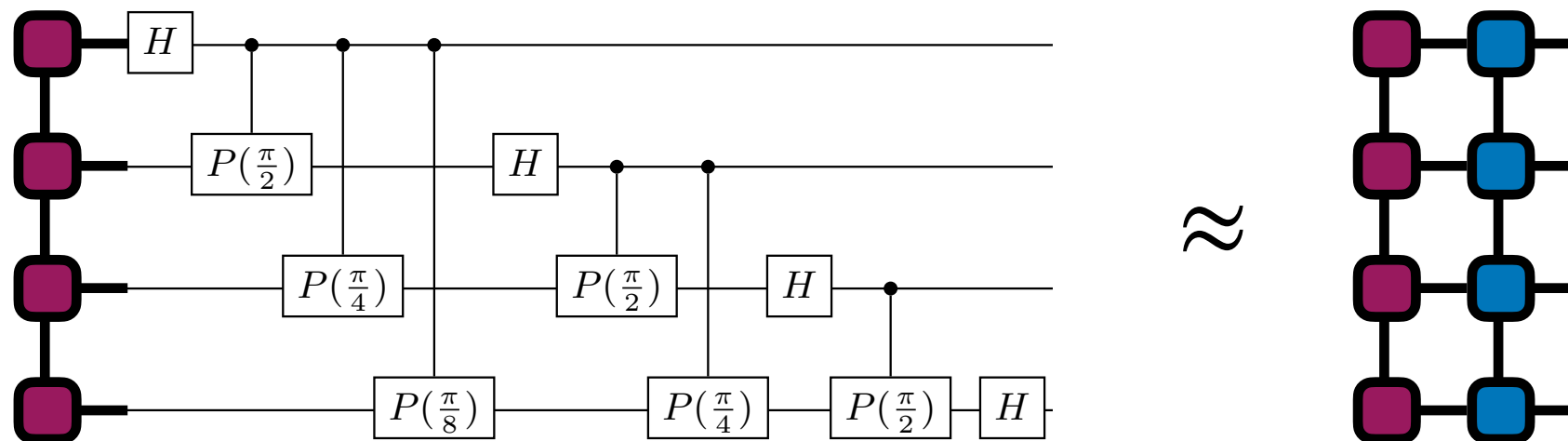


1. Taming High-Dimensional Spaces with Tensor Networks



July 2025

E.M. Stoudenmire

Numerical Methods in QIS, Amherst

 **FLATIRON**
INSTITUTE

SIMONS FOUNDATION

Flatiron Institute



SIMONS FOUNDATION

The mission of the Flatiron Institute is to advance scientific research through computational methods, including data analysis, modeling and simulation.



CCA: Center for Computational Astrophysics

CCB: Center for Computational Biology

CCQ: Center for Computational Quantum Physics

CCM: Center for Computational Mathematics

CCN: Center for Computational Neuroscience

ICC: Initiative for Computational Catalysis

Acknowledgements

Special thanks to:

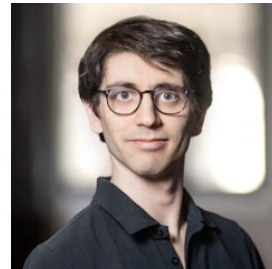
Karl Pierce
Flatiron Software Fellow



And current ITensor team



Matt Fishman



Olivier Gauthé



Lukas Devos

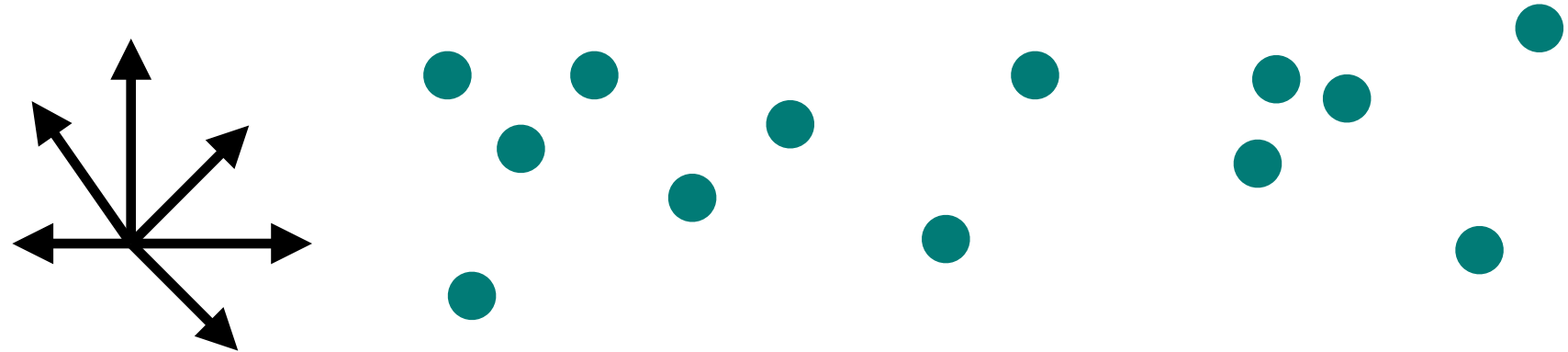


Joey Tindall

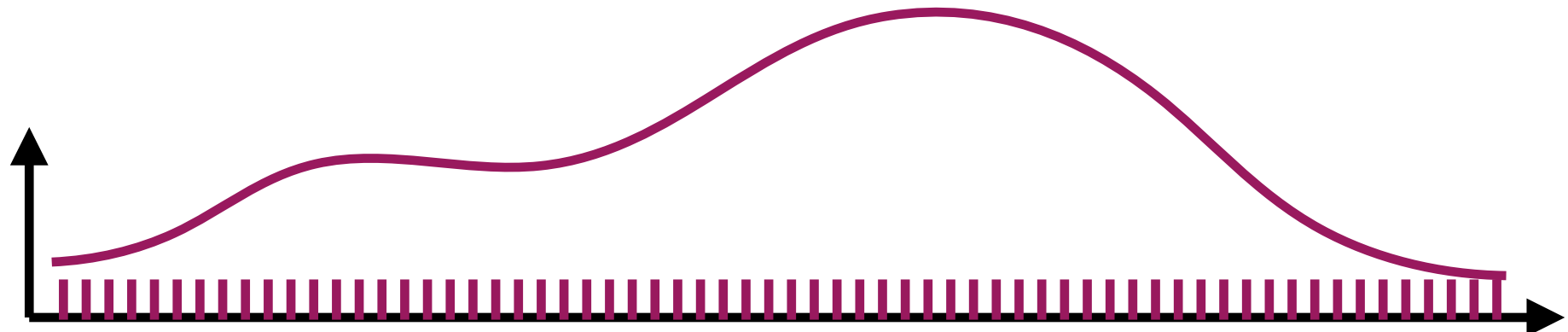
Plus alumni: Ryan Levy, Katie Hyatt, Giacomo Torlai
And Steven White.

Motivation

High-dimensional problems are hard for classical computers (when using brute force)



High-dimensional can mean many variables but (surprisingly) also continuous variables



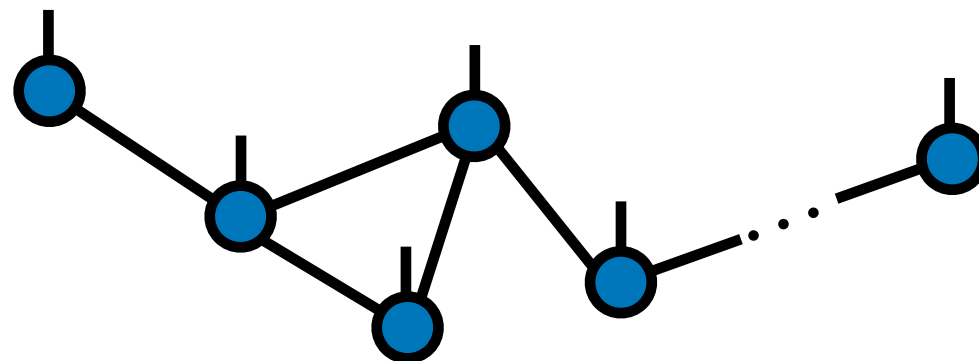
Motivation

Strength of **quantum computers** is working in high dimensional space (highly entangled states)



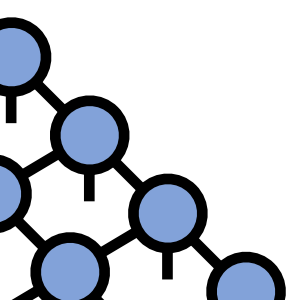
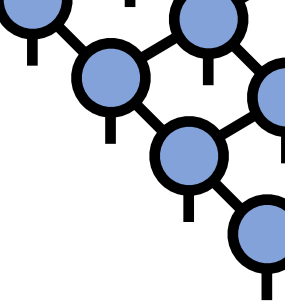
But can **classical computers** handle high dimensions with good **algorithms**?

Yes, sometimes! With **tensor networks**



Plan of this Talk

- Tensors and **Tensor Networks**
- Tensor Network **Algorithms**
 - DMRG algorithm
 - Time evolution algorithms



Introduction to Tensors and Tensor Networks

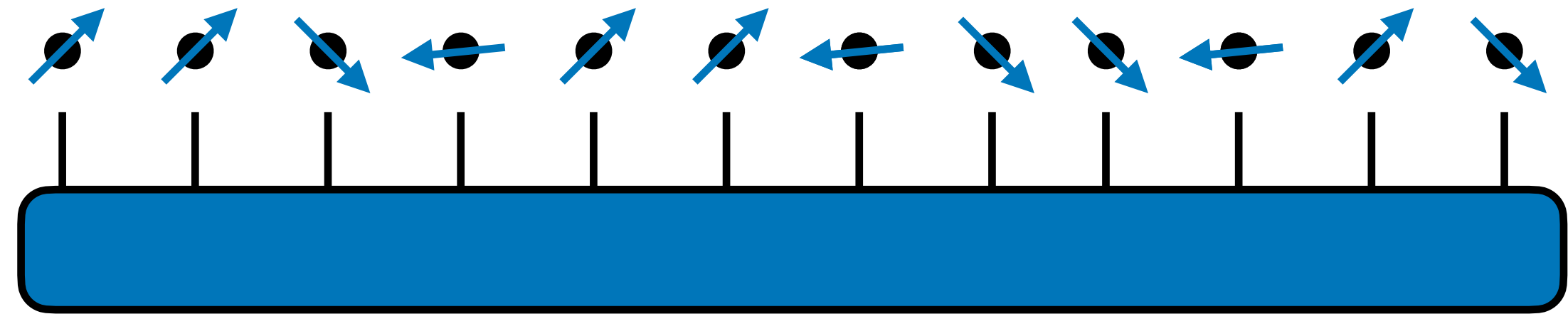
What is a tensor?

Why are they challenging?

Classical Methods for Quantum Systems

How hard is quantum mechanics?

Consider n quantum spins or qubits



quantum wavefunction Ψ

2^n parameters inside

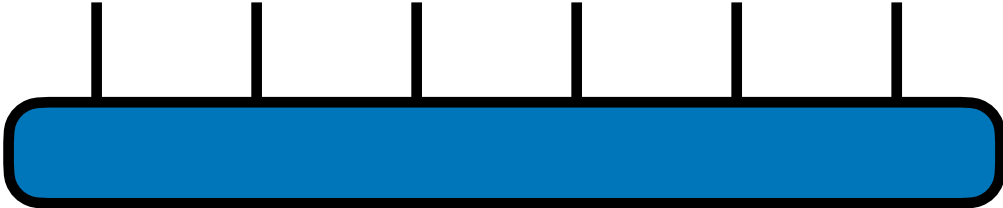
exponentially hard to store & manipulate

Classical Methods for Quantum Systems

General wavefunction of n qubits

$$|\Psi\rangle = \sum_{s_1 s_2 s_3 \cdots s_n} \Psi^{s_1 s_2 s_3 \cdots s_n} |s_1 s_2 s_3 \cdots s_n\rangle \quad s_j \in 0, 1$$

Amplitudes form a big tensor!

$$\Psi^{s_1 s_2 s_3 s_4 s_5 s_6} =$$


Classical Methods for Quantum Systems

What is a tensor?

vector

$$v = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$v_2 = 3$$

matrix

$$M = \begin{bmatrix} 5 & 7 \\ 8 & 9 \end{bmatrix}$$

$$M_{12} = 7$$

order-3
tensor

$$T = \begin{bmatrix} 3 & \begin{bmatrix} 5 & 4 \end{bmatrix} & 7 \\ 1 & \begin{bmatrix} 3 & 2 \end{bmatrix} & 5 \end{bmatrix}$$

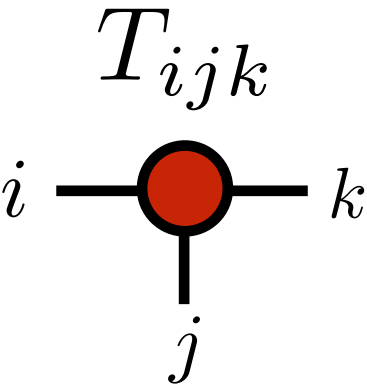
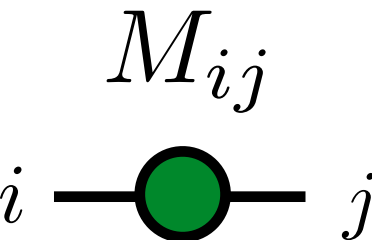
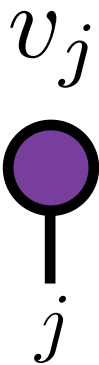
$$T_{112} = 5$$

Classical Methods for Quantum Systems

N-index tensor = shape with N lines

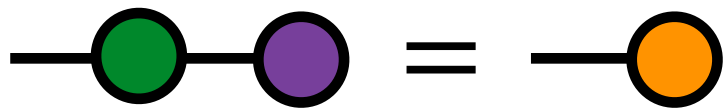
$$T^{s_1 s_2 s_3 \cdots s_N} = \text{[Diagram: A light gray rounded rectangle with 10 vertical lines extending upwards from its top edge. Above the lines are labels: } s_1, s_2, s_3, s_4, \cdot, \cdot, \cdot, \cdot, \cdot, s_N \text{.]}$$

Low-order examples:

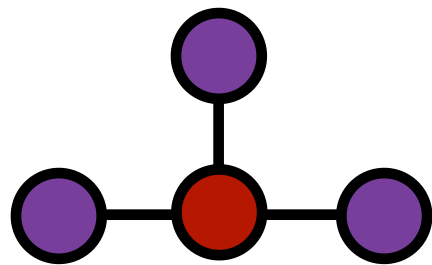


Tensors – Penrose Diagram Notation

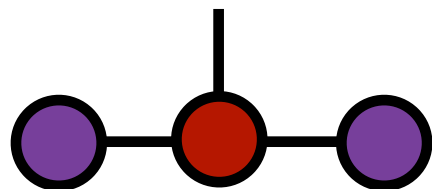
Joining wires means contraction:



$$\sum_j M_{ij} v_j = w_i$$



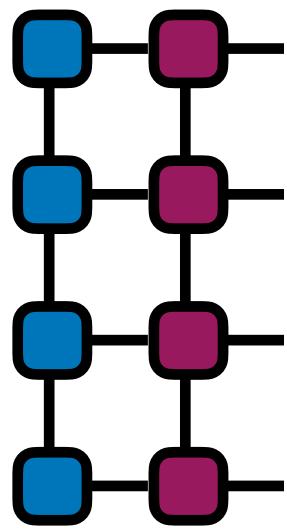
$$\sum_{i,j,k} T^{ijk} v_i v_j v_k$$



$$\sum_{i,k} T^{ijk} v_i v_k = z^j$$

Tensors – Penrose Diagram Notation

Much simpler than conventional expressions



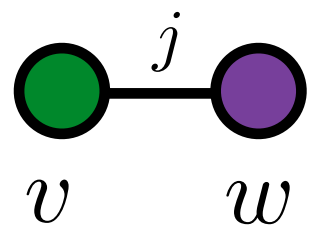
$$= \sum_{\{\alpha\}, \{\beta\}, \{s\}} M_{\alpha_1}^{s_1} M_{\alpha_1, \alpha_2}^{s_2} M_{\alpha_2, \alpha_3}^{s_3} M_{\alpha_3}^{s_4} \cdot W_{\beta_1}^{s_1, t_1} W_{\beta_1, \beta_2}^{s_2, t_2} W_{\beta_2, \beta_3}^{s_3, t_3} W_{\beta_3}^{s_4, t_4}$$

Equally rigorous

When using circuit diagrams, you're using this notation!

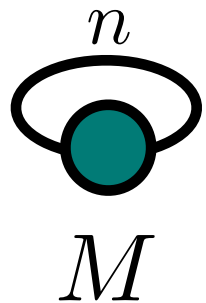
Tensors – Penrose Diagram Notation

Test your knowledge – describe these operations
in your own words



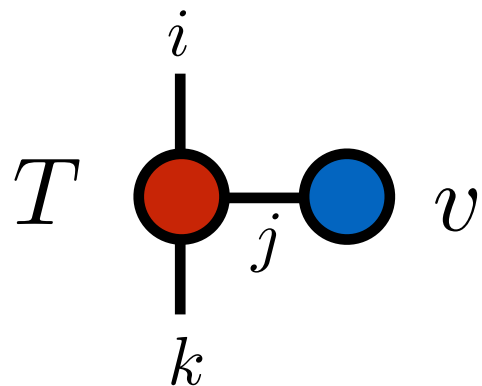
$$\sum_j v_j w_j$$

vector inner product



$$\sum_n M_{nn}$$

trace of a matrix

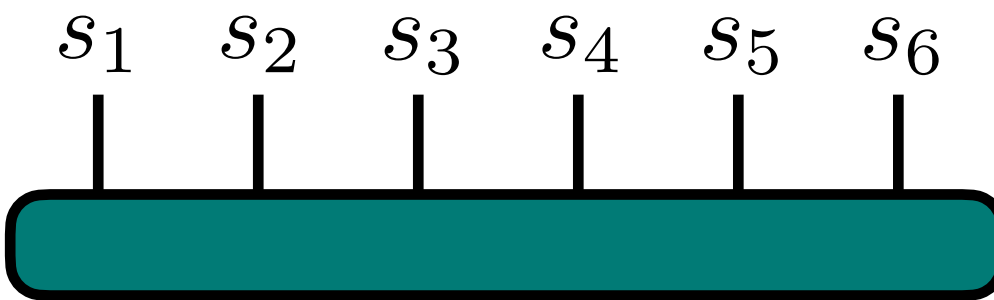


$$\sum_j T_{ijk} v_j = R_{ik}$$

contract order-3
tensor with vector

Tensors

Any function of discrete variables can be represented as a tensor

$$f(s_1, s_2, s_3, s_4, s_5, s_6) =$$


All values of f inside



Tensors

Any function of discrete variables can be represented as a tensor

$$f(1, 2, 2, 2, 1, 2) = \text{[teal bar with indices 1, 2, 2, 2, 1, 2]} = 1.2$$

Tensors

Any function of discrete variables can be represented as a tensor

$$f(2, 2, 2, 2, 2, 1) = \text{[teal bar with 6 ticks labeled 2, 2, 2, 2, 2, 1]} = 0.3$$

Tensors

Any function of discrete variables can be represented as a tensor

$$f(2, 1, 2, 2, 2, 2) = \text{[teal bar with indices 2, 1, 2, 2, 2, 2]} = -0.2$$

Tensors

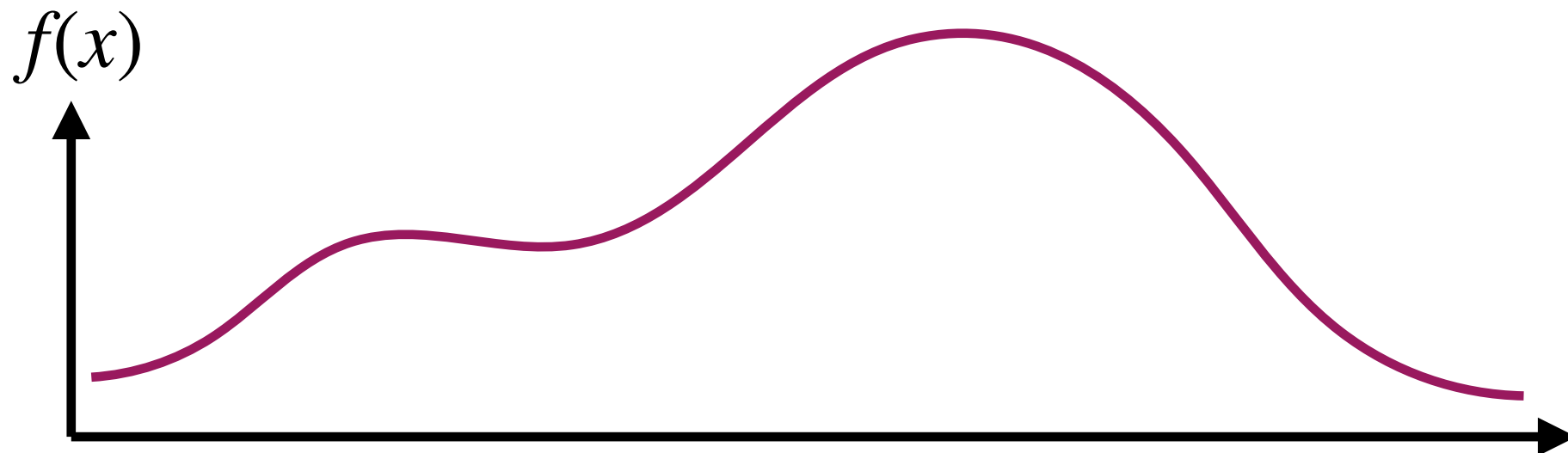
Any function of discrete variables can be represented as a tensor

$$f(2, 1, 1, 2, 2, 2) = \text{[teal bar with indices 2, 1, 1, 2, 2, 2]} = 2.7$$

Tensors

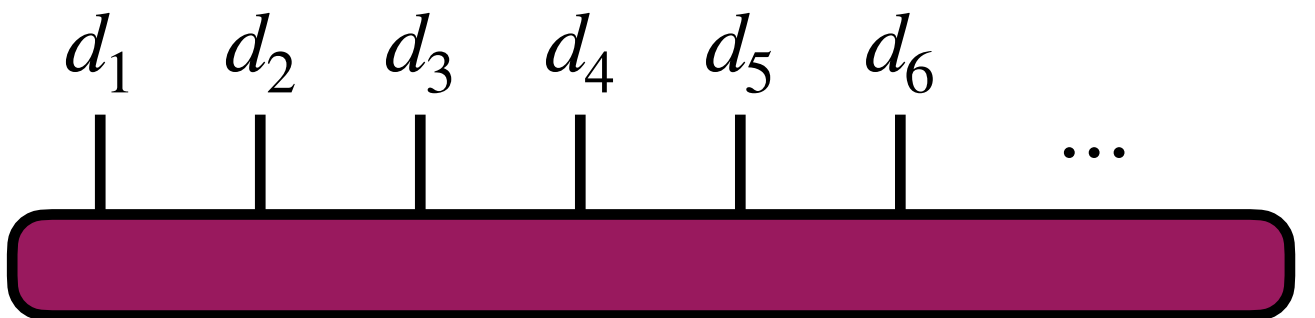
By a simple trick, can encode
continuous arguments too

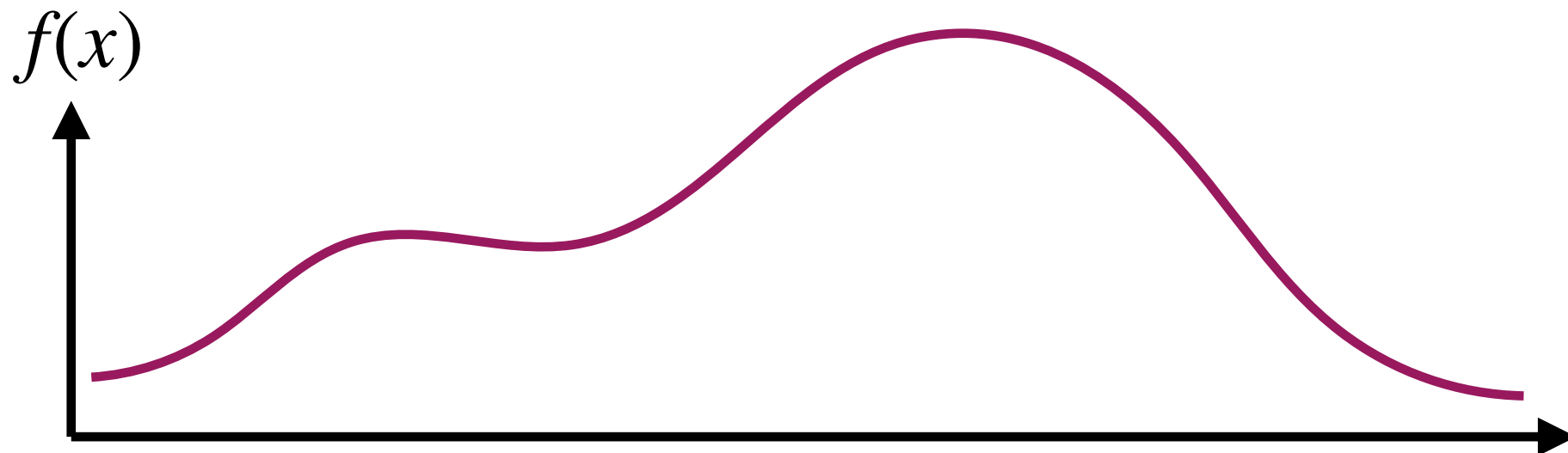
$$f(x) = \text{ⓧ}_x$$



Tensors

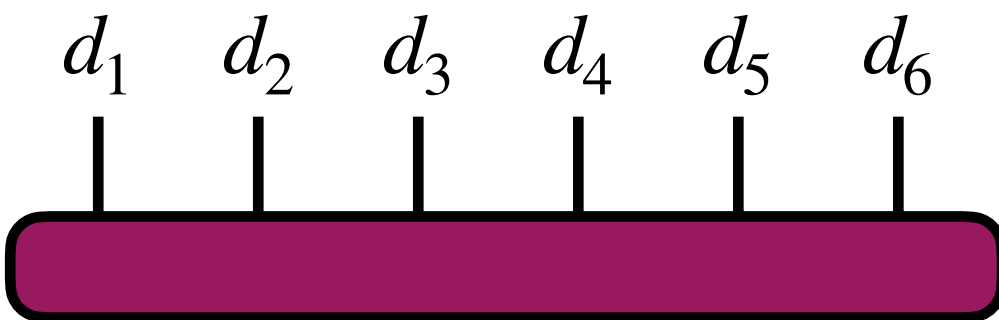
By a simple trick, can encode
continuous arguments too

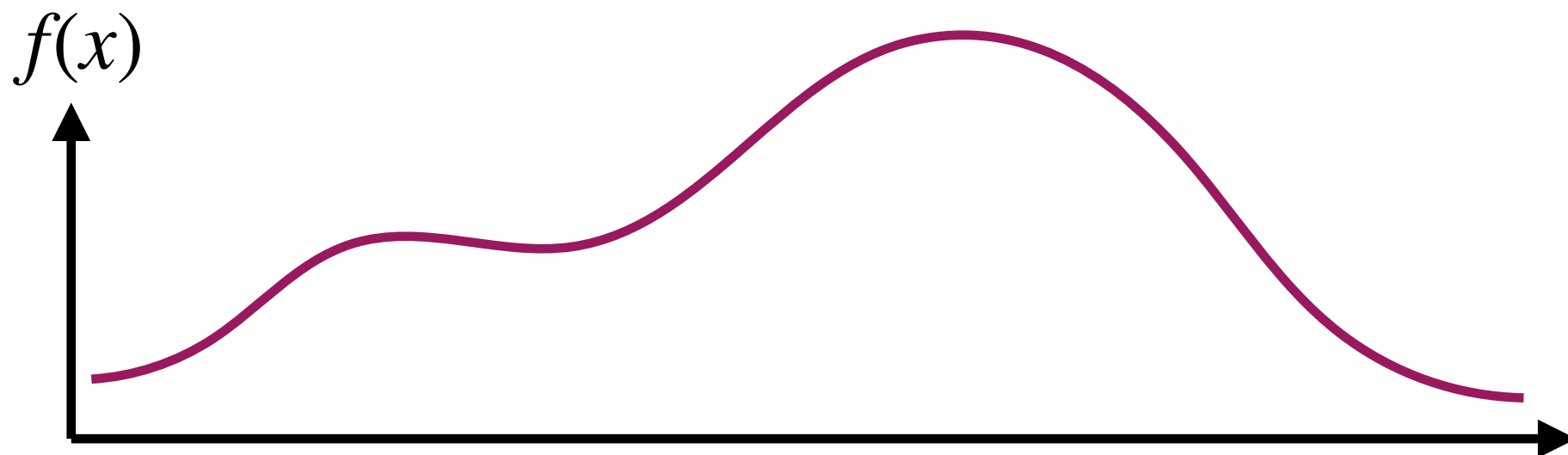
$$f(x) =$$




Tensors

By a simple trick, can encode
continuous arguments too

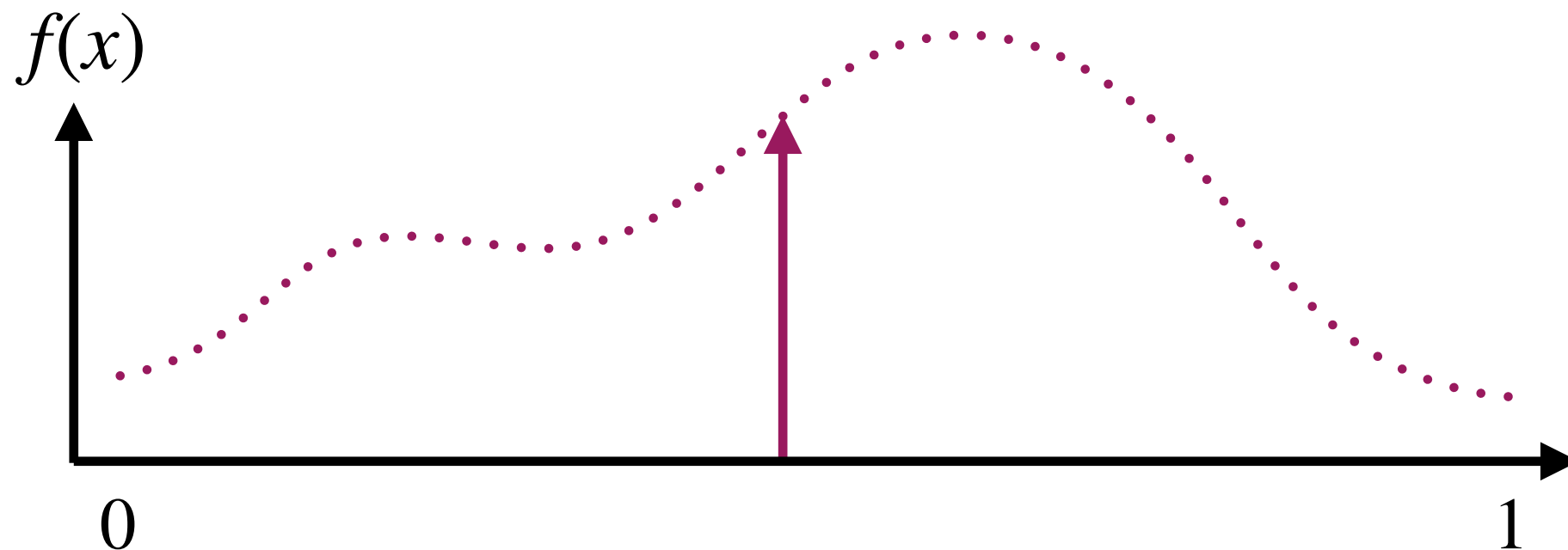
$$f(x) \approx f(0.d_1d_2d_3d_4d_5d_6) =$$




Why Tensors?

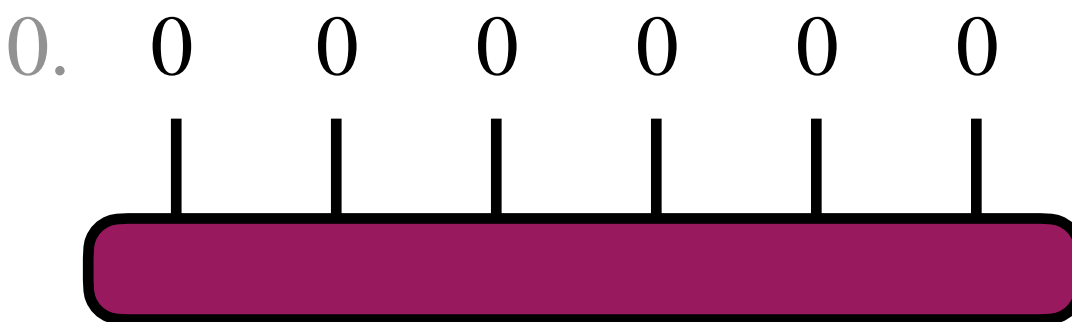
By a simple trick, can encode
continuous arguments too

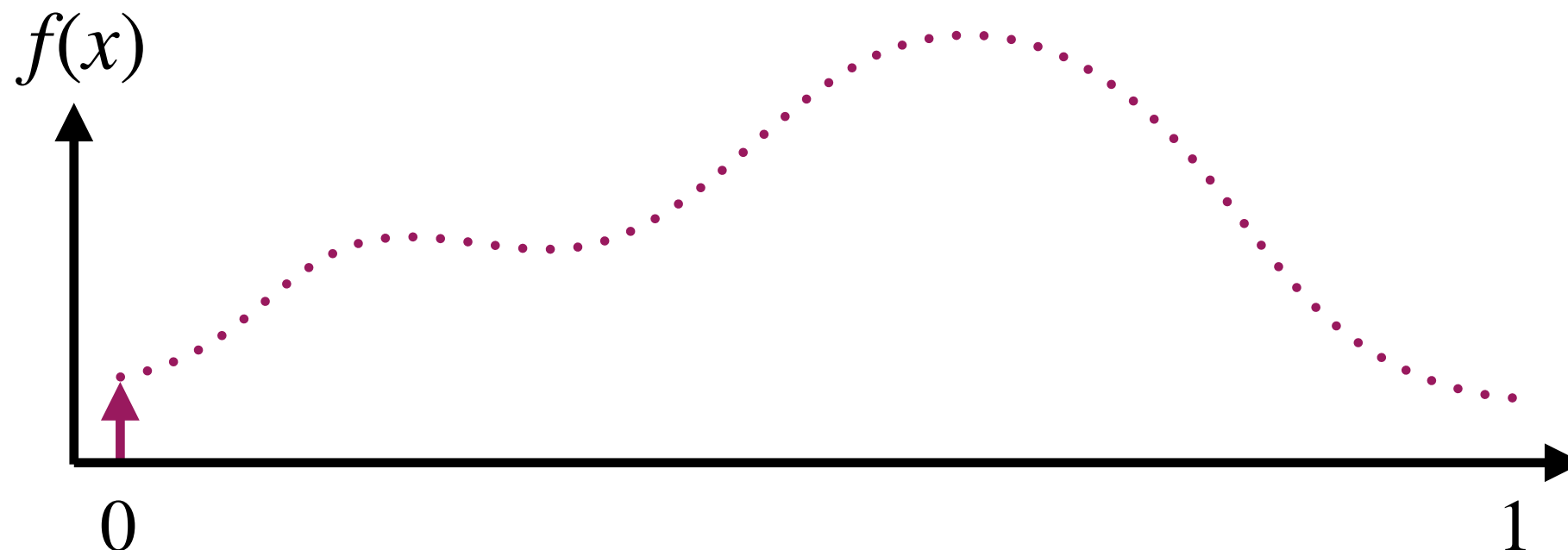
$$f(x) \approx f(0.500000) =$$

Why Tensors?

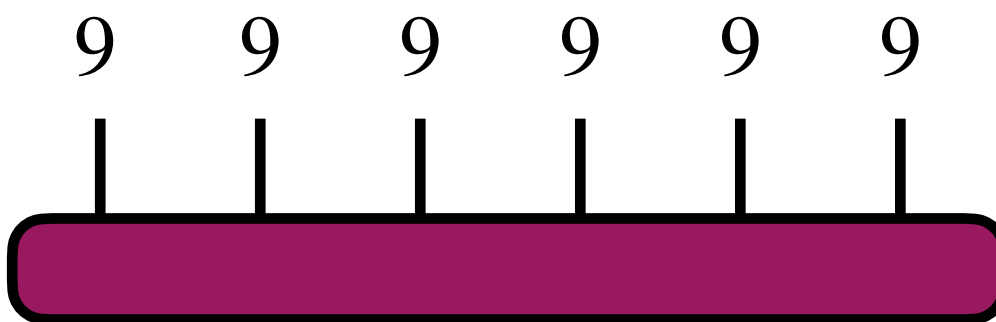
By a simple trick, can encode
continuous arguments too

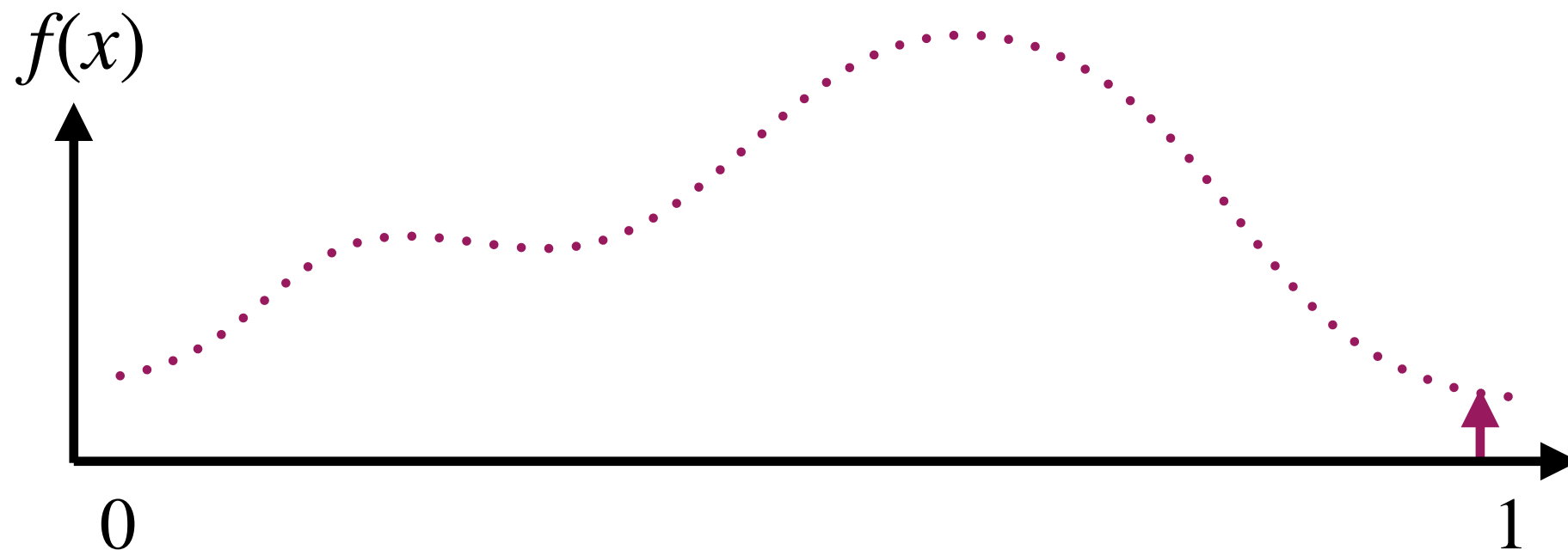
$$f(x) \approx f(0.000000) =$$




Why Tensors?

By a simple trick, can encode
continuous arguments too

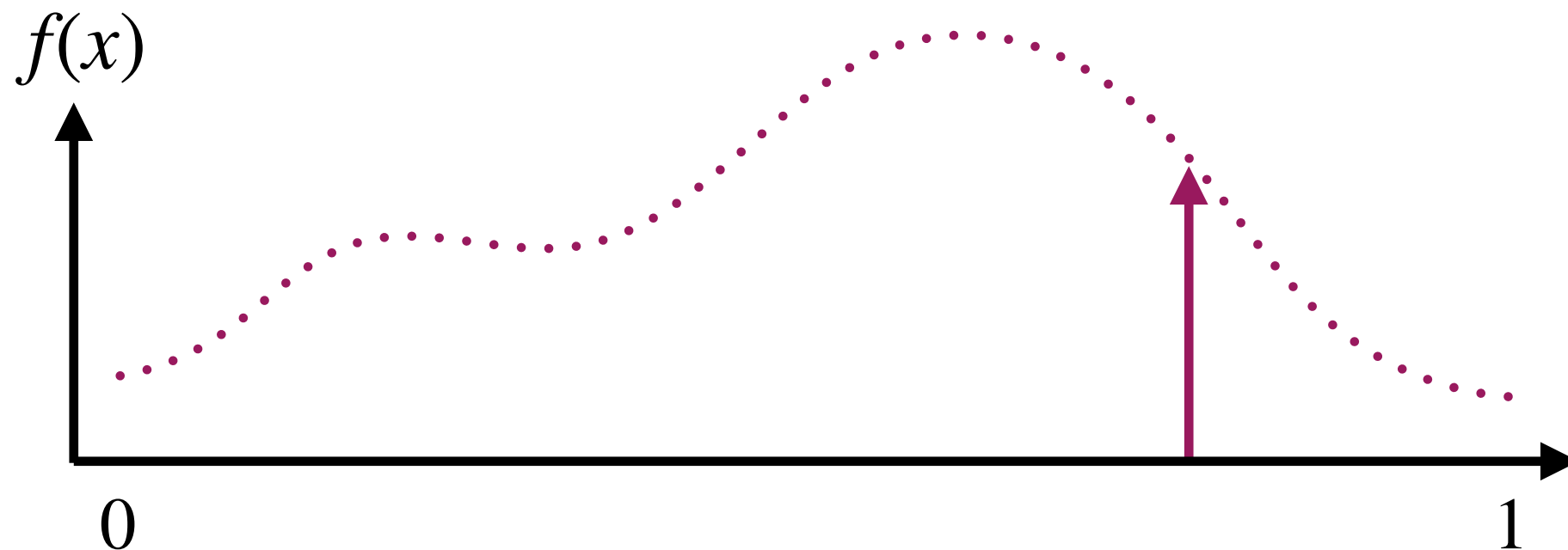
$$f(x) \approx f(0.9999999) =$$




Why Tensors?

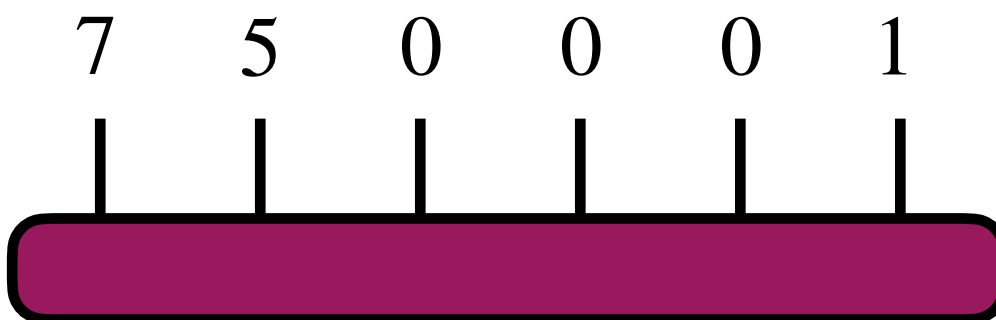
By a simple trick, can encode
continuous arguments too

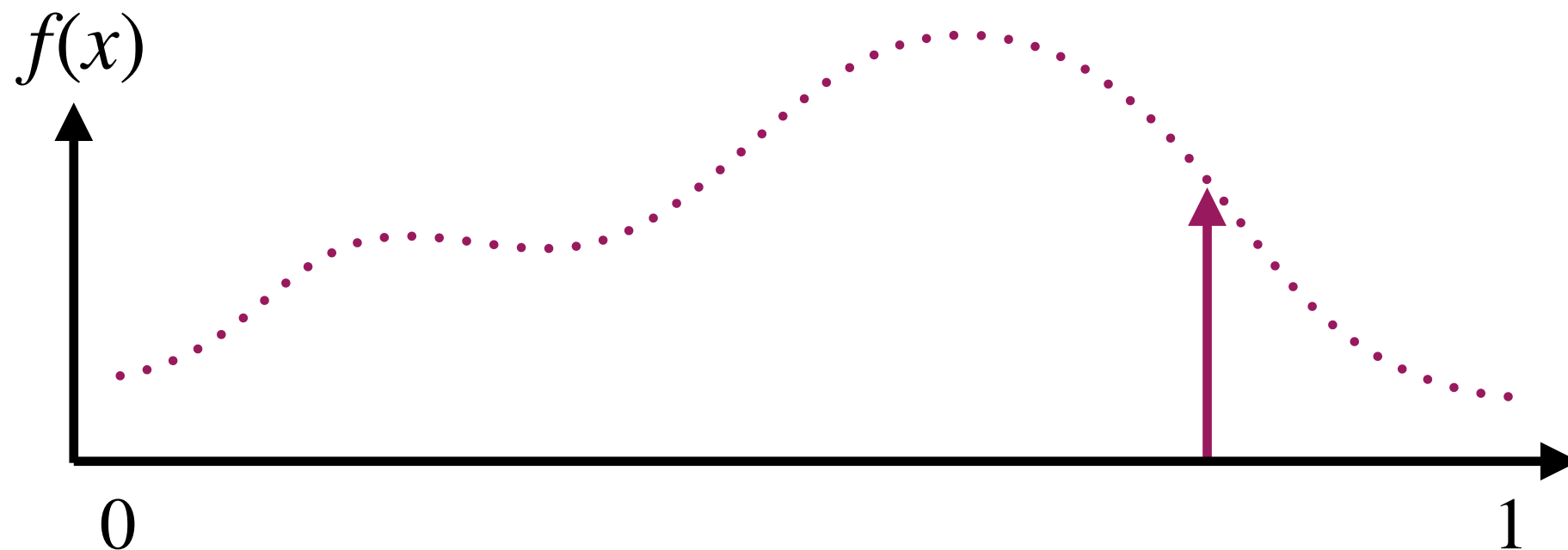
$$f(x) \approx f(0.750000) =$$

Why Tensors?

By a simple trick, can encode
continuous arguments too

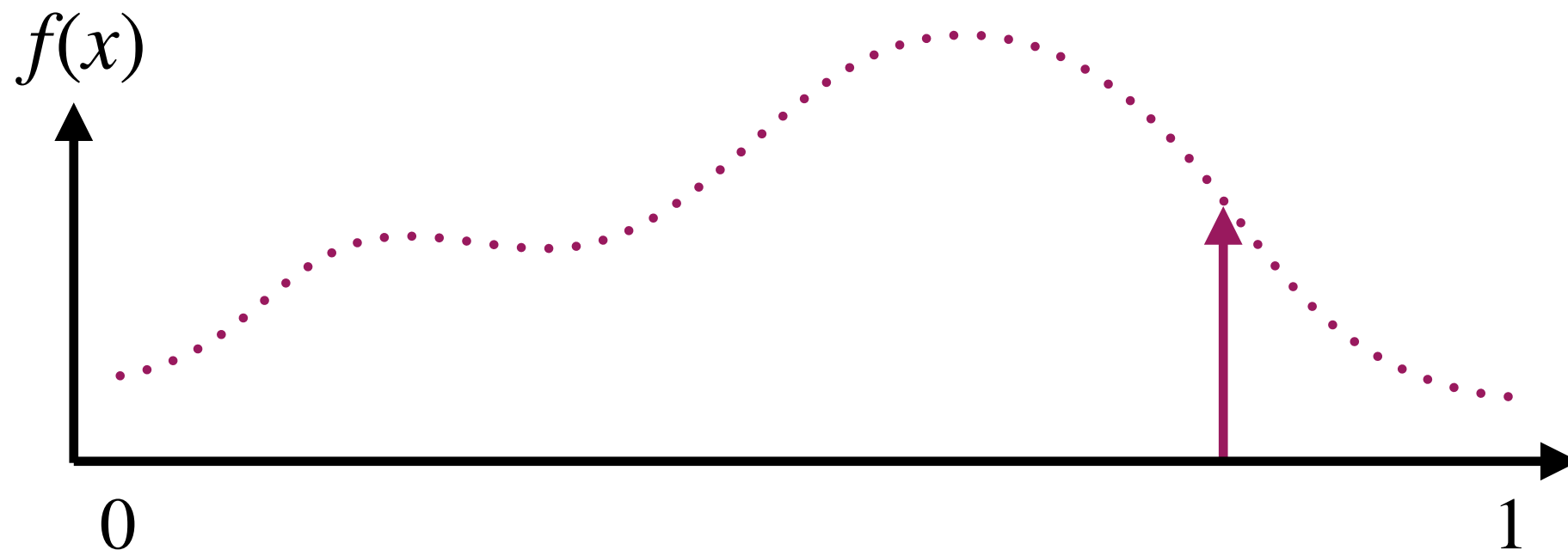
$$f(x) \approx f(0.750001) =$$




Why Tensors?

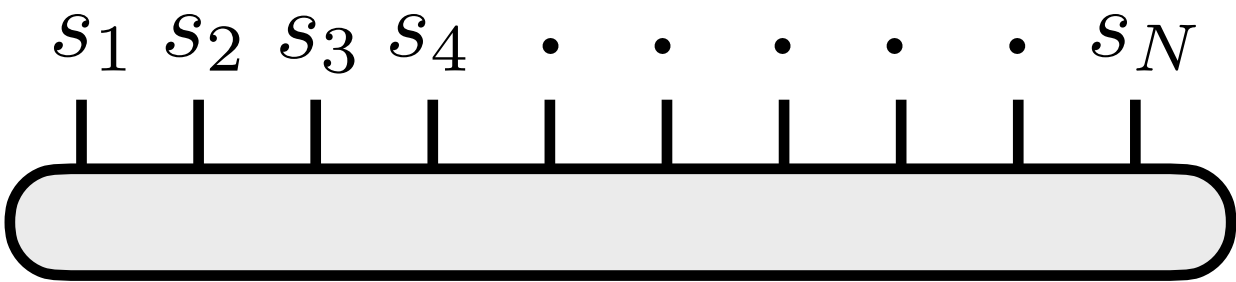
By a simple trick, can encode
continuous arguments too

$$f(x) \approx f(0.750002) =$$

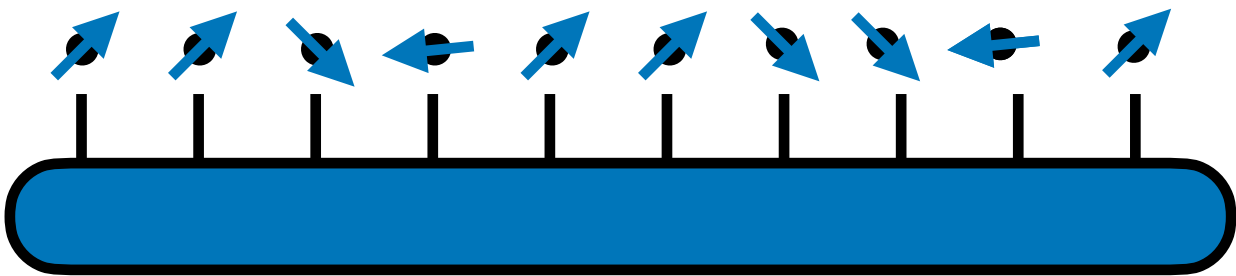



Classical Methods for Quantum Systems

N-index tensor **exponential** to store

$$T^{s_1 s_2 s_3 \cdots s_N} =$$


Tensor version of "**many-body problem**"

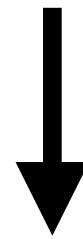
$$|\Psi\rangle =$$


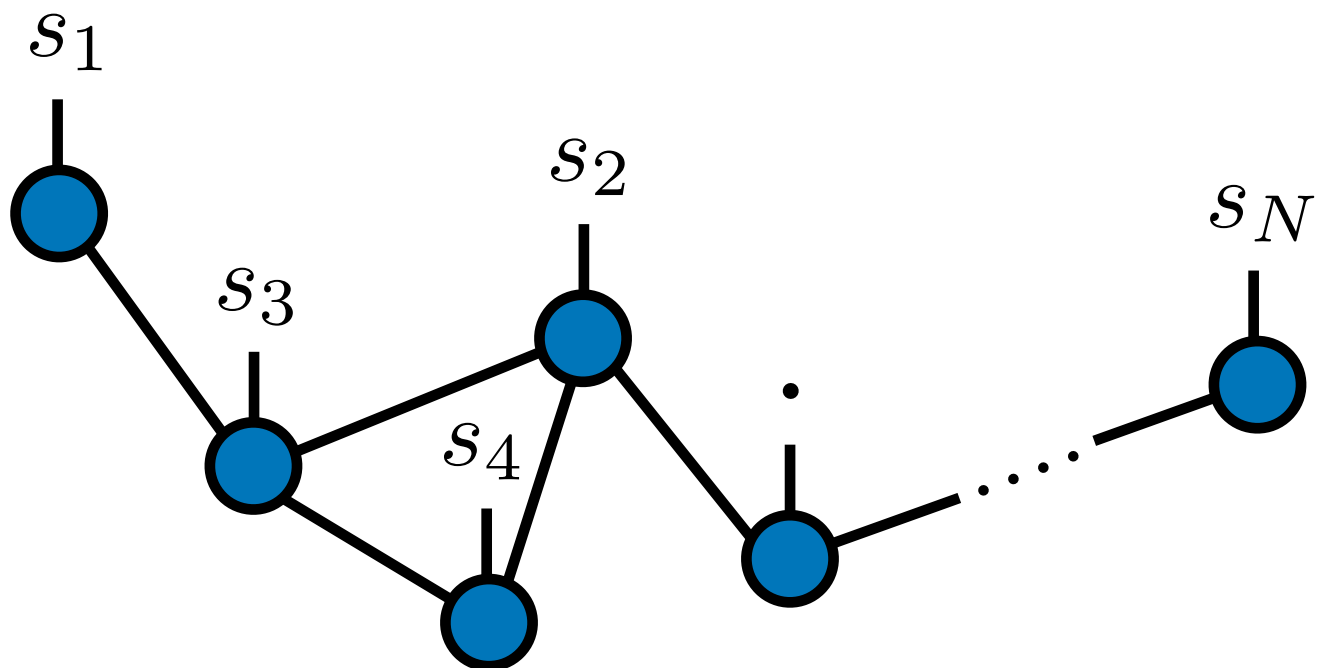
Curse of Dimensionality

Tensor networks give a way to break the curse

$$T^{s_1 s_2 s_3 \cdots s_N} =$$


A horizontal bar with rounded ends, representing a high-dimensional tensor. Above the bar, there are vertical tick marks corresponding to indices s_1, s_2, s_3, s_4 , followed by three dots, and then s_N .



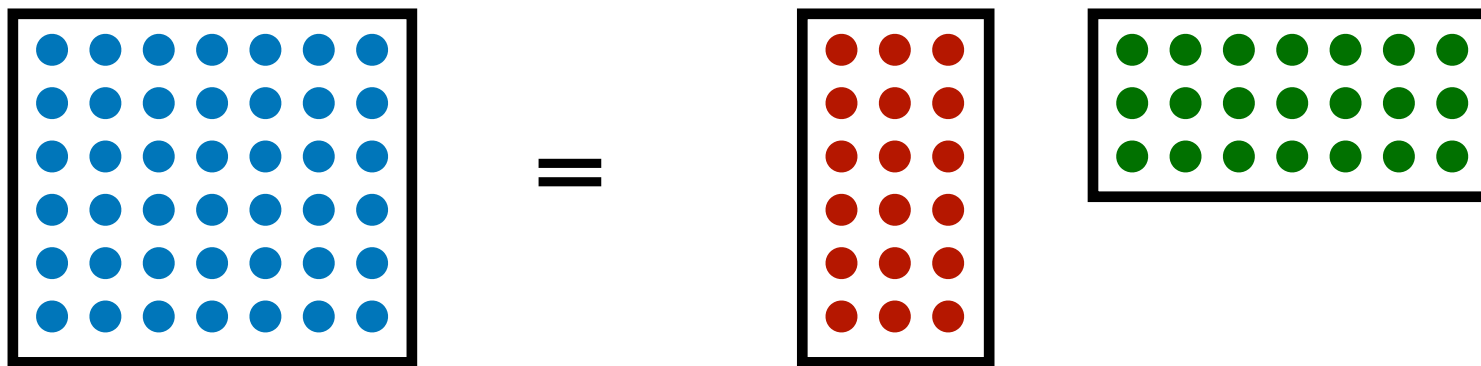
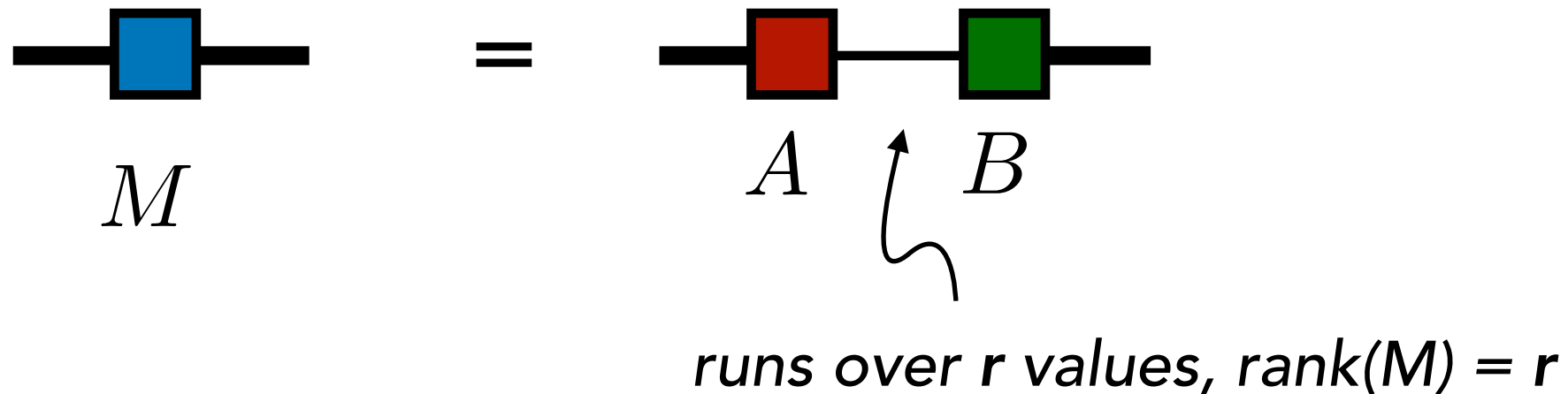
$$T^{s_1 s_2 s_3 \cdots s_N} \approx$$


A tensor network diagram consisting of blue circular nodes connected by black lines. The nodes are arranged in a zig-zag pattern. The first node is connected to s_1 . The second node is connected to s_3 . The third node is connected to s_4 . The fourth node is connected to s_2 . The fifth node is connected to a vertical ellipsis. The sixth node is connected to s_N . The diagram illustrates a decomposition of the high-dimensional tensor into a network of lower-dimensional tensors.

Low Rank Structure and the SVD

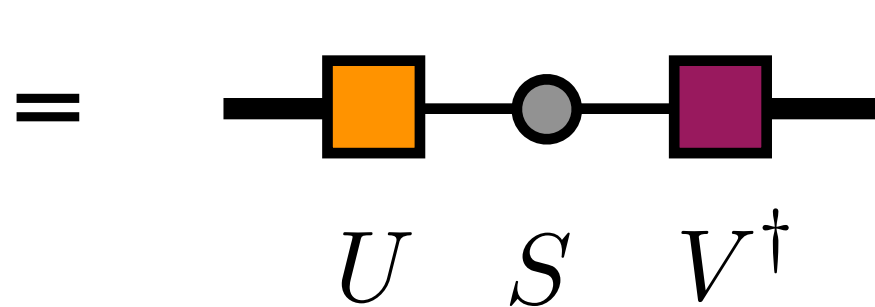
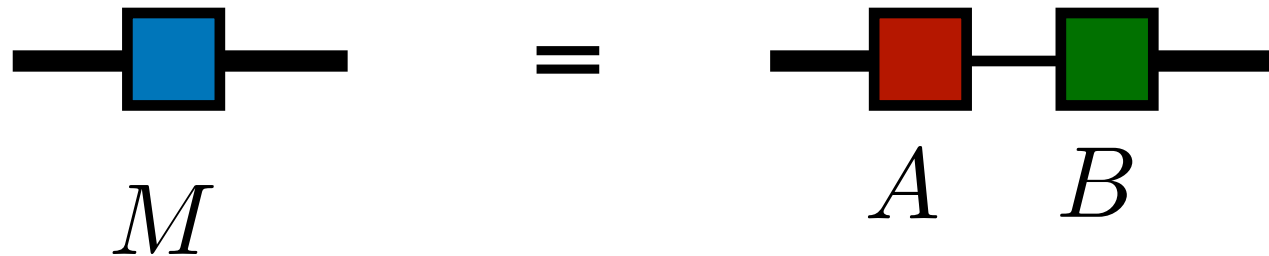
Low-rank Structure

Finding low-rank structure solved for matrices



Low-rank Structure

Finding low-rank structure solved for matrices



$$A = U\sqrt{S}$$
$$B = \sqrt{S}V^\dagger$$

Solved by singular value
decomposition (**SVD**)

Review: *Singular Value Decomposition (SVD)*

Given rectangular (4x3) matrix M

$$M = \begin{bmatrix} 0.435839 & 0.223707 & 0.10 \\ 0.435839 & 0.223707 & -0.10 \\ 0.223707 & 0.435839 & 0.10 \\ 0.223707 & 0.435839 & -0.10 \end{bmatrix}$$

Can factorize as

$$\begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 0.933 & 0 & 0 \\ 0 & 0.300 & 0 \\ 0 & 0 & 0.200 \end{bmatrix} \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 0.933 & 0 & 0 \\ 0 & 0.300 & 0 \\ 0 & 0 & 0.200 \end{bmatrix} \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

U
 S
 V^T

Matrices U and V have orthonormal columns:

$$U^T U = 1$$

$$V^T V = 1$$

S diagonal = "singular values"

Elements of S always:

- 1) Real
- 2) Non-negative
- 3) Decreasing

Keep fewer and fewer elements of S:

$$\begin{array}{c}
 \textcolor{red}{U} \\
 \begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \end{bmatrix}
 \end{array}
 \begin{array}{c}
 S \\
 \begin{bmatrix} 0.933 & 0 & 0 \\ 0 & 0.300 & 0 \\ 0 & 0 & 0.200 \end{bmatrix}
 \end{array}
 \begin{array}{c}
 \textcolor{blue}{V}^T \\
 \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}
 \end{array}$$

$$= M = \begin{bmatrix} 0.435839 & 0.223707 & 0.10 \\ 0.435839 & 0.223707 & -0.10 \\ 0.223707 & 0.435839 & 0.10 \\ 0.223707 & 0.435839 & -0.10 \end{bmatrix}$$

$$||M - M||^2 = 0$$

Keep fewer and fewer elements of S:

$$\begin{array}{c} \textcolor{red}{U} \end{array} \begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \end{bmatrix} \begin{array}{c} S \end{array} \begin{bmatrix} 0.933 & 0 & 0 \\ 0 & 0.300 & 0 \\ 0 & 0 & 0.200 \end{bmatrix} \begin{array}{c} \textcolor{blue}{V}^T \end{array} \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= M_2 = \begin{bmatrix} 0.435839 & 0.223707 & 0 \\ 0.435839 & 0.223707 & 0 \\ 0.223707 & 0.435839 & 0 \\ 0.223707 & 0.435839 & 0 \end{bmatrix}$$

$$\|\cancel{M_2} - M\|^2 = 0.04 = (0.2)^2$$

Keep fewer and fewer elements of S:

$$\begin{matrix}
 \textcolor{red}{U} & S & \textcolor{blue}{V}^T \\
 \begin{bmatrix} 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & -1/2 \\ 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 \end{bmatrix} & \begin{bmatrix} 0.933 & 0 & 0 \\ 0 & 0.300 & 0 \\ 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \end{bmatrix}
 \end{matrix}$$

$$= M_3 = \begin{bmatrix} 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \end{bmatrix}$$

Truncating SVD =
Controlled
approximation for M

$$||M_3 - M||^2 = 0.03 = (0.3)^2 + (0.2)^2$$

Keep fewer and fewer elements of S:

$$\begin{matrix} \textcolor{red}{U} & S & \textcolor{blue}{V}^T \\ \begin{bmatrix} 1/2 \\ 1/2 \\ 1/2 \\ 1/2 \end{bmatrix} & \begin{bmatrix} 0.933 \end{bmatrix} & \begin{bmatrix} 0.707107 & 0.707107 & 0 \end{bmatrix} \end{matrix}$$

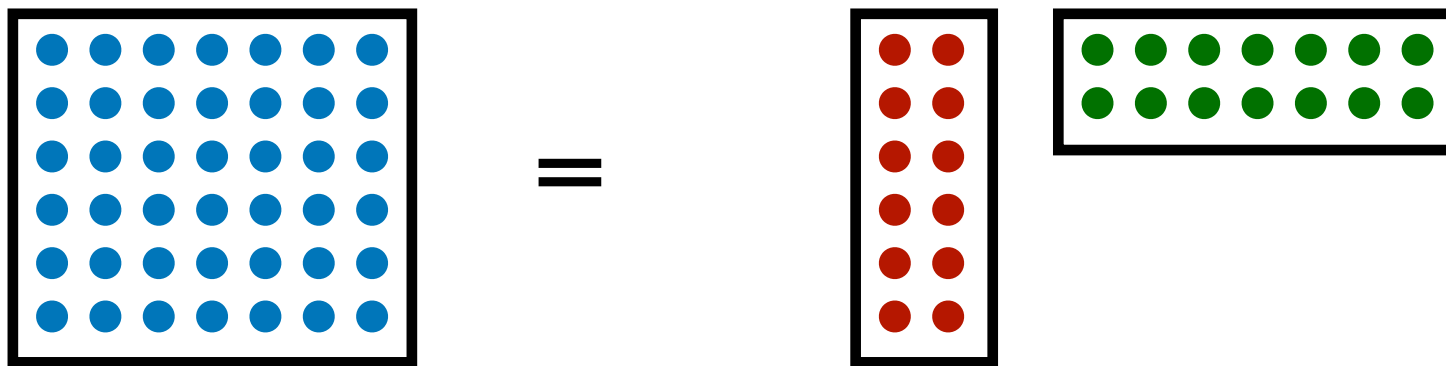
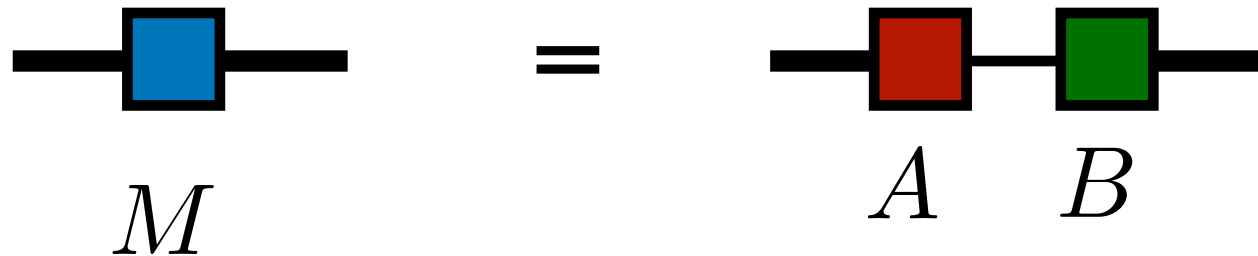
$$= M_3 = \begin{bmatrix} 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \\ 0.329773 & 0.329773 & 0 \end{bmatrix}$$

Truncating SVD =
Controlled
approximation for M

$$||M_3 - M||^2 = 0.13 = (0.3)^2 + (0.2)^2$$

Low-rank Structure

Low-rank saves memory and compute



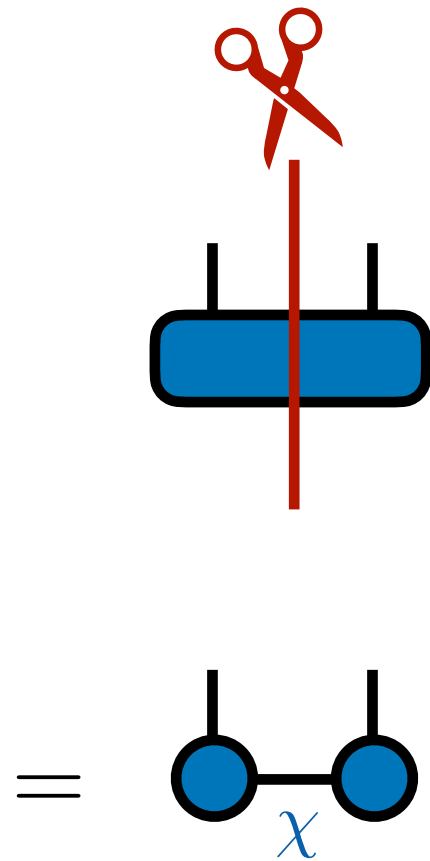
↑
Many parameters

↑
Few parameters

Matrix Product State (MPS) Tensor Networks

Classical Methods for Quantum Systems

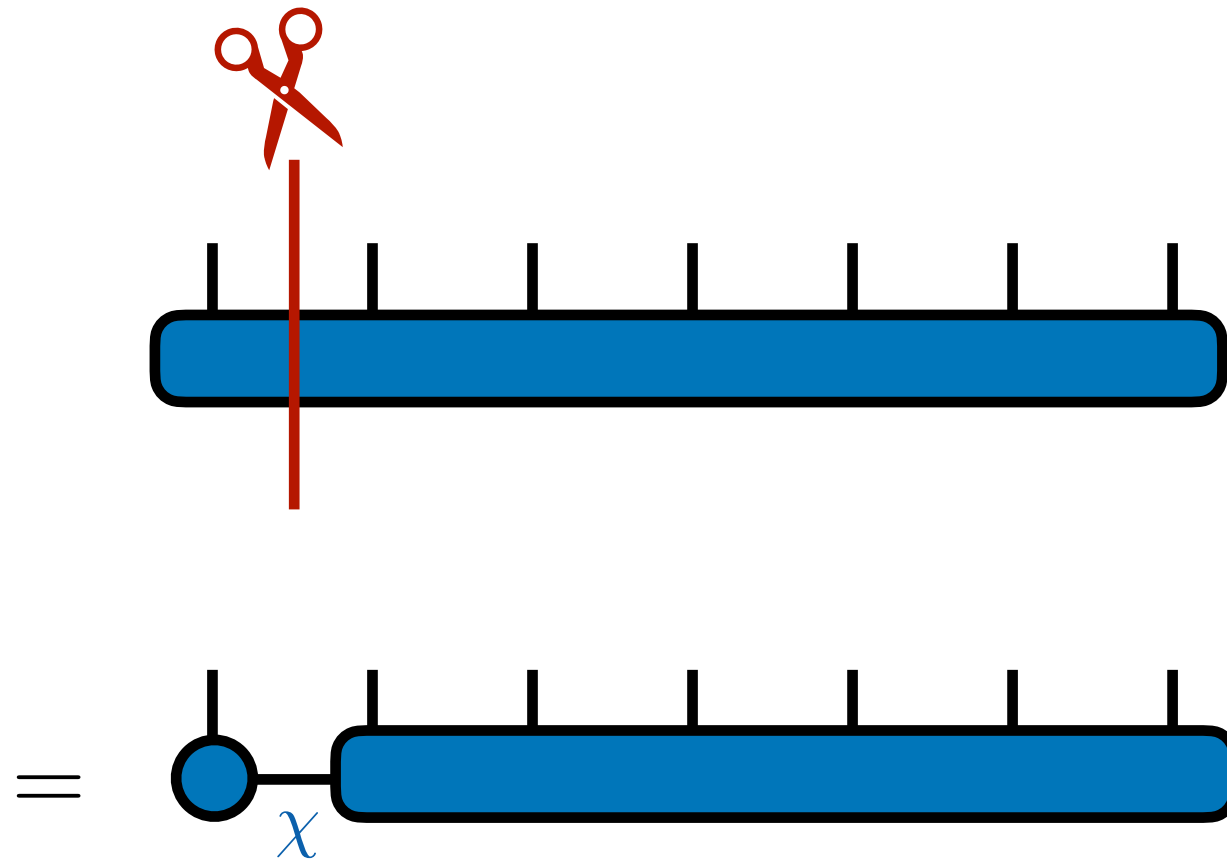
Just as factorizing a matrix reduces cost
(memory and compute)



χ is matrix rank

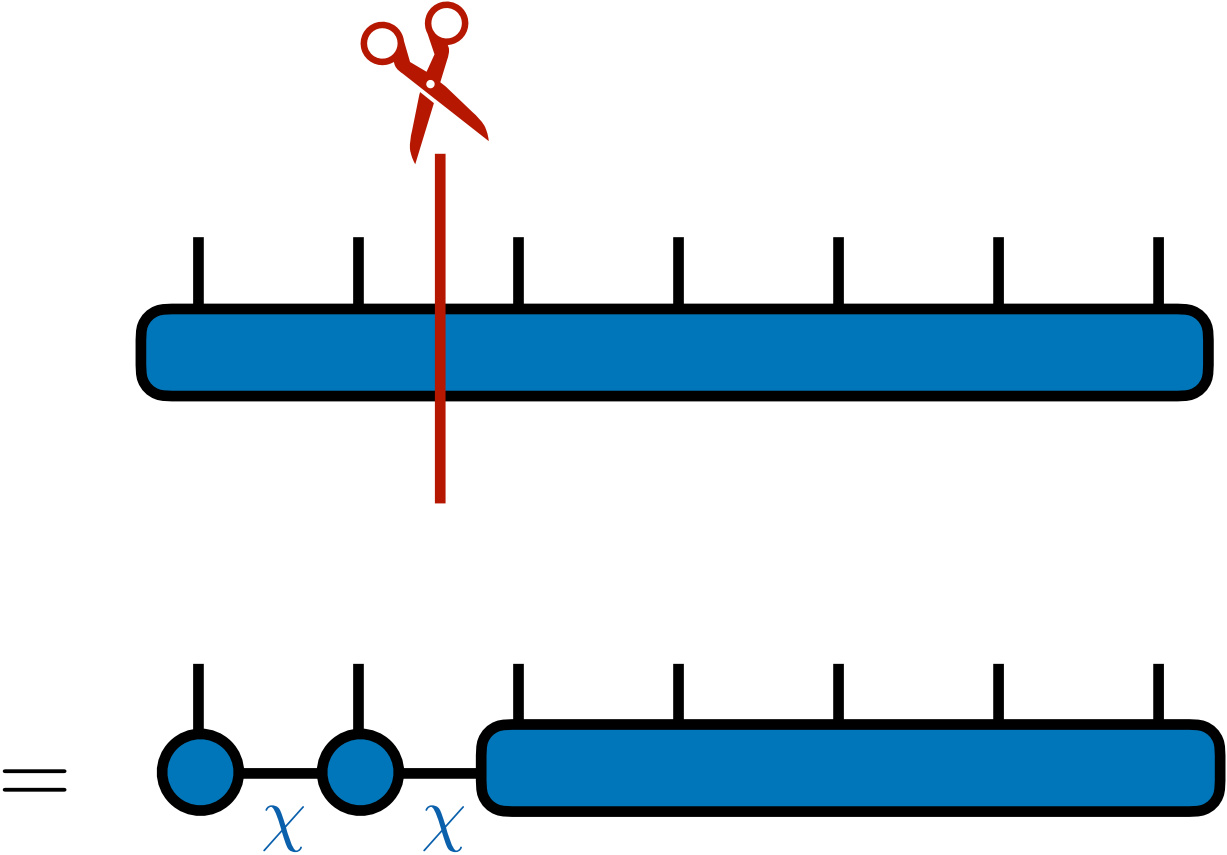
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



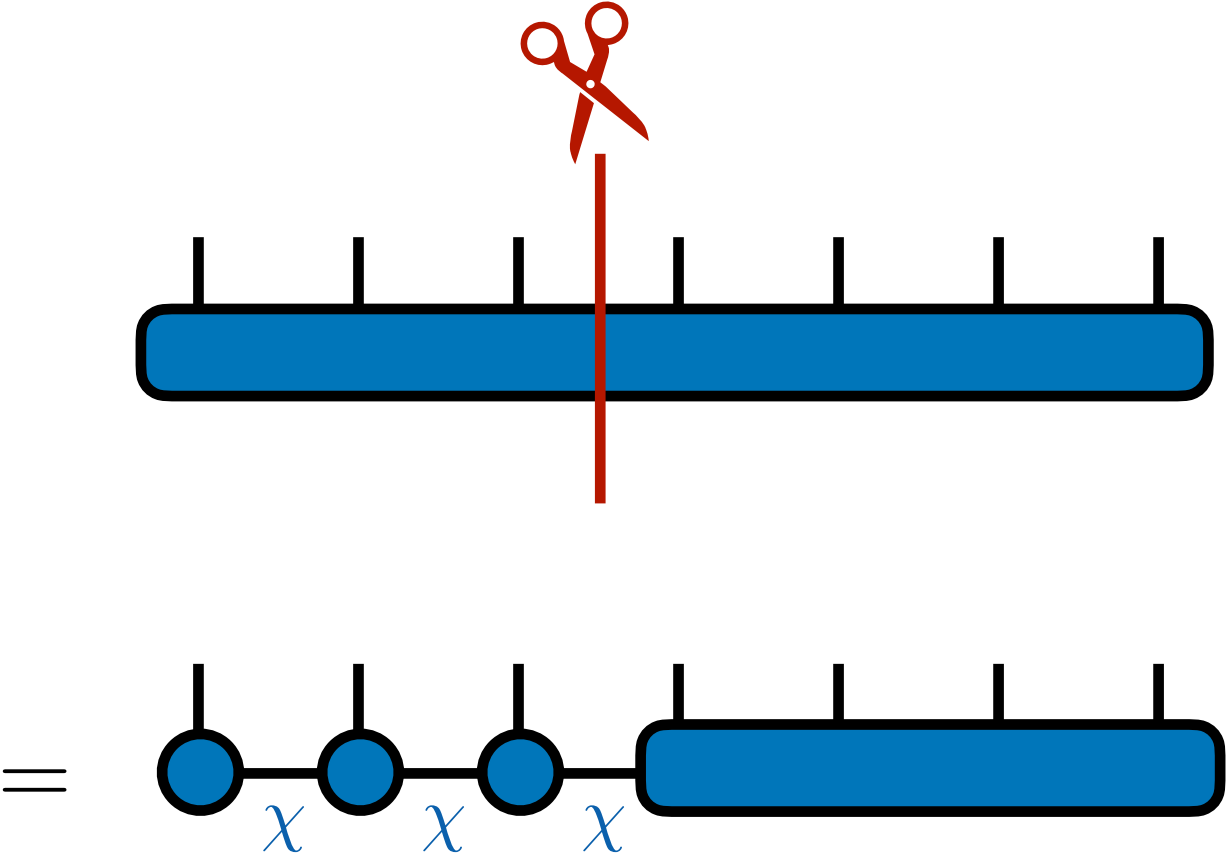
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



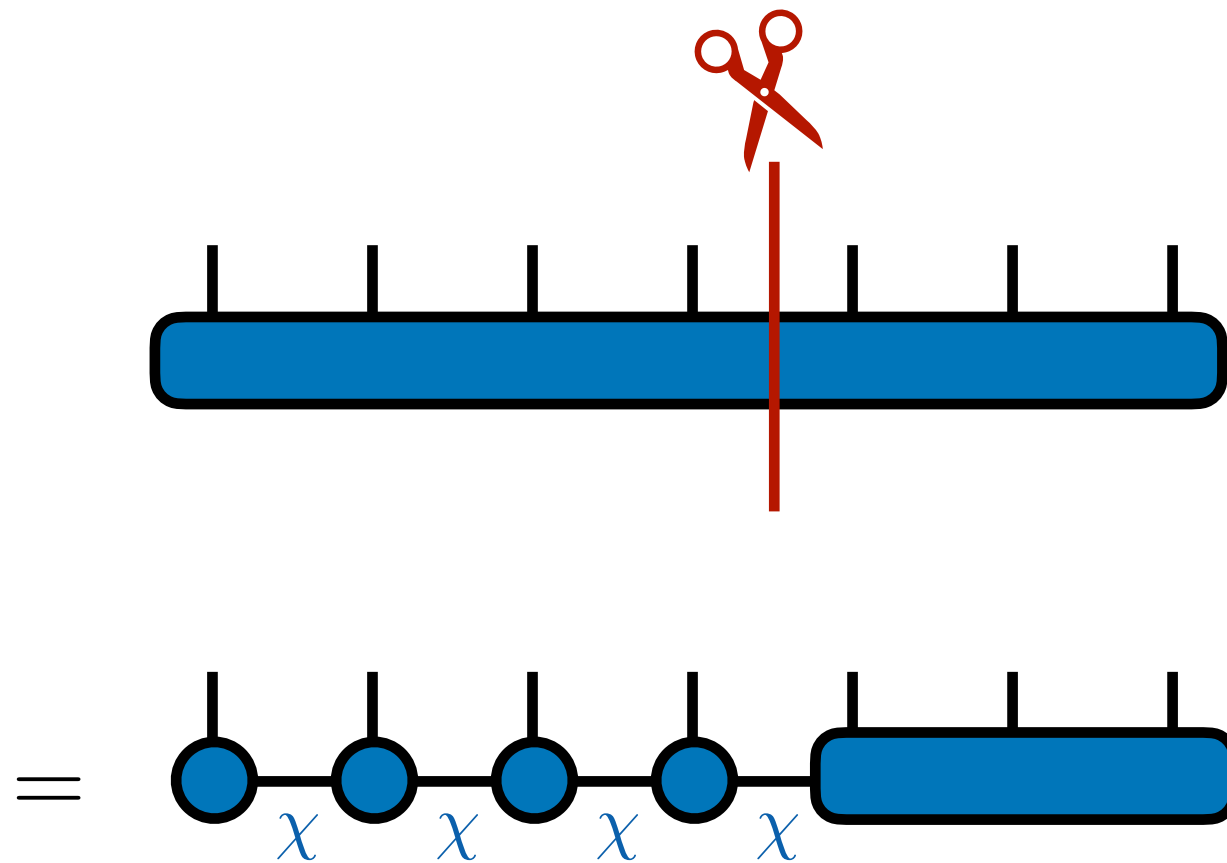
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



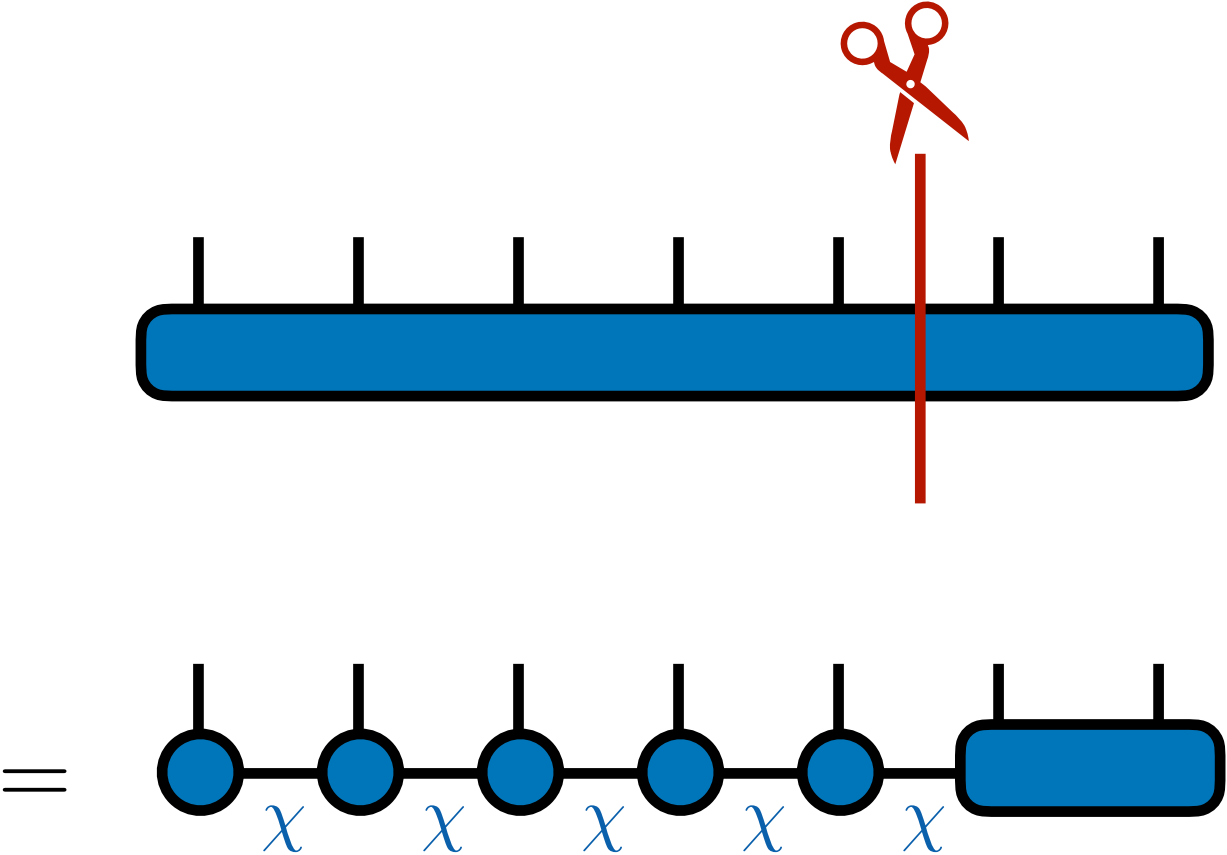
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



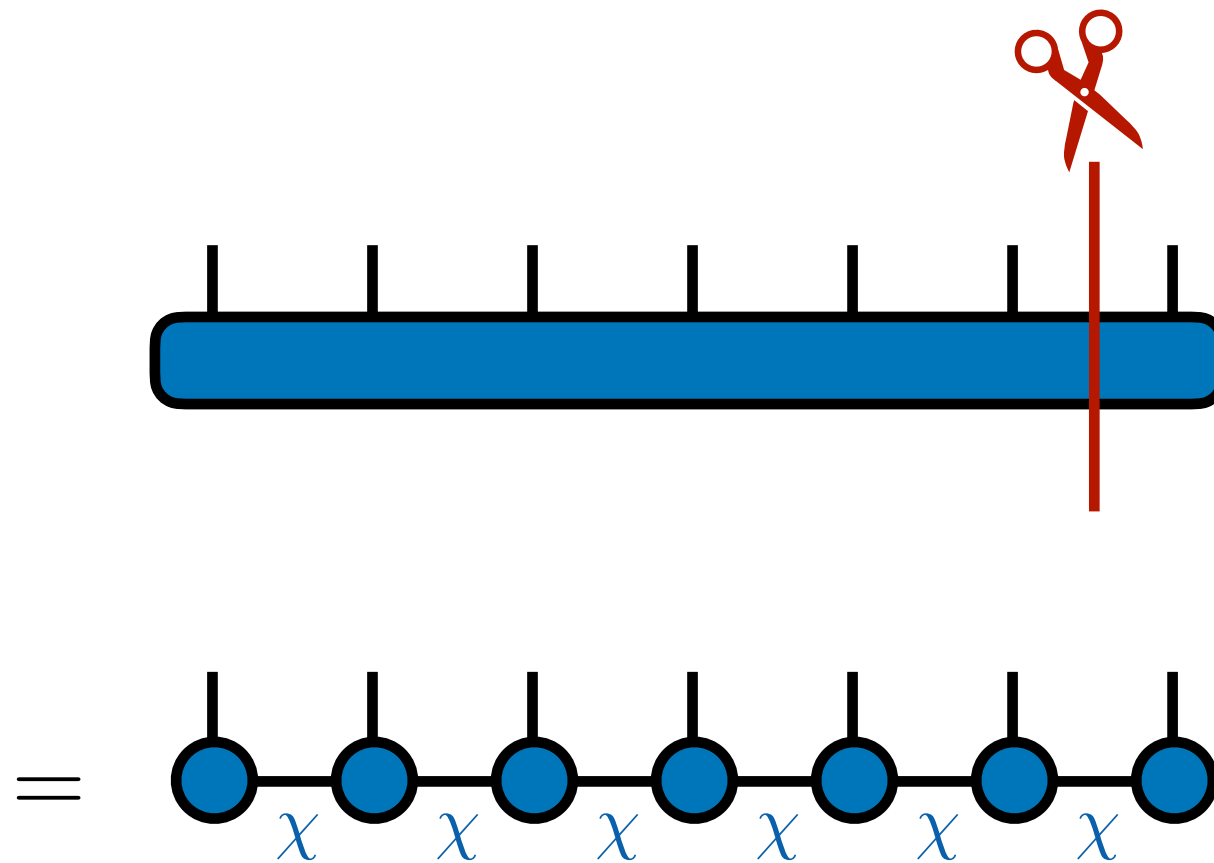
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



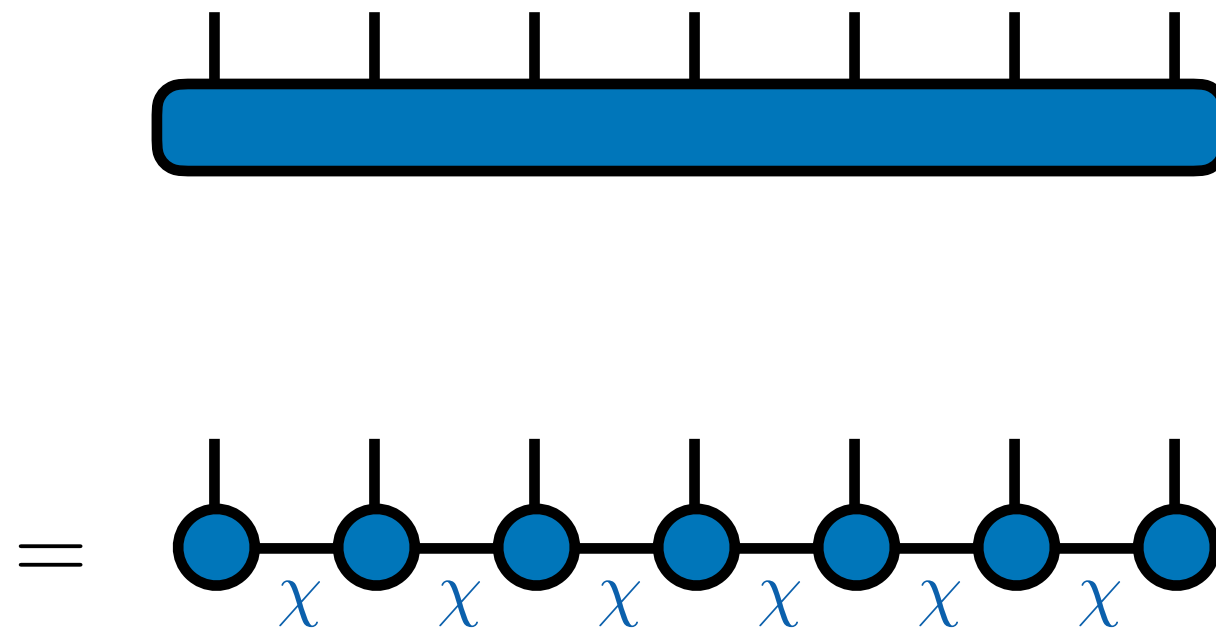
Classical Methods for Quantum Systems

Can recursively factor (compress) a tensor as well



Classical Methods for Quantum Systems

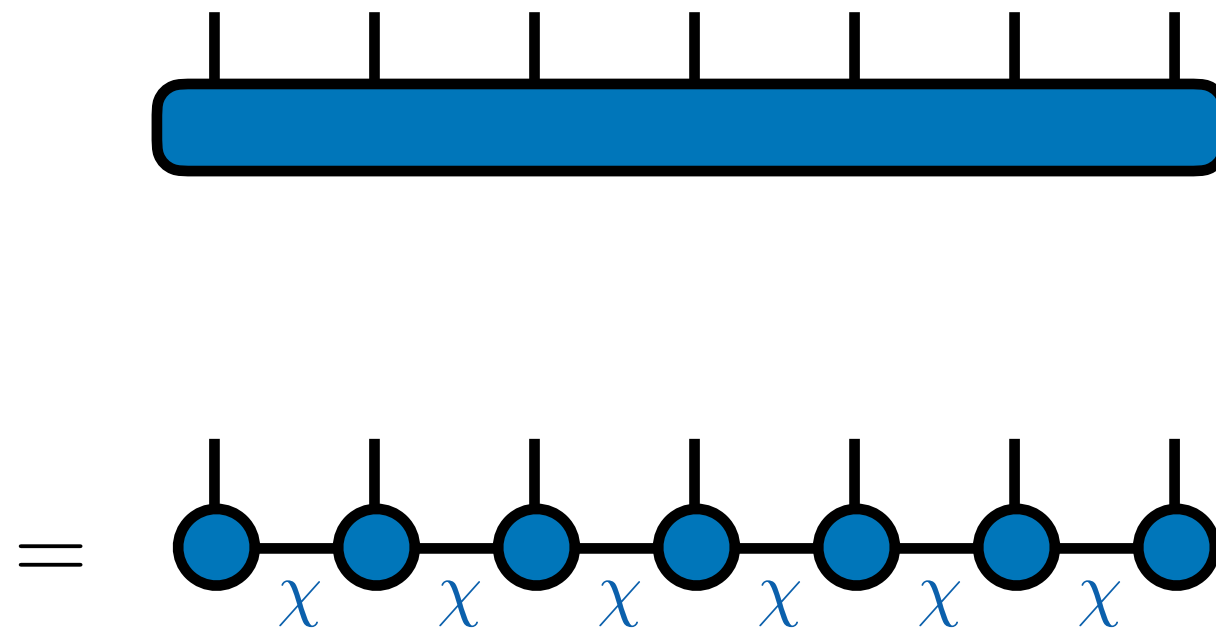
Result is a "matrix product state" (MPS)



If "ranks" χ small, save on **memory** and **compute**

Classical Methods for Quantum Systems

Result is a "matrix product state" (MPS)

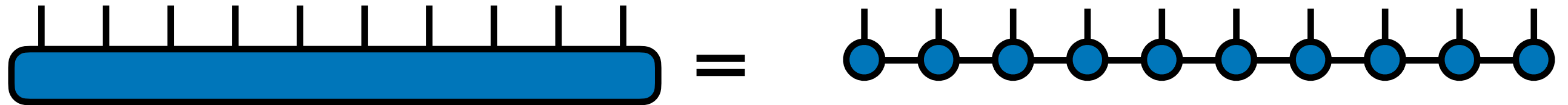


n tensors of dimension $\chi \cdot 2 \cdot \chi$

Total number of parameters: $2n\chi^2$

An *exponential* compression: $2^n \rightarrow 2n\chi^2$

Matrix product state (MPS) tensor network

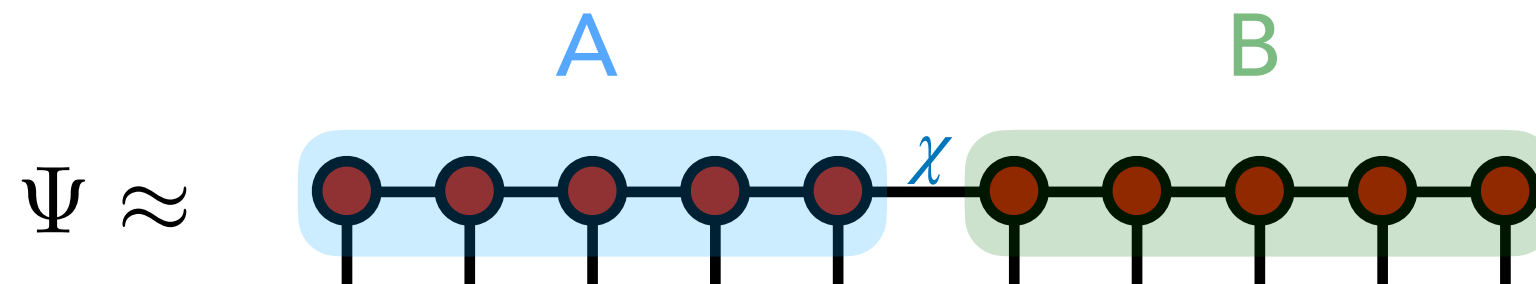


Can view as multi-SVD of a tensor



*Or special set of tensors
(a low-rank manifold)*

Entanglement of MPS



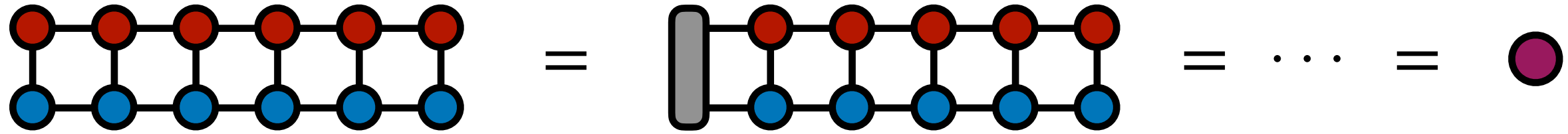
Partitioning MPS into left (A) and right (B)
gives rank χ matrix factorization

Schmidt decomposition with χ values

Entanglement bounded by $\log(\chi)$

MPS capture moderately-entangled states

Inner product of two MPS tensors



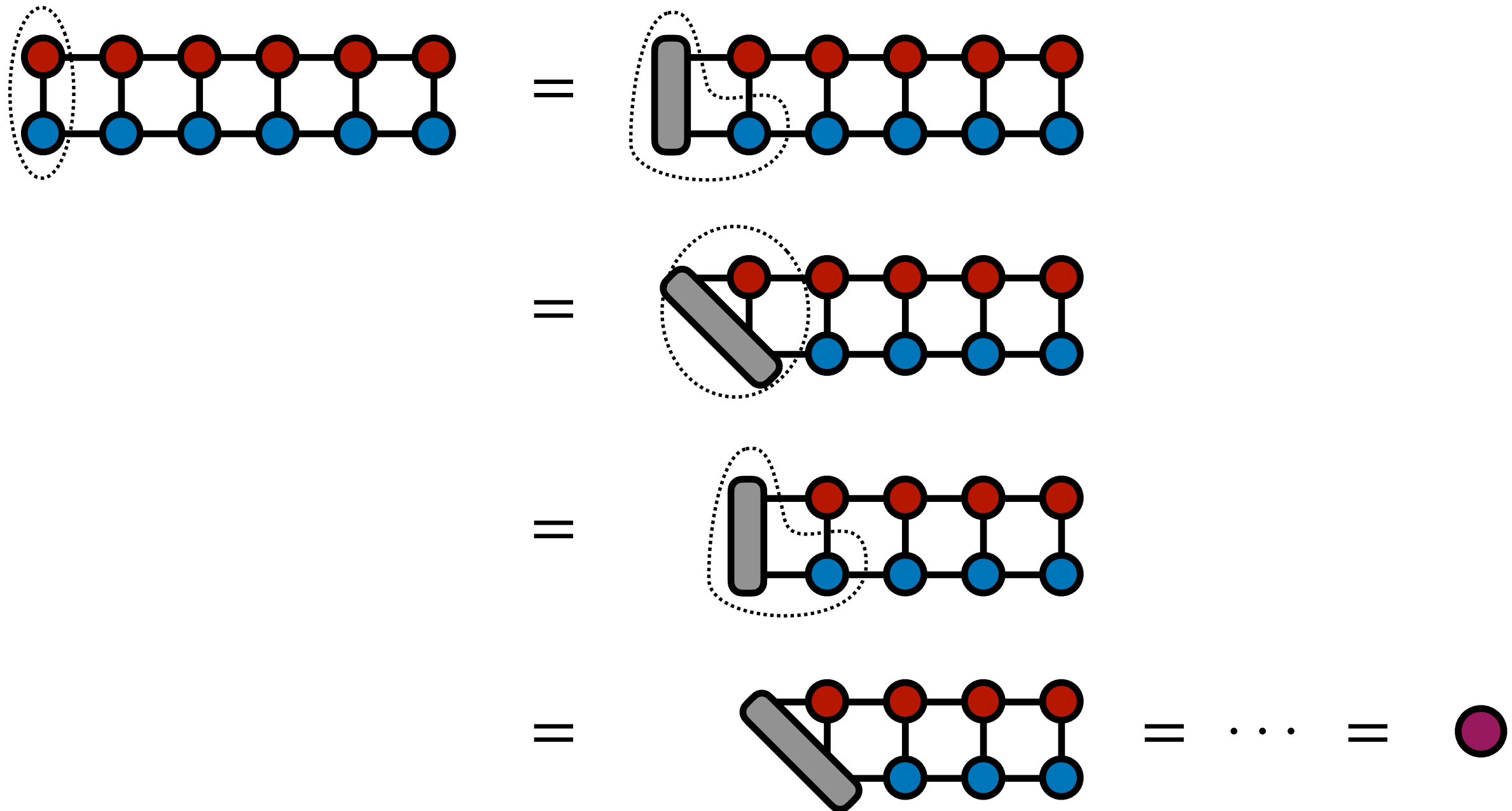
If each MPS represents tensor with 40 indices of dimension 2

Then above algorithm computes dot product of two 'vectors' of a *trillion* entries each

Takes $\sim 10\text{ms}$ for bond dimension $\chi = 100$

More detailed tensor network algorithm

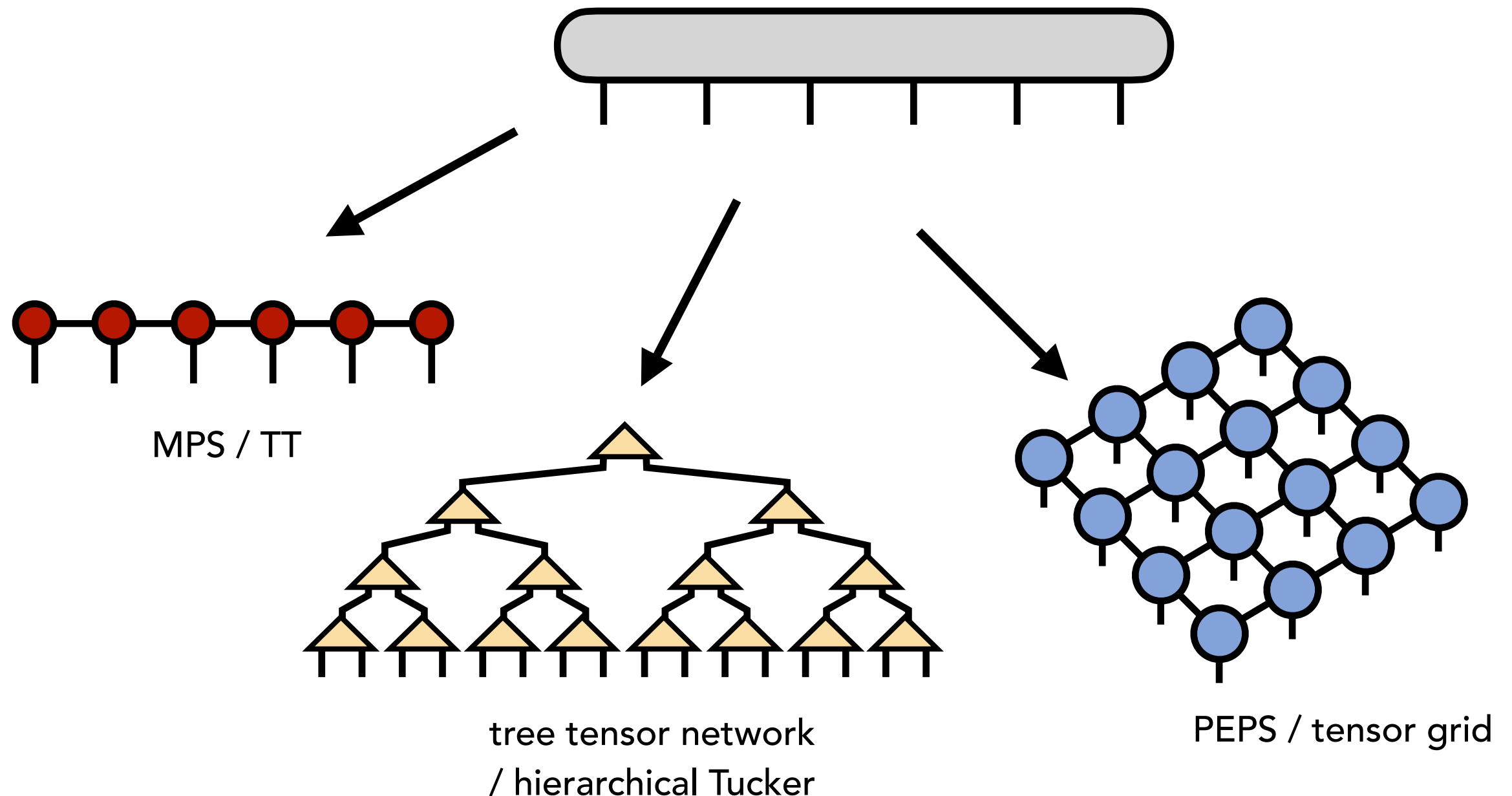
Inner product of two MPS tensors



Cost $\sim \chi^3$, memory usage $\sim \chi^2$

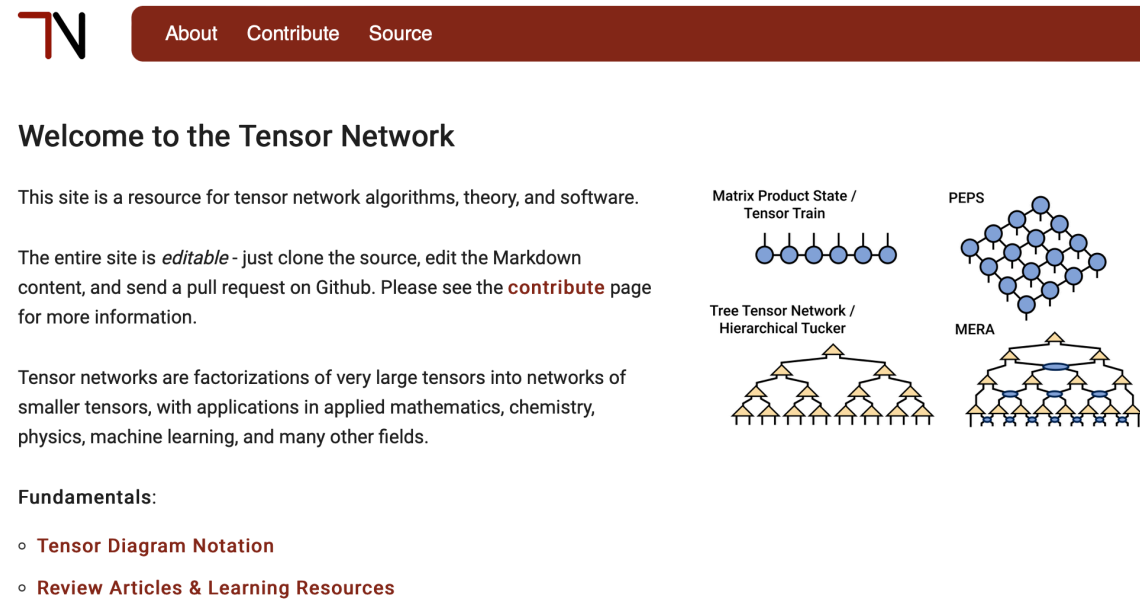
Tensor Networks

There are other tensor networks too,
with their own algorithms and degrees of expressive power

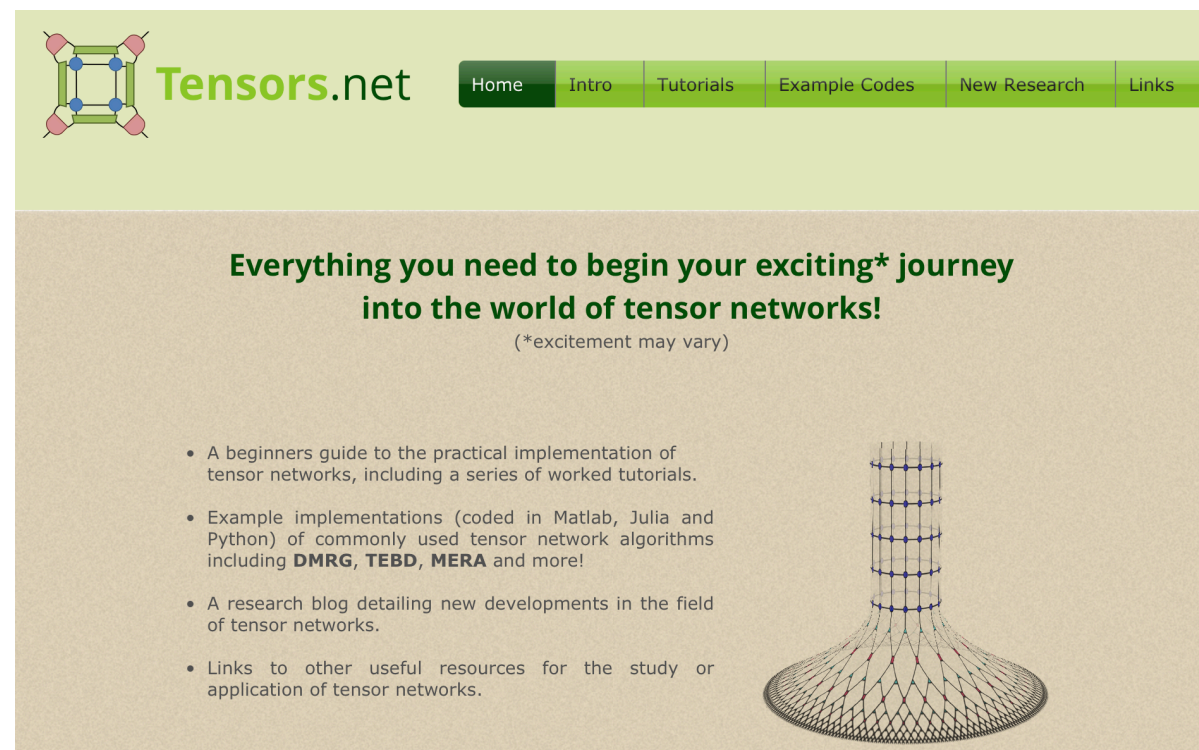


More Resources about Tensor Networks

- [TensorNetwork.org](https://www.tensornetwork.org): list of [review articles](#), video lectures, pages with details of algorithms



- [Tensors.net](https://tensors.net): tutorials and sample codes



Tensor Network Algorithms

Tensor Network Algorithms

Most important aspect of tensor networks are
the algorithms

Two examples

- algorithm for finding ground states ("DMRG")^[1,2]
- algorithm for applying gates ("TEBD")^[3]

[1] White, PRL 69, 2863 (1992)

[2] Schollwöck, Annals of Phys. 326, 96 (2011)

[3] Vidal, PRL 91, 147902 (2003)

Tensor Network Algorithms

Seminal tensor network algorithm is DMRG
(density matrix renormalization group)

Given a Hamiltonian H

Finds ground state and its energy

$$\min_{\psi} \frac{\langle \psi | H | \psi \rangle}{\langle \psi | \psi \rangle} = E_0$$

Tensor Network Algorithms

DMRG algorithm

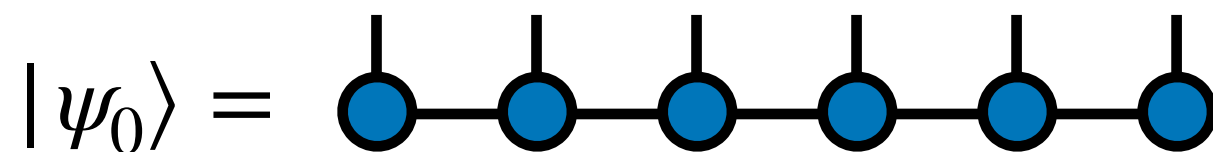
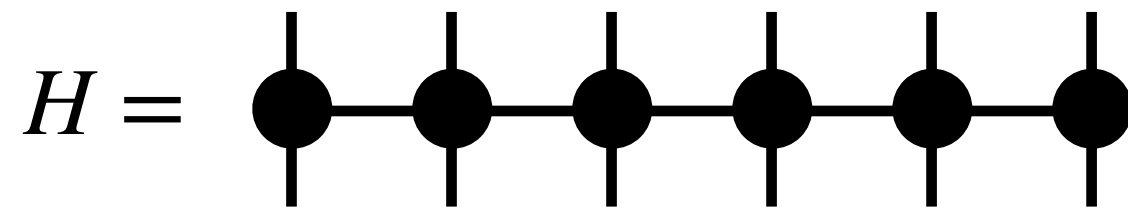
Assume we can write H as a tensor network

$$H = \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---}$$

Tensor Network Algorithms

DMRG algorithm

DMRG finds its ground state as an MPS tensor network



Tensor Network Algorithms

DMRG algorithm

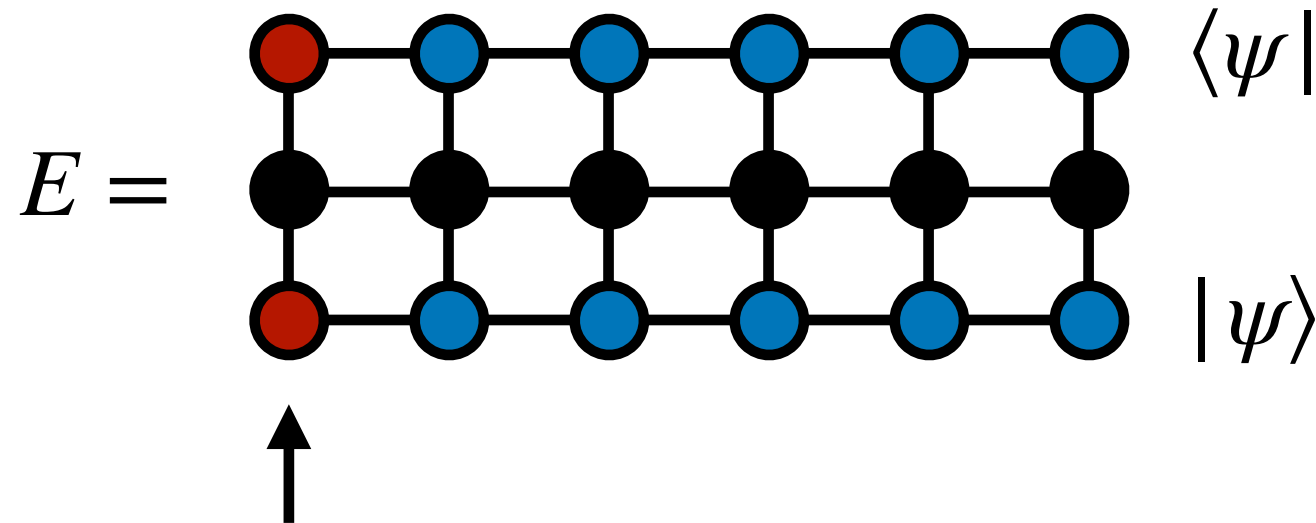
Energy is

$$E = \begin{array}{cccccc} \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & \langle \psi | \\ | & | & | & | & | & | & \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & \\ | & | & | & | & | & | & \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & | \psi \rangle \end{array}$$

Tensor Network Algorithms

DMRG algorithm

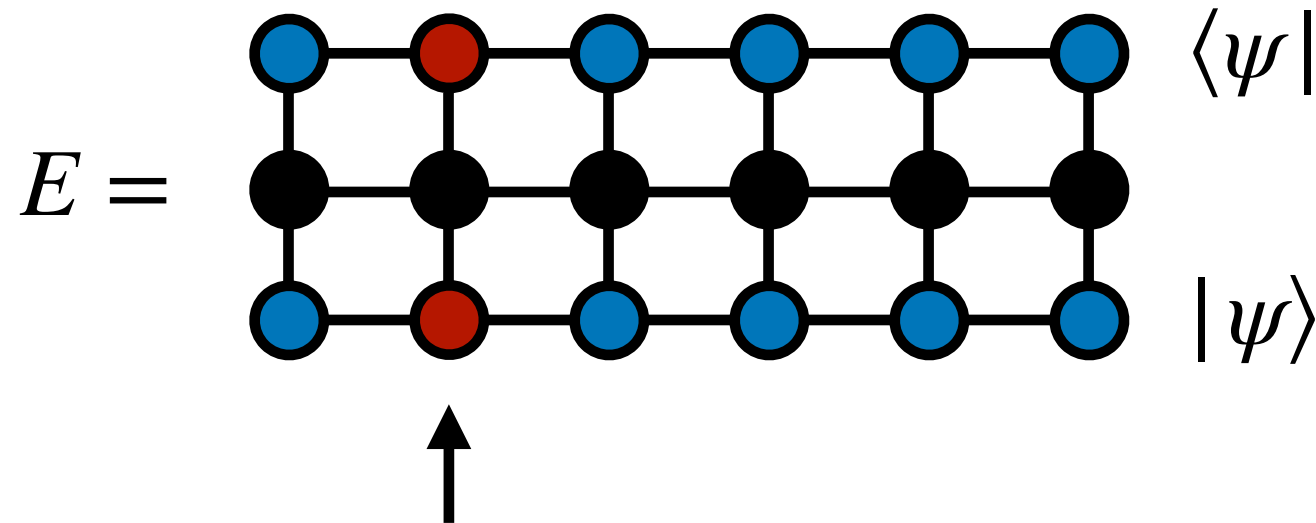
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

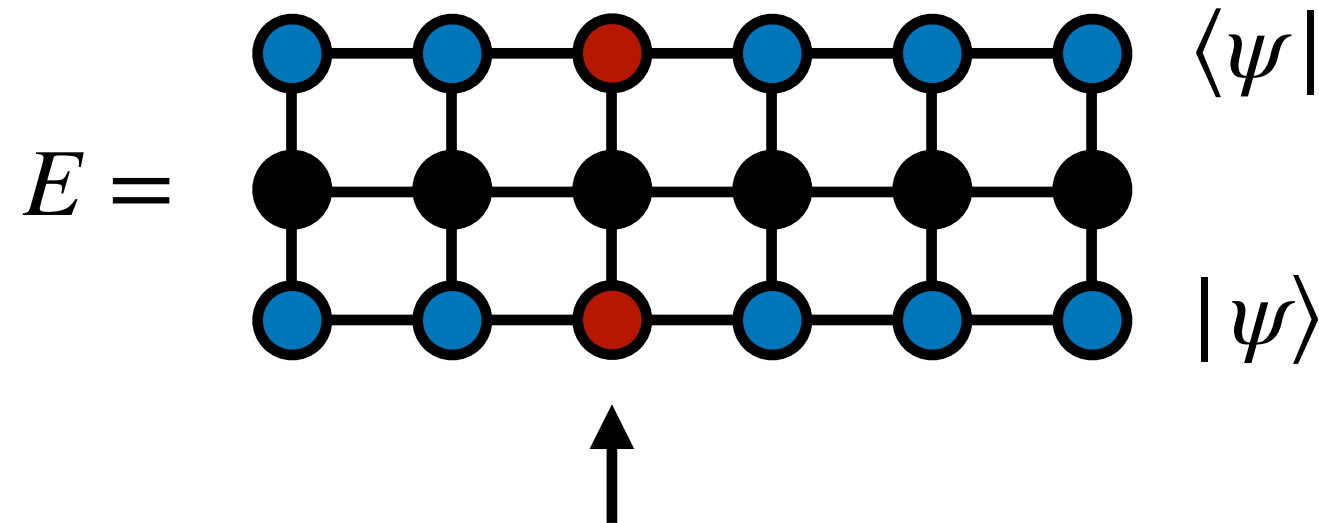
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

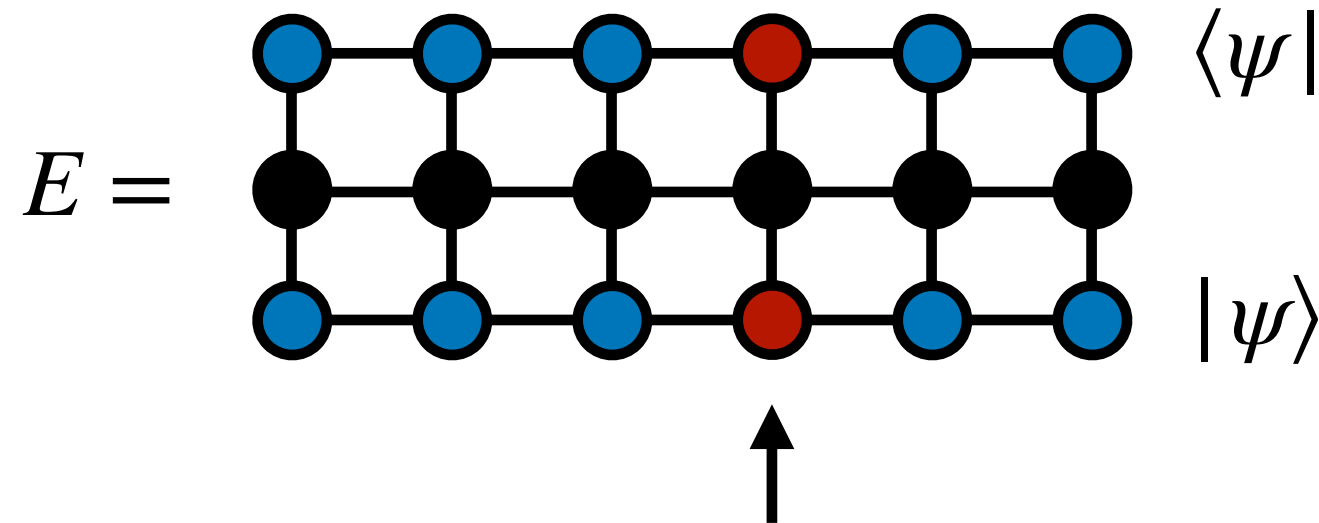
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

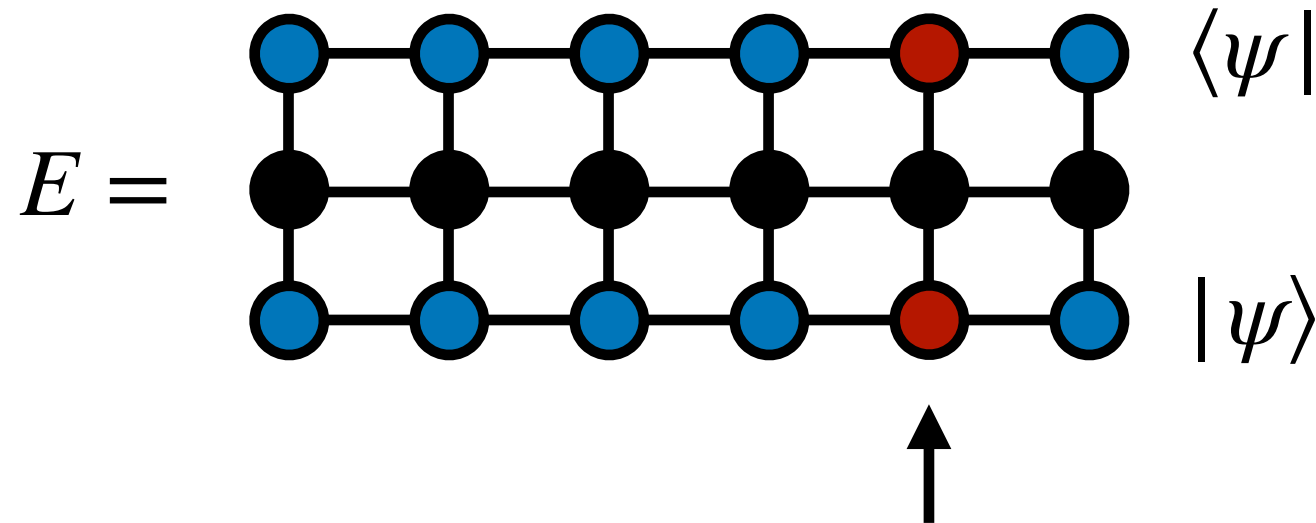
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

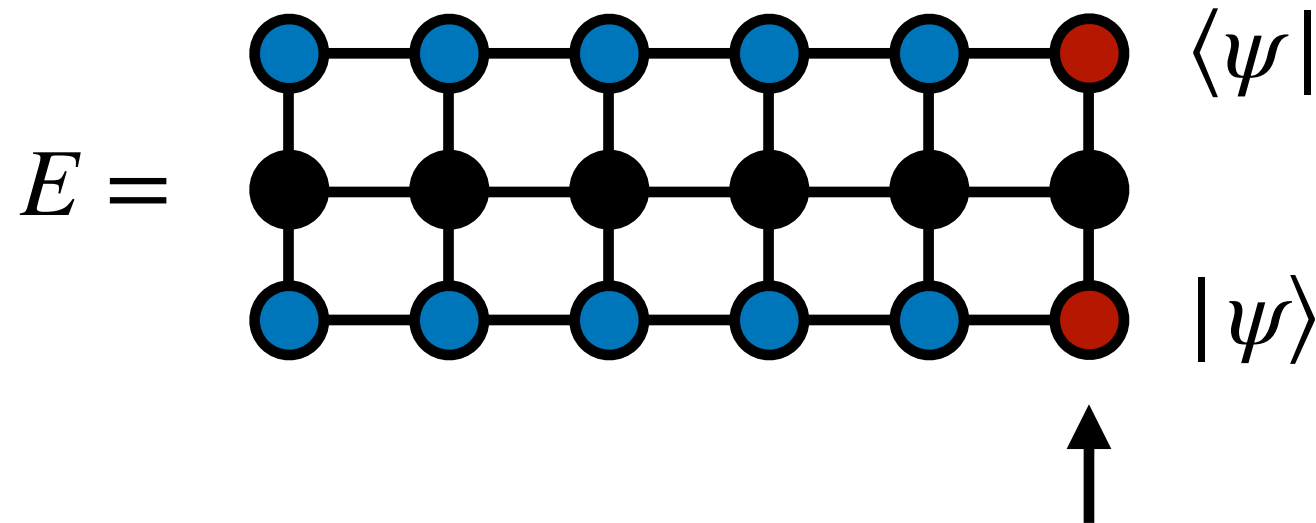
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

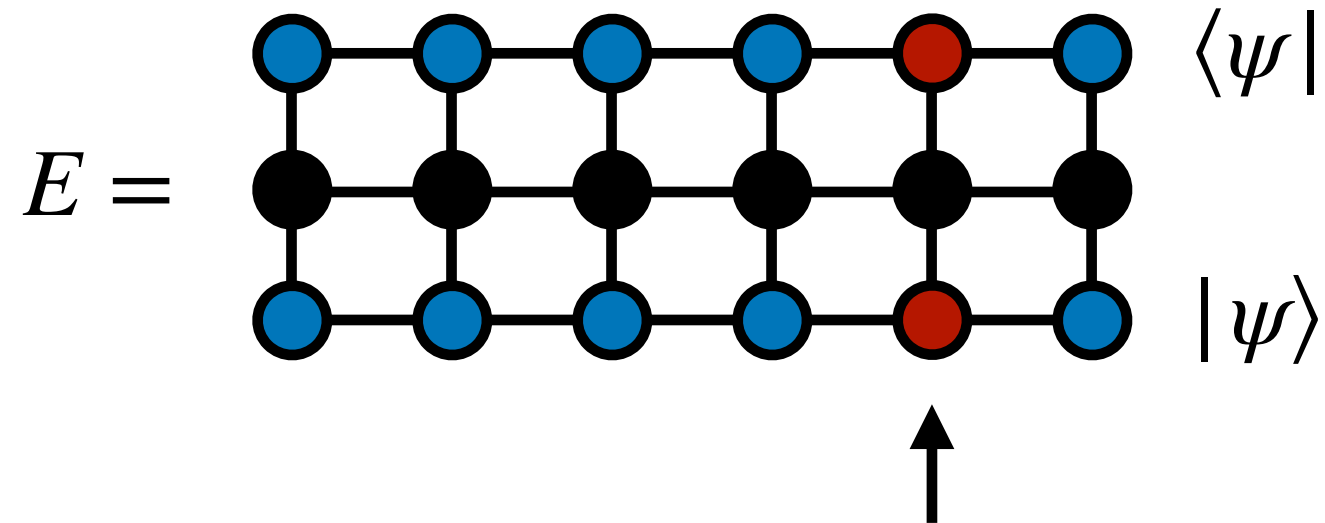
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

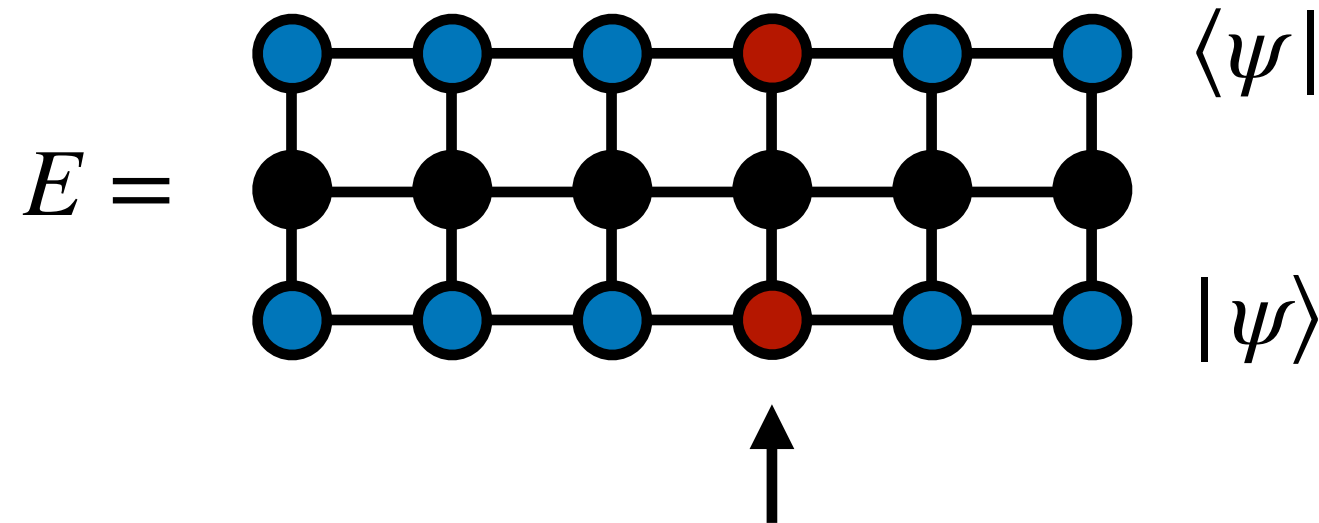
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

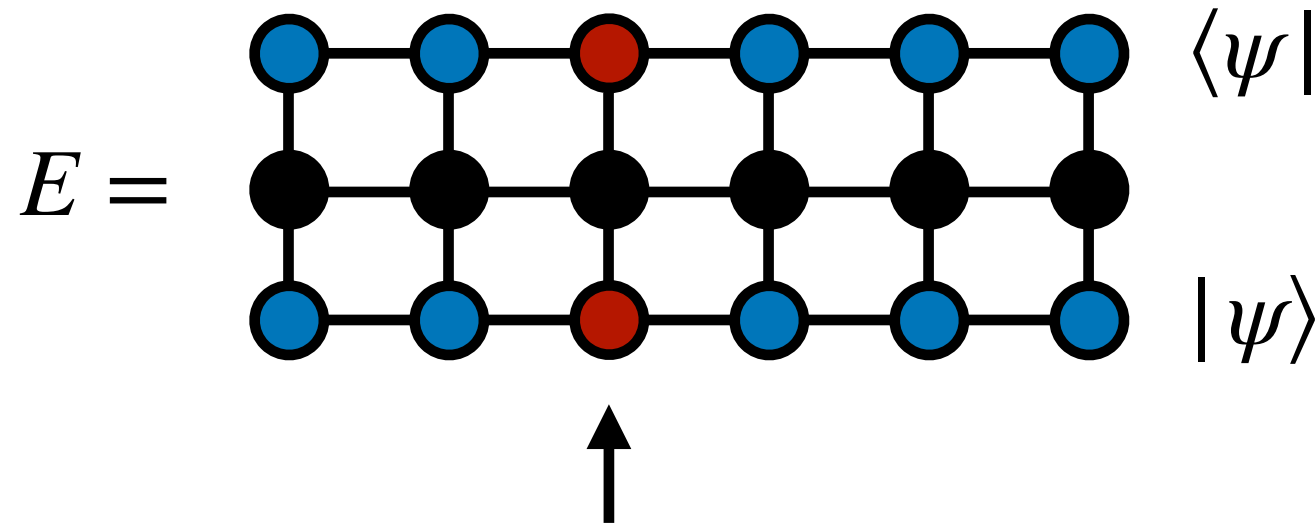
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

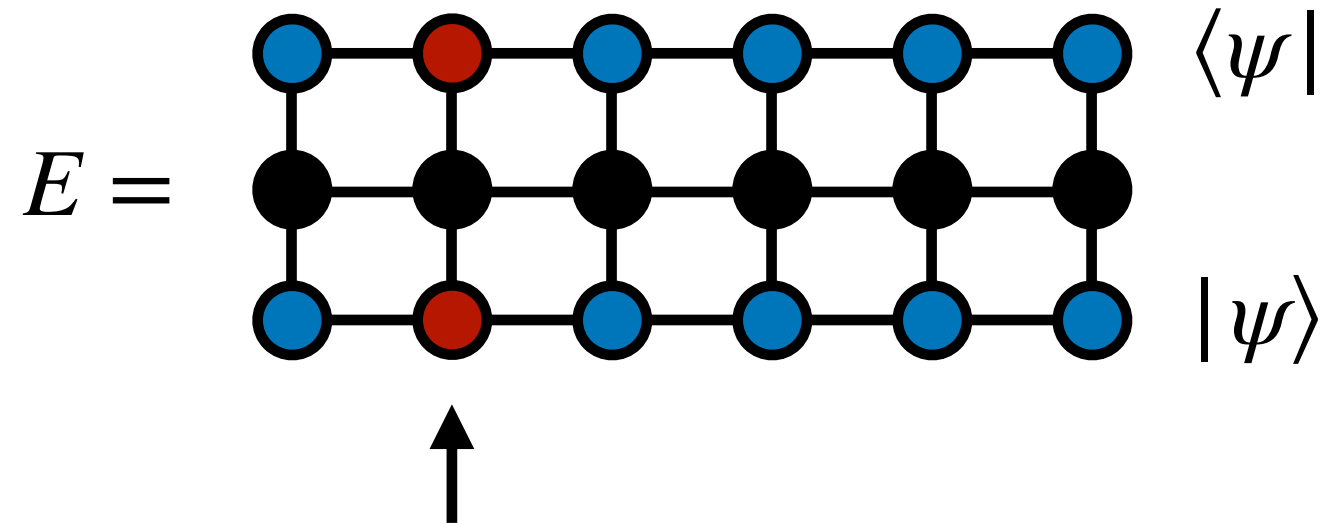
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

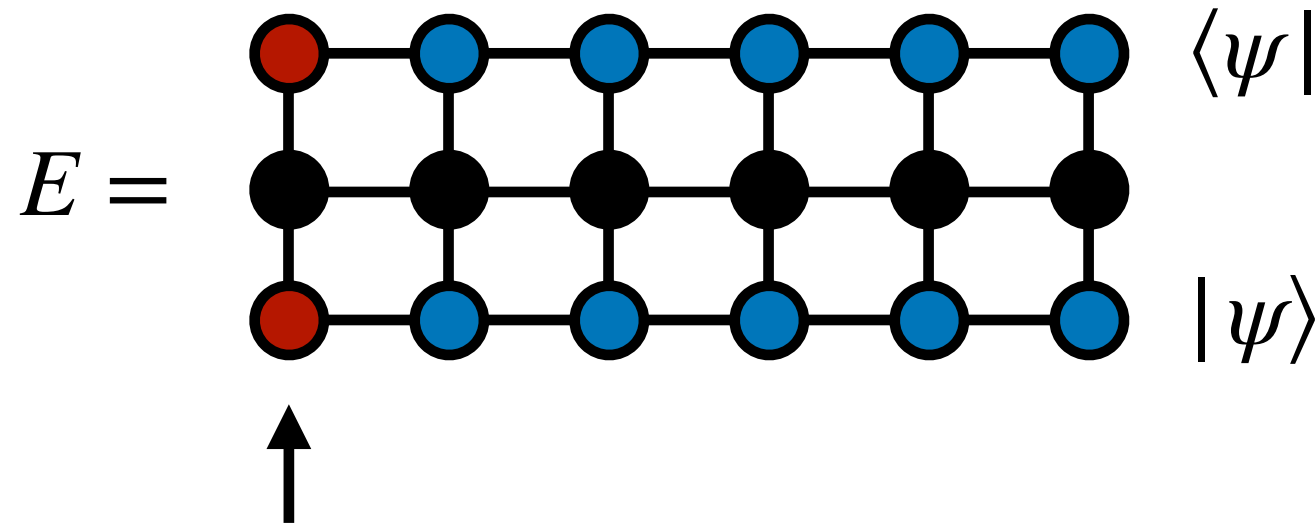
DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

DMRG uses an "alternating" strategy to optimize



Tensor Network Algorithms

DMRG algorithm

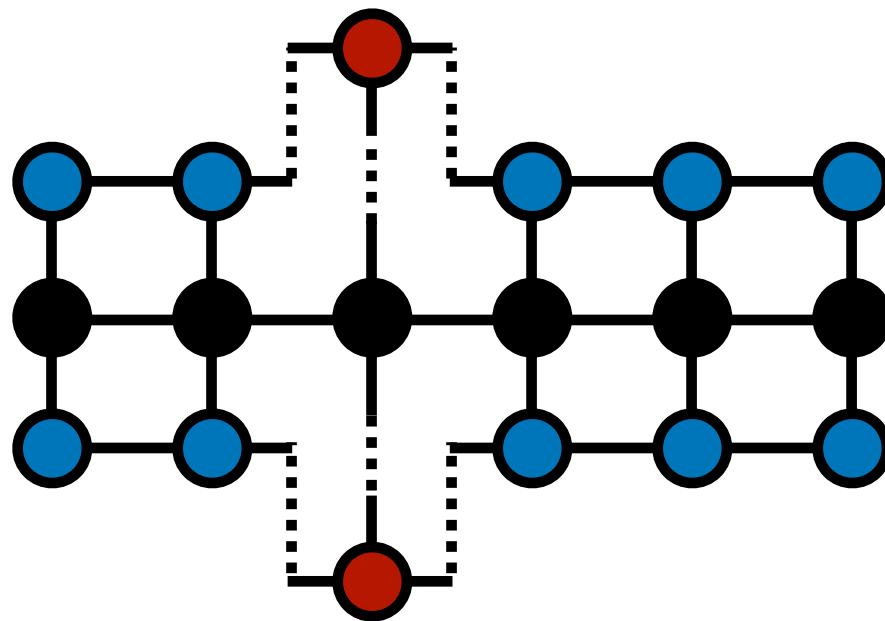
At each step, solve a "mini" diagonalization problem

$$E = \begin{array}{cccccc} \text{blue} & \text{blue} & \text{red} & \text{blue} & \text{blue} & \text{blue} & \langle \psi | \\ \text{black} & \text{black} & \text{black} & \text{black} & \text{black} & \text{black} & \\ \text{blue} & \text{blue} & \text{red} & \text{blue} & \text{blue} & \text{blue} & | \psi \rangle \end{array}$$

Tensor Network Algorithms

DMRG algorithm

At each step, solve a "mini" diagonalization problem*

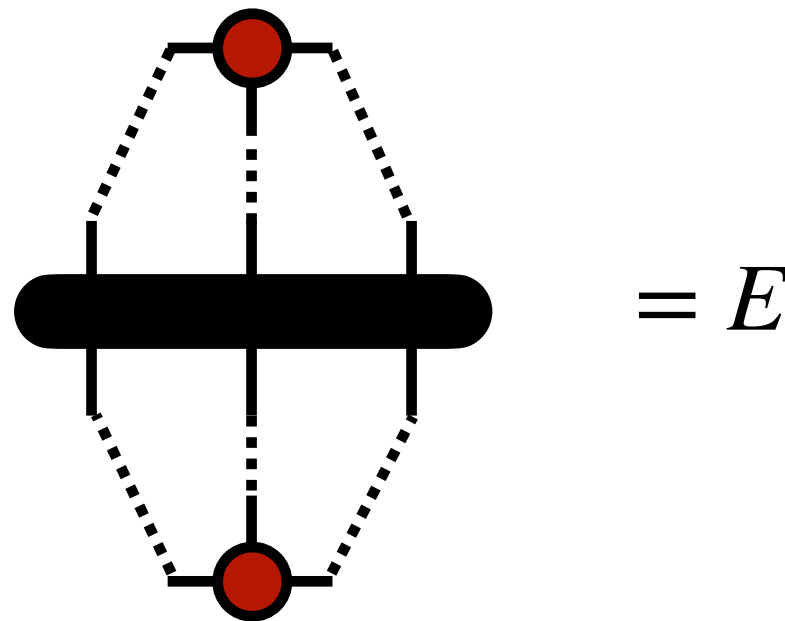


*technical note: for efficiency, frozen tensors are contracted in three groups, not exactly as shown above

Tensor Network Algorithms

DMRG algorithm

At each step, solve a "mini" diagonalization problem*

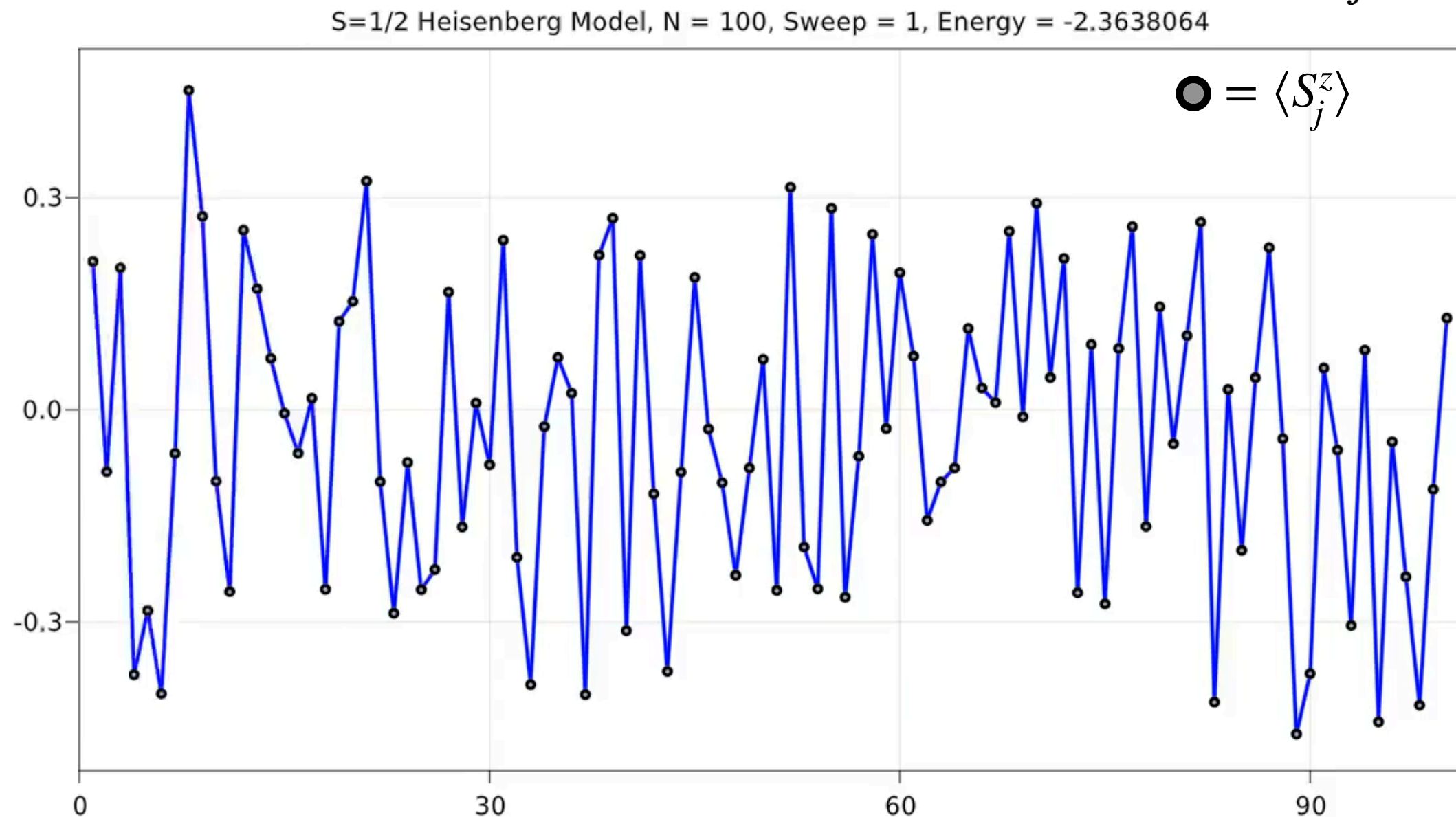


*technical note: for efficiency, frozen tensors are contracted in three groups, not exactly as shown above

Tensor Network Algorithms

DMRG ground-state finding algorithm

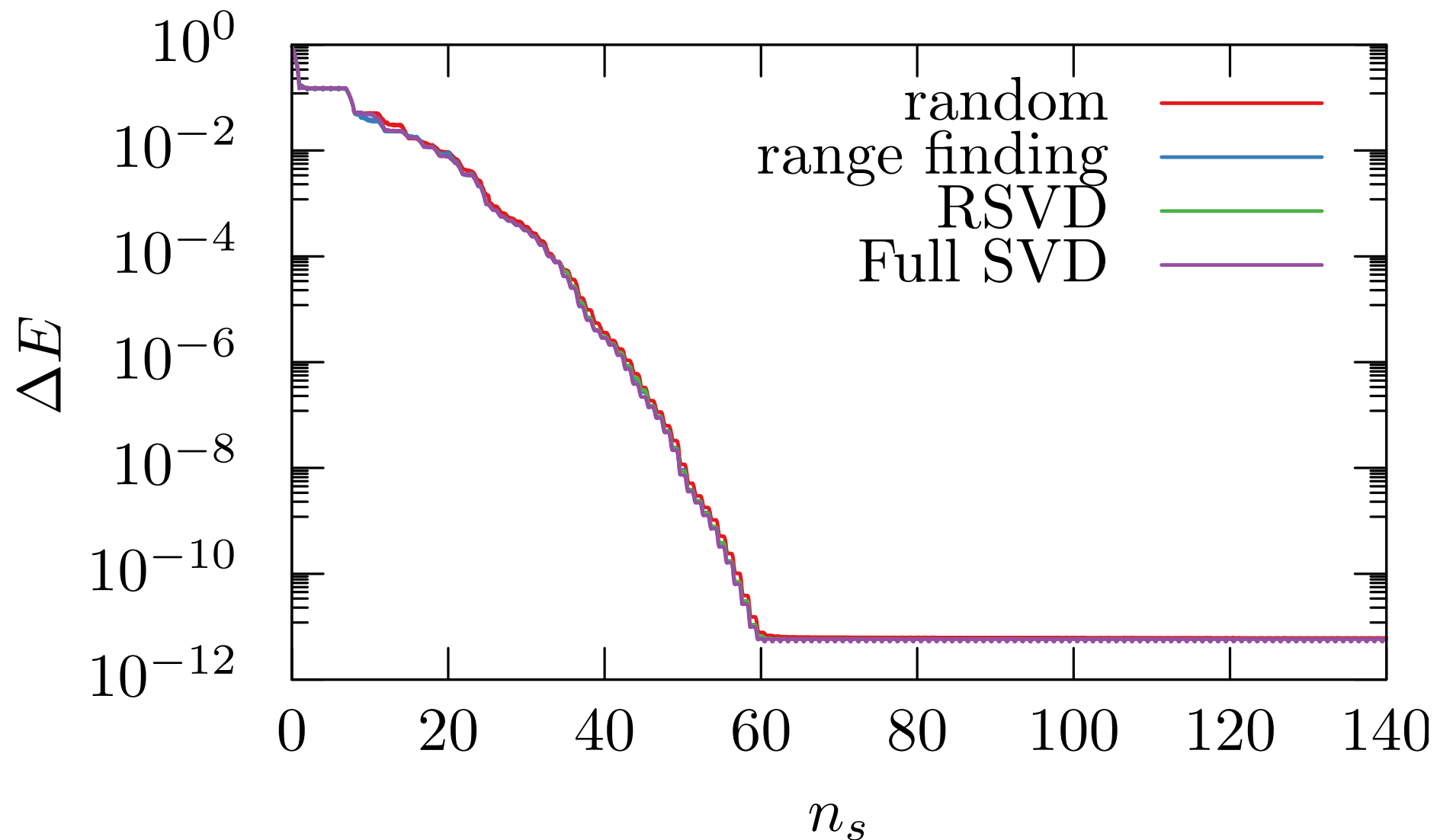
DMRG in action – solving Heisenberg chain $H = \sum_j \vec{S}_j \cdot \vec{S}_{j+1}$



Tensor Network Algorithms

DMRG algorithm is extremely precise

Energy error vs number of 'sweeps' of DMRG



Tensor Network Algorithms

Another important capability is
time evolving tensor networks

$$e^{-iHt}|\Psi(0)\rangle = |\Psi(t)\rangle$$

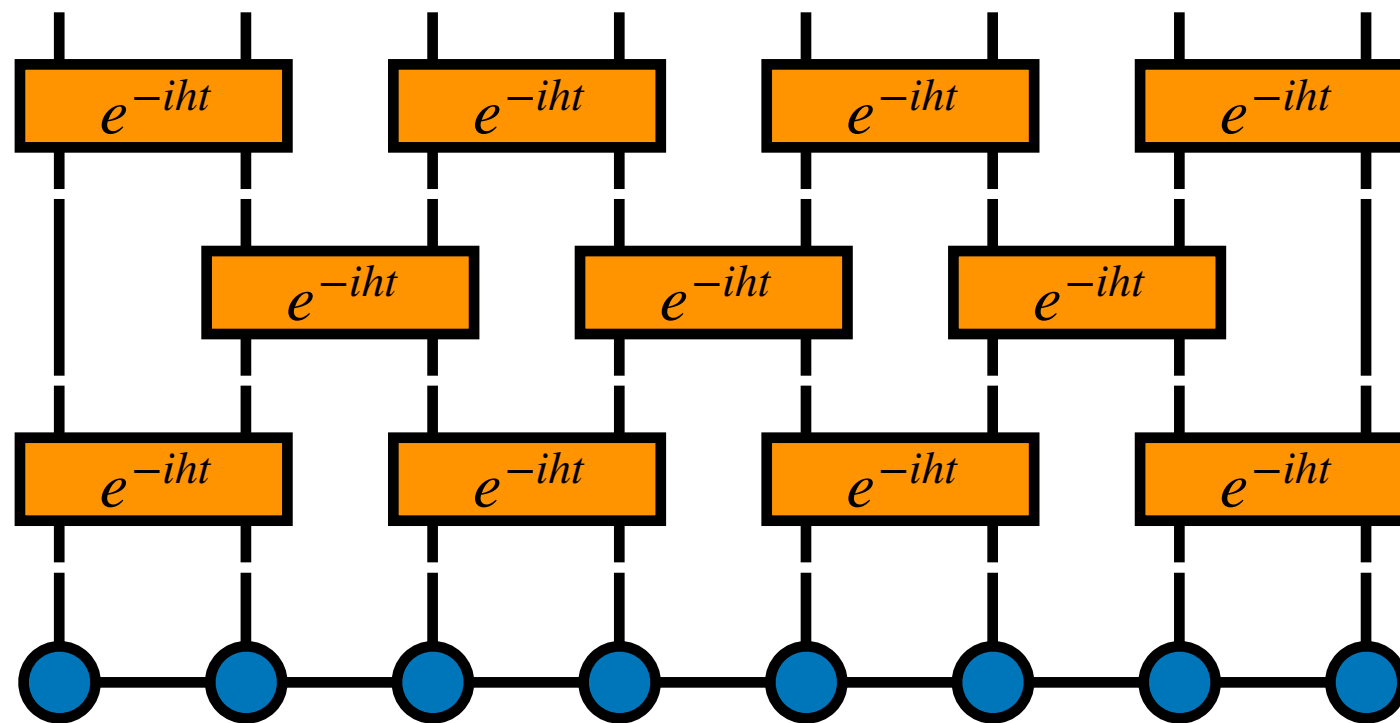
Algorithms:

- Time evolving block decimation (TEBD)
- Time-dependent variational principle (TDVP)

Tensor Network Algorithms

TEBD algorithm

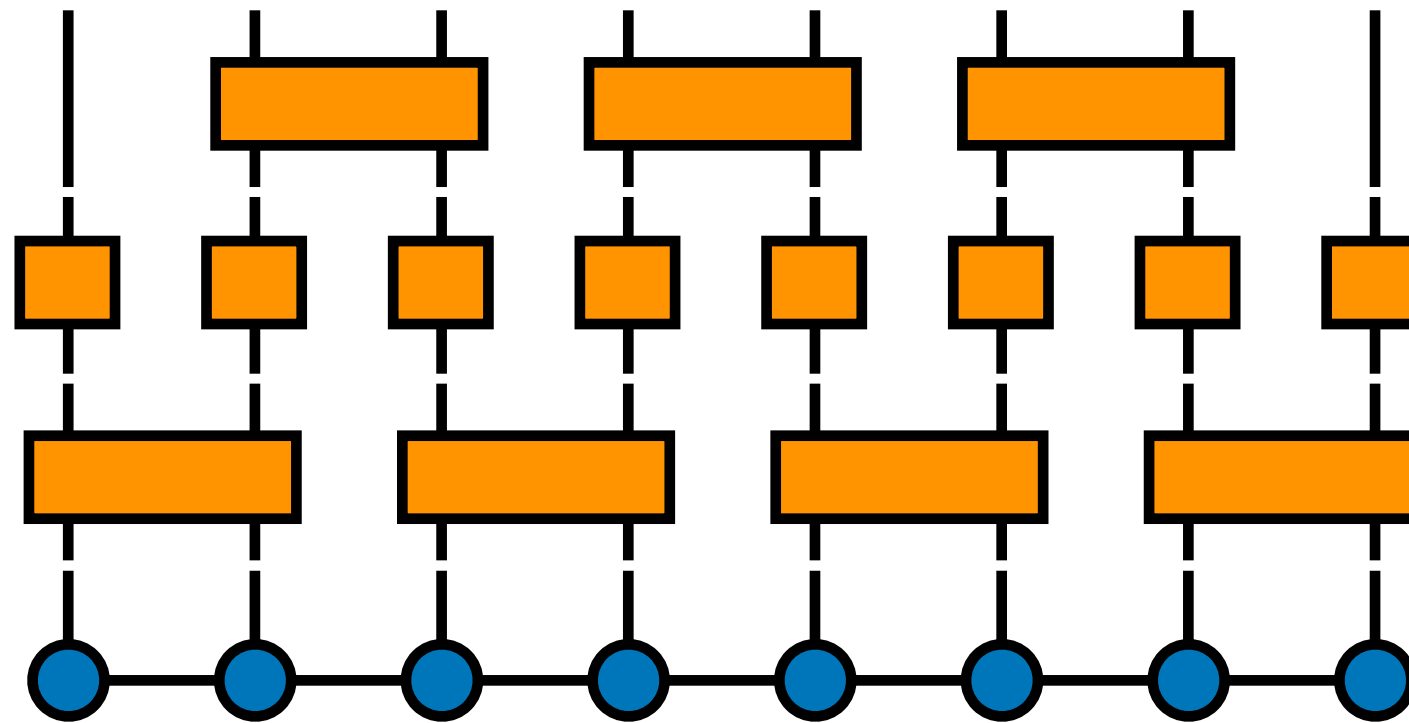
Can time evolve by decomposing e^{-iHt} into Trotter gates



Tensor Network Algorithms

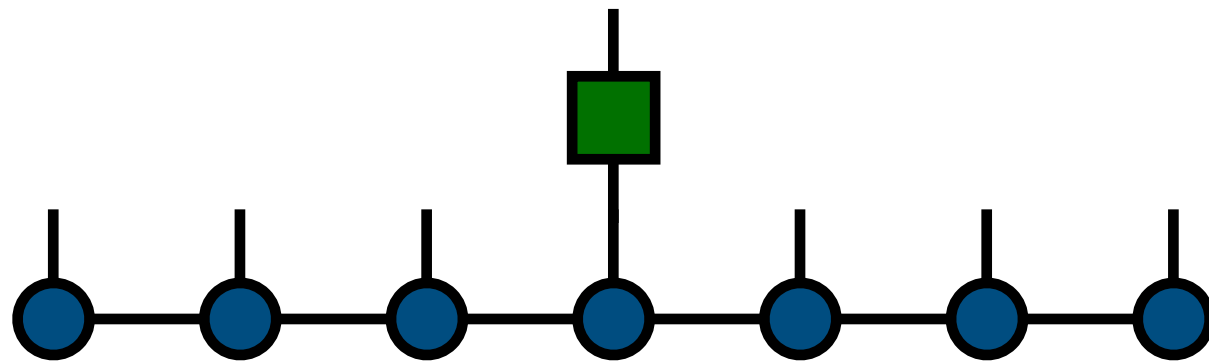
TEBD algorithm

Can also be used to simulate quantum circuits

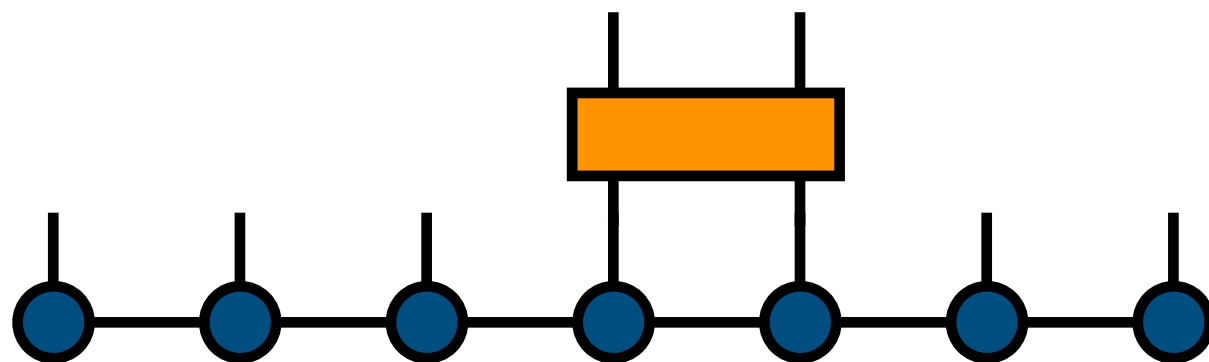


Tensor Network Algorithms

TEBD = controlled application of one-qubit and two-qubit gates to an MPS



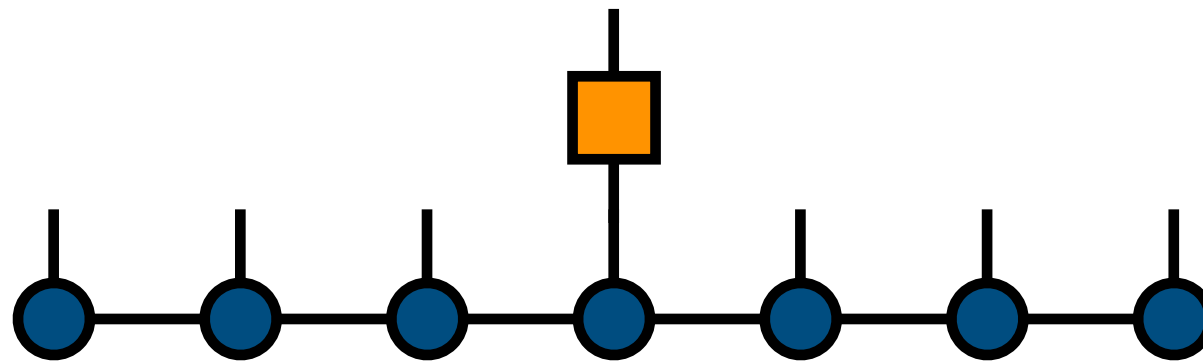
one-qubit gate is exact



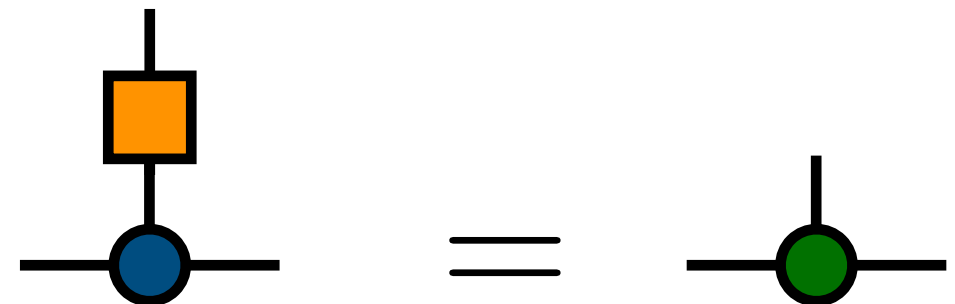
two-qubit gate incurs small error controlled by bond-dimension χ

Tensor Network Algorithms

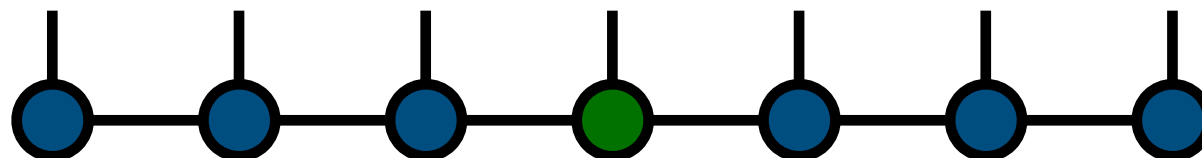
Say we want to act a single-qubit gate on a wavefunction in MPS form



Just operate on one tensor:

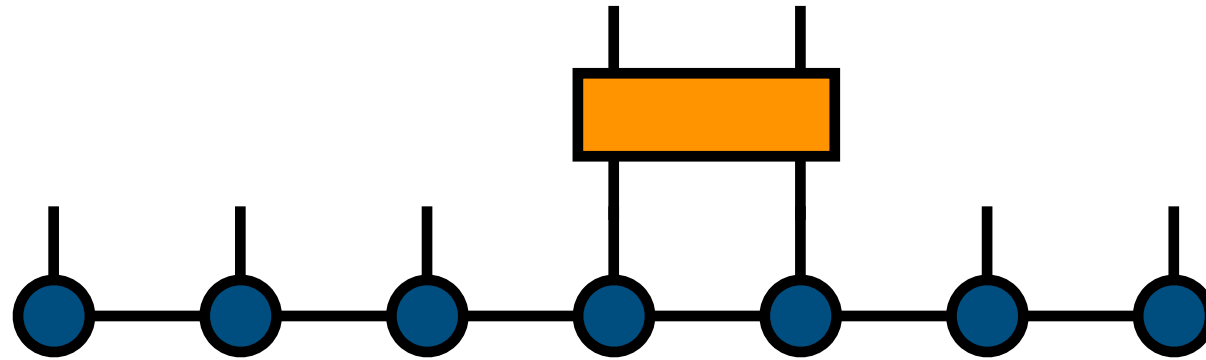


Result:

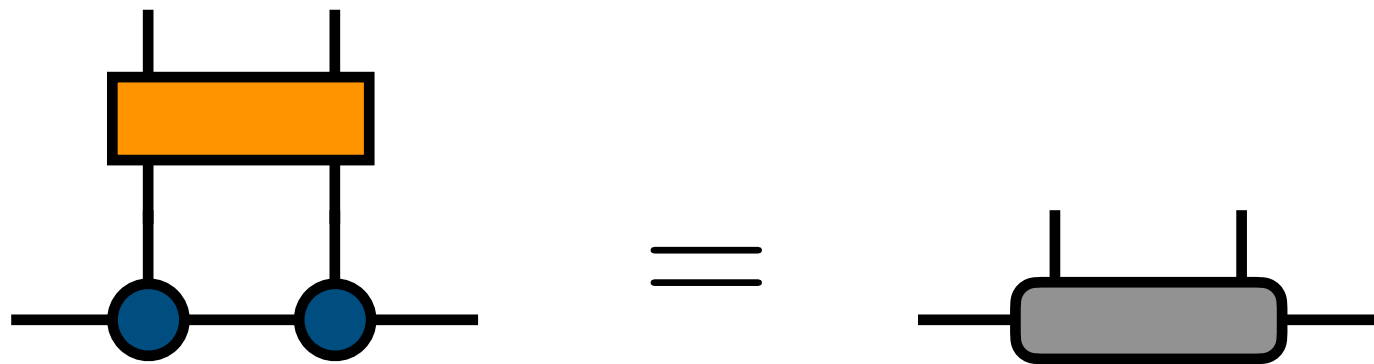


*same bond
dimension*

Say we want to act two-qubit gate on a wavefunction in MPS form

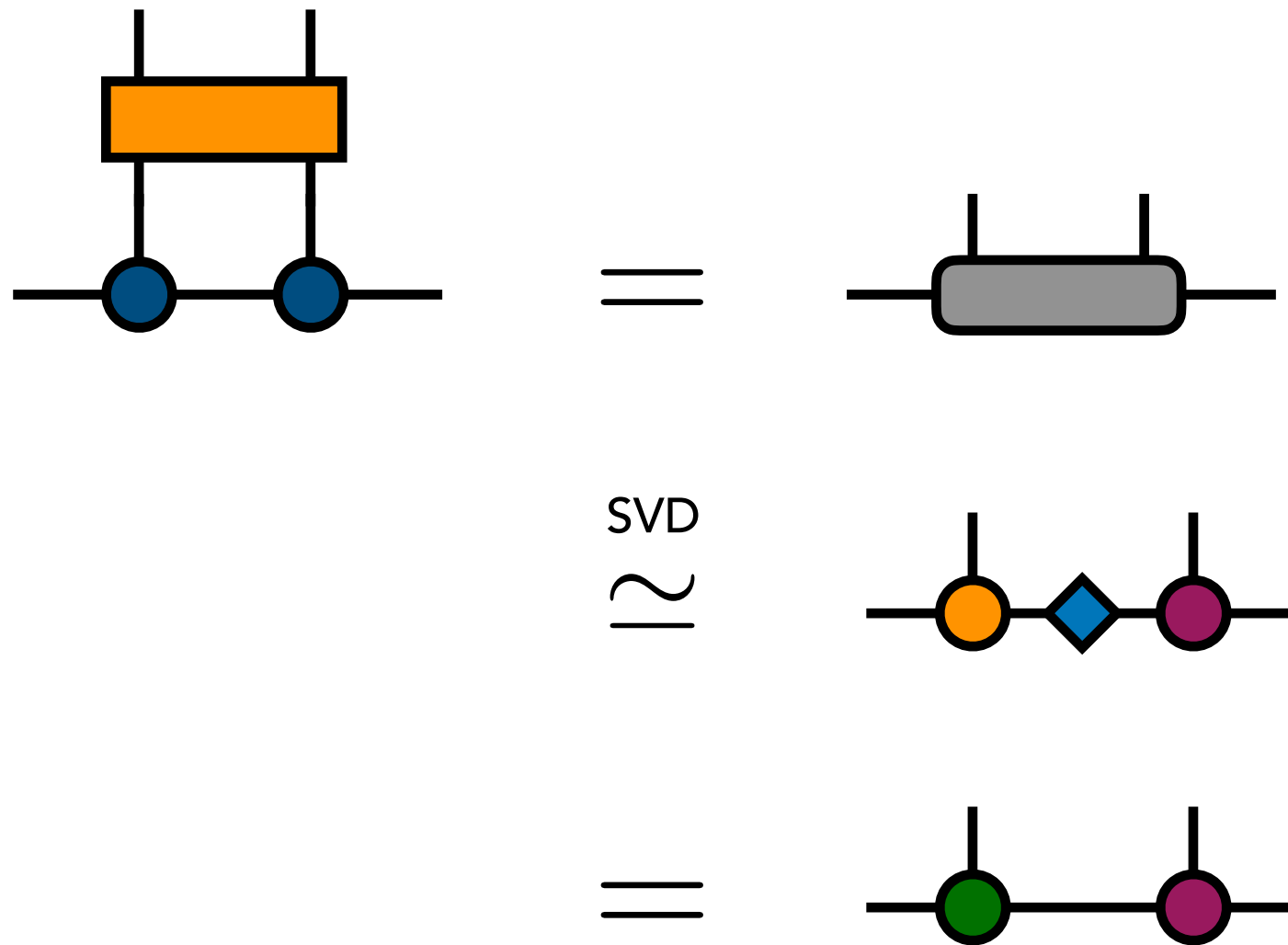


Operate on two tensors:



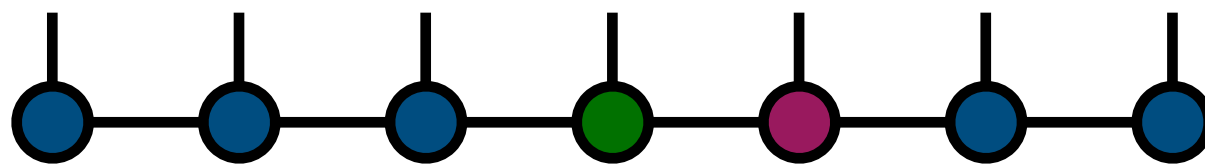
*destroy
MPS form
locally*

But can recover MPS form using truncated SVD:



*keep top
 χ values*

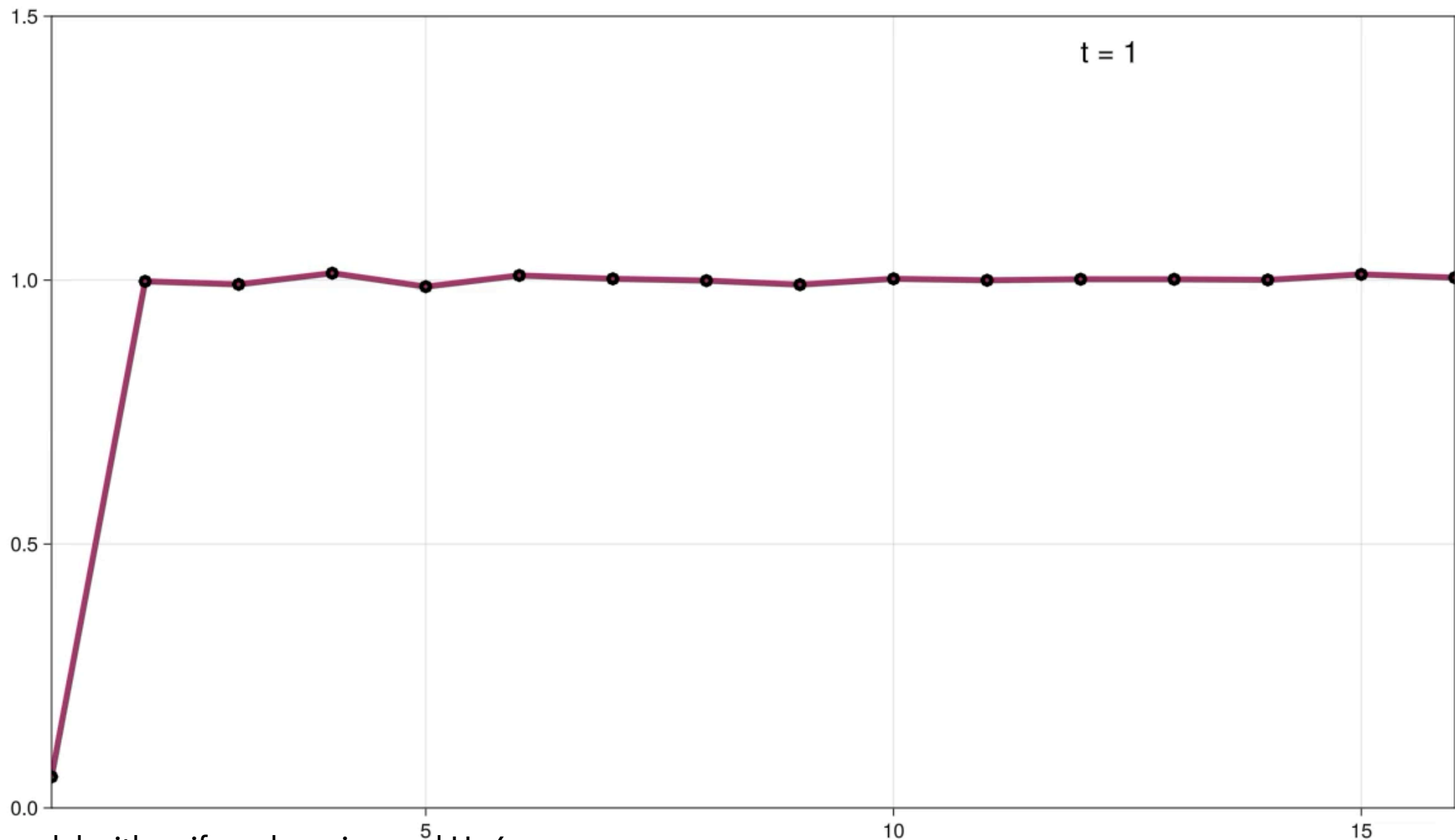
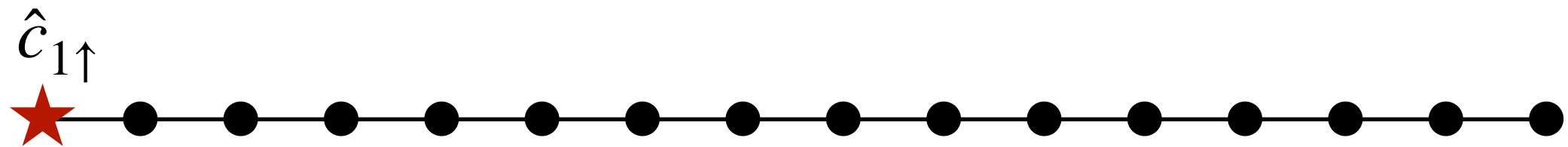
Result:



*with small
loss of
fidelity*

Tensor Network Algorithms

TEBD results: electron chain* after removing particle on site 1



*Impurity model with uniform hopping and $U=6$

Tensor Network Algorithms

Limitations of tensor network algorithms:

- DMRG can sometimes fail to find ground state (rare)
- Entanglement of states generically **grows** in time
- Entanglement capacity is limited by type of tensor networks (MPS = 1D and quasi-1D systems)

Summary

Tensor networks can give **exponentially good** compression of quantum states and other large tensors

Matrix product state is iterated SVD of a tensor

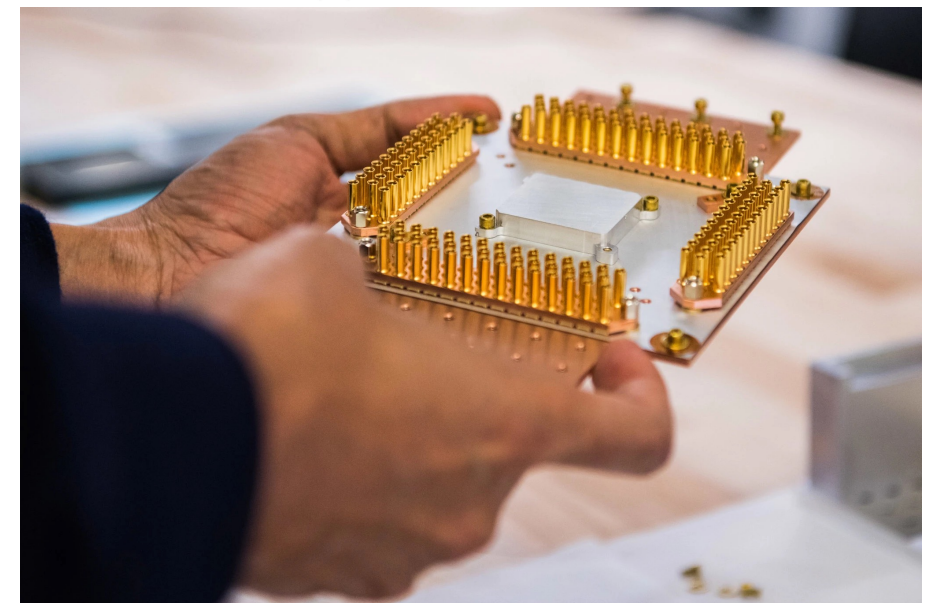
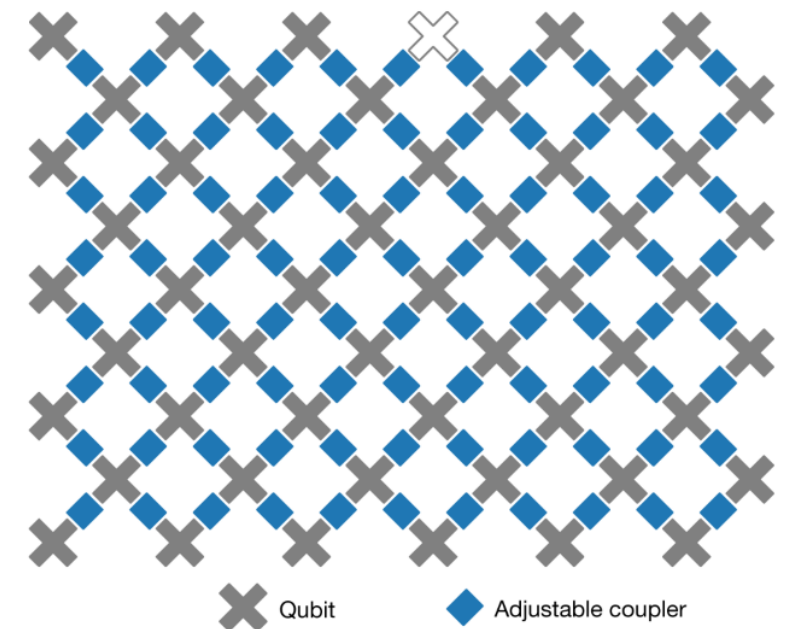
Powerful algorithms for tensor networks

- **DMRG** for ground state finding
- **TEBD** for circuit and time evolution

Quantum Computing as a Tensor Network

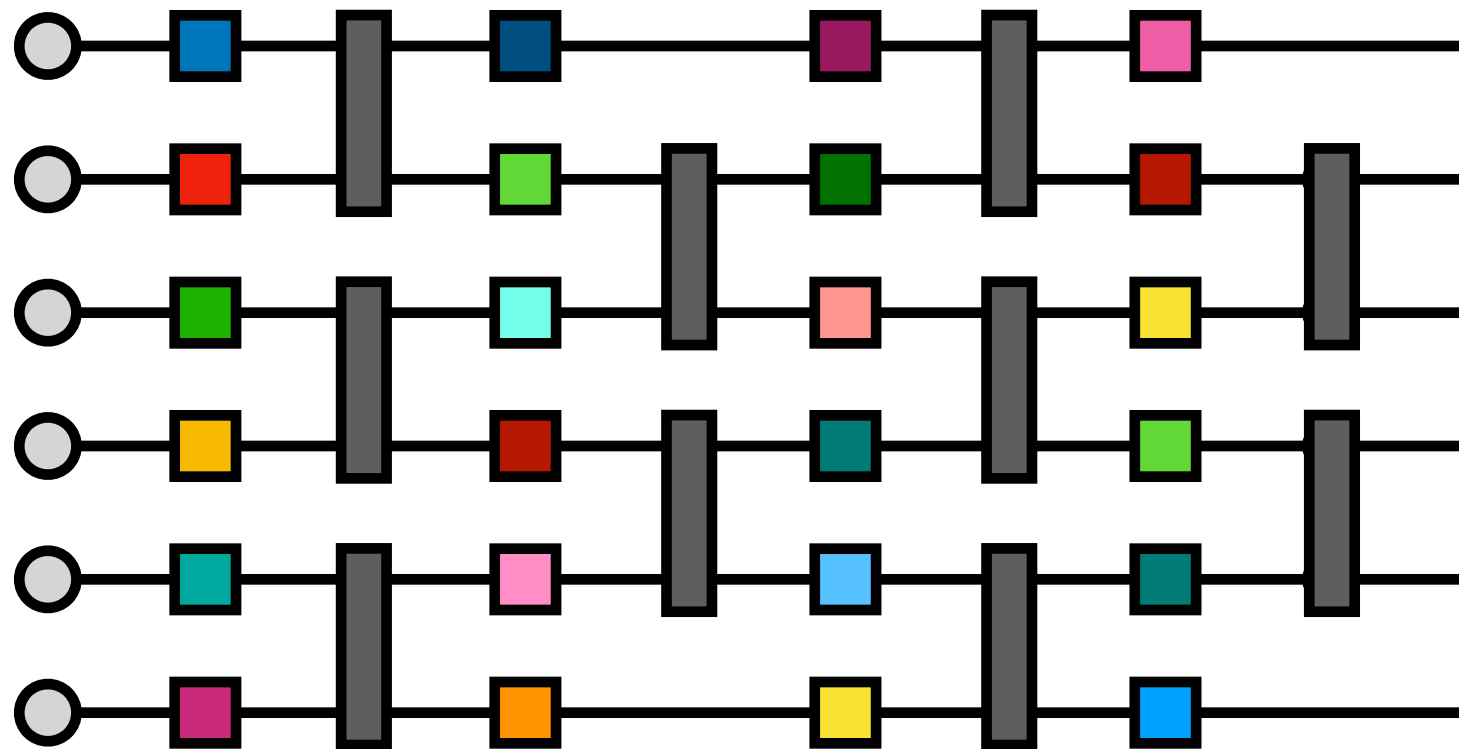
A **quantum computer** is a machine of *quantum bits*

Carries out instructions, exploring an exponentially large state space



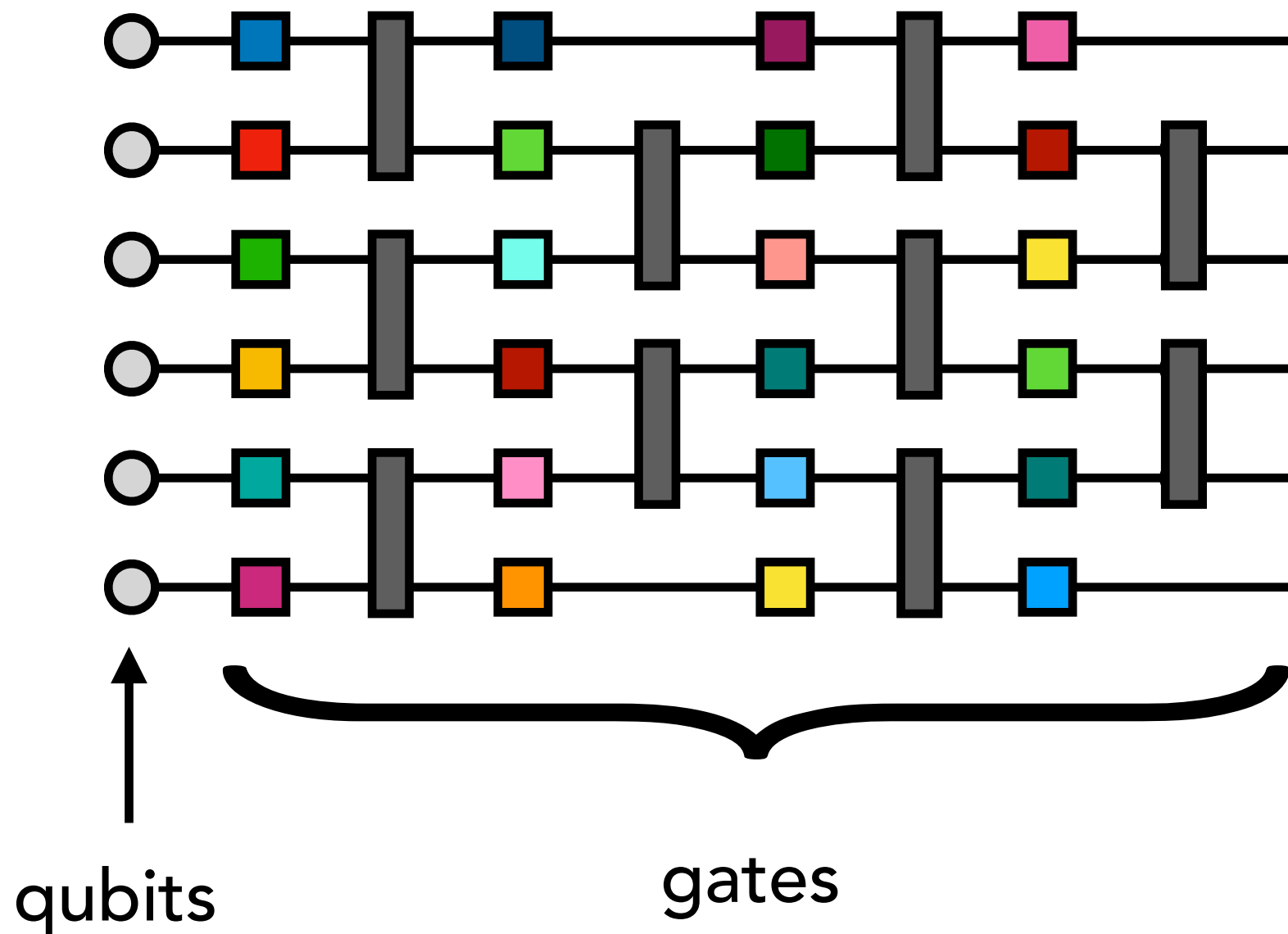
Quantum Circuits

"Program" for a quantum computer is a
quantum circuit



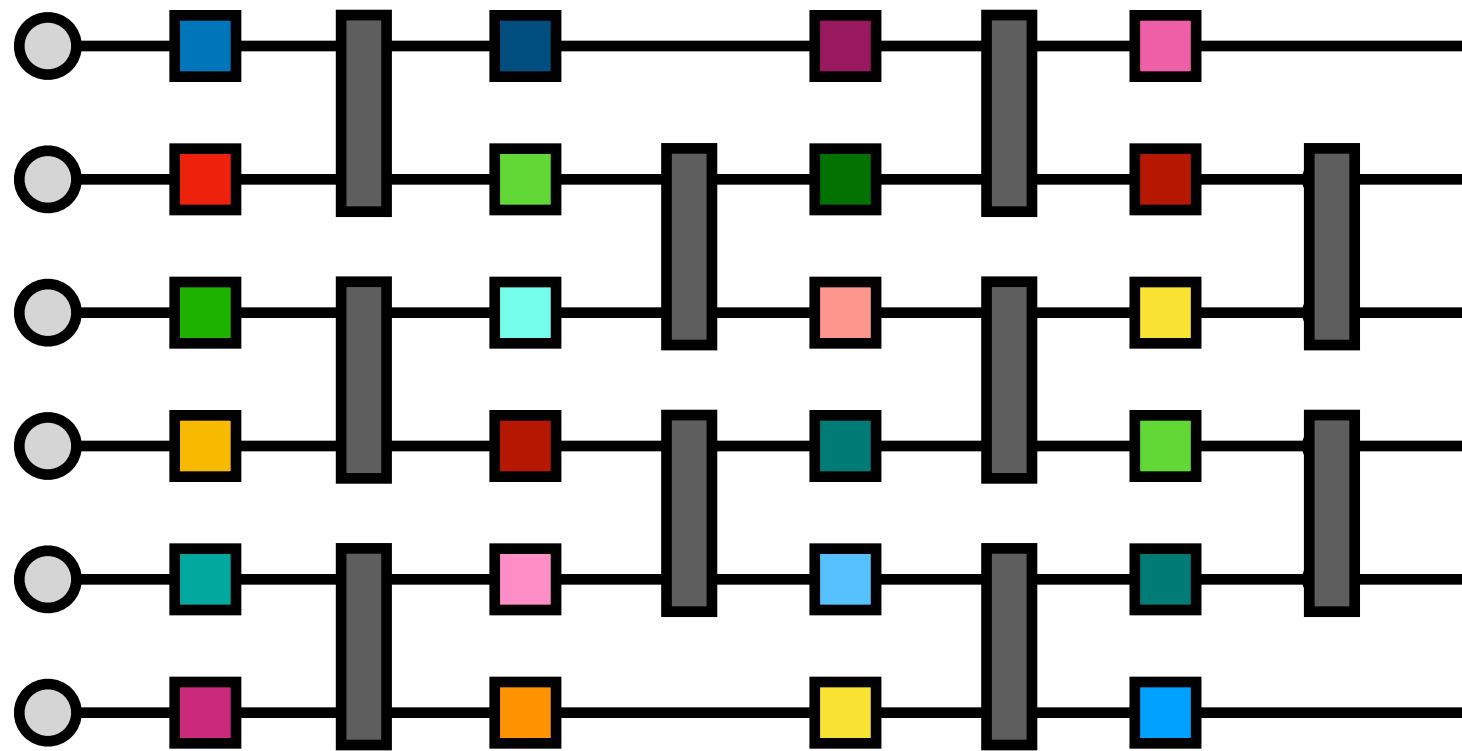
Quantum Circuits

"Program" for a quantum computer is a
quantum circuit



Quantum Circuits

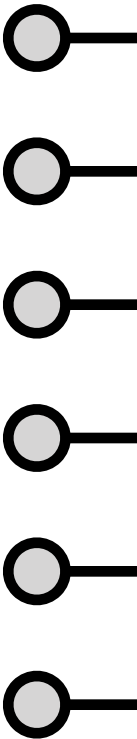
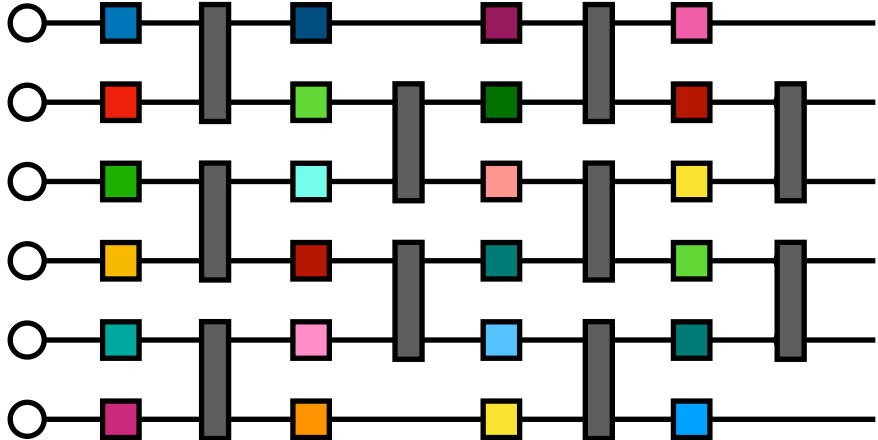
Every quantum circuit in one-to-one correspondence with a tensor network (same diagram)



Let's see how...

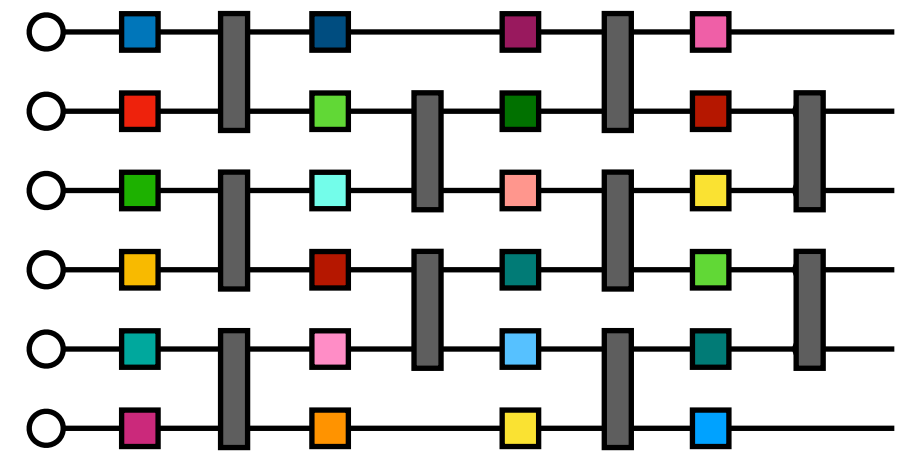
Quantum Circuits

Vectors representing initial state



 = $\begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$

Quantum Circuits



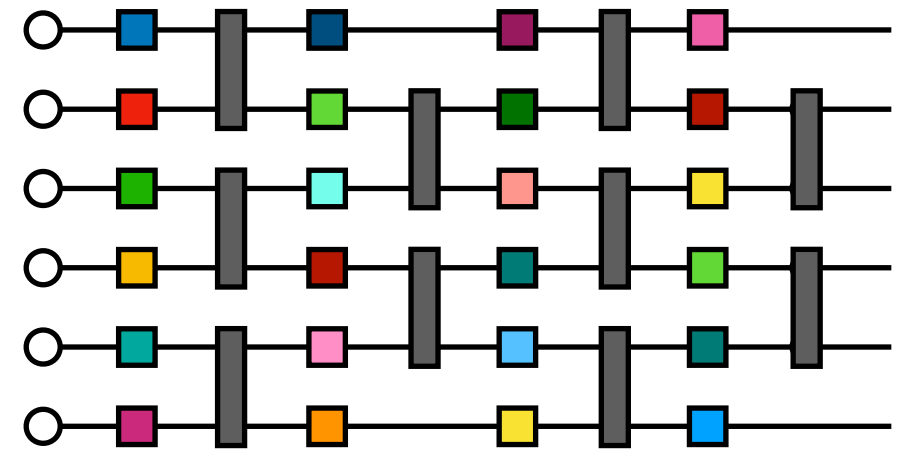
Matrices representing single-qubit gates

$$s \text{ --- } \text{[blue square]} \text{ --- } s' = U_{ss'}$$

For example, Hadamard gate

$$U = H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Quantum Circuits



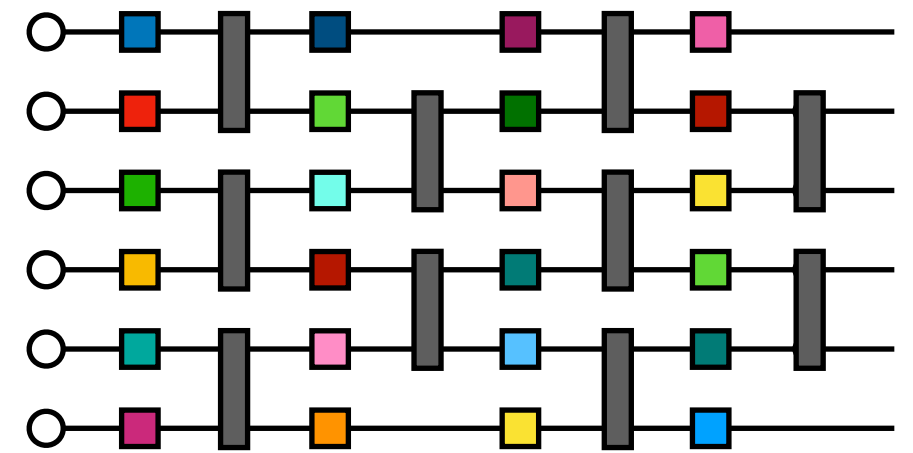
Matrices representing single-qubit gates

$$s \text{ --- } \text{■} \text{ --- } s' = U_{ss'}$$

For example, NOT gate

$$U = NOT = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Quantum Circuits



Tensors representing two-qubit gates

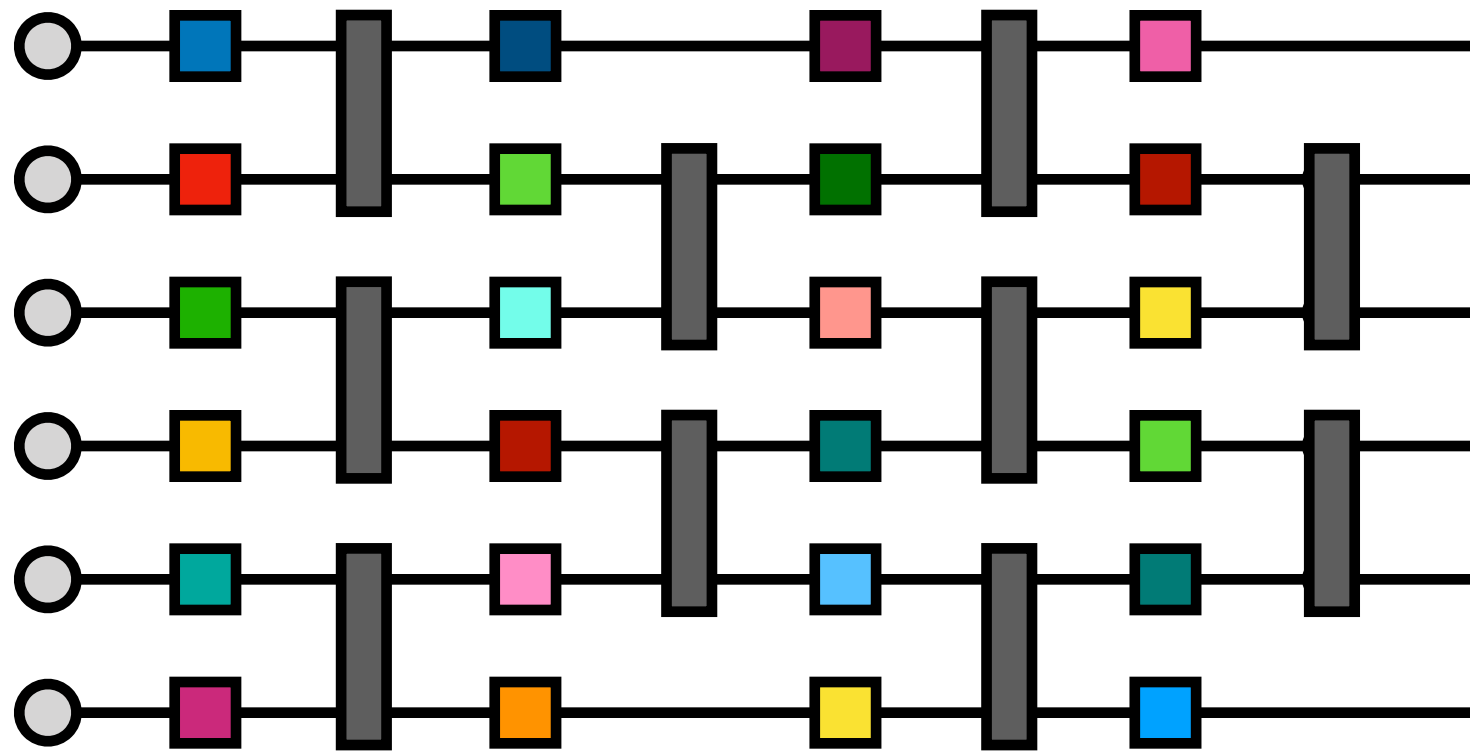
$$\begin{array}{ccc} s_1 & \text{---} \text{ [red vertical bar] } & s'_1 \\ s_2 & \text{---} & s'_2 \end{array} = U_{s_1 s_2}^{s'_1 s'_2}$$

For example, controlled not (CNOT) gate

$$U = CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

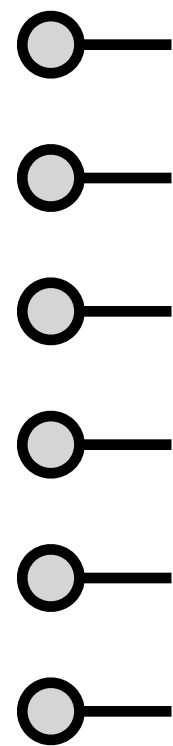
Quantum Circuits

The state after executing the quantum circuit is given by contracting all the tensors



Quantum Circuits

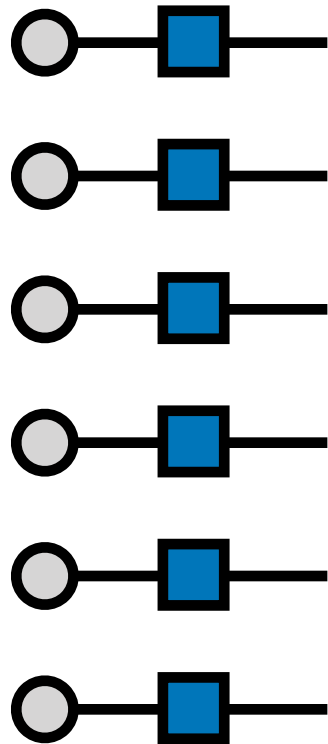
Example:



$$|\Psi\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix}_2 \begin{bmatrix} 1 \\ 0 \end{bmatrix}_3 \begin{bmatrix} 1 \\ 0 \end{bmatrix}_4 \begin{bmatrix} 1 \\ 0 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 0 \end{bmatrix}_6$$

Quantum Circuits

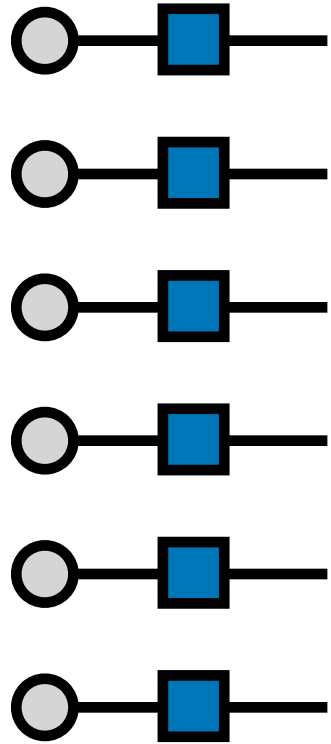
Act with Hadamard gates



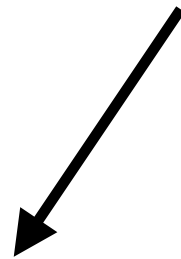
$$|\Psi\rangle = \begin{bmatrix} 1 \\ 1 \end{bmatrix}_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_6 / \sqrt{2}^6$$

Quantum Circuits

Act with Hadamard gates



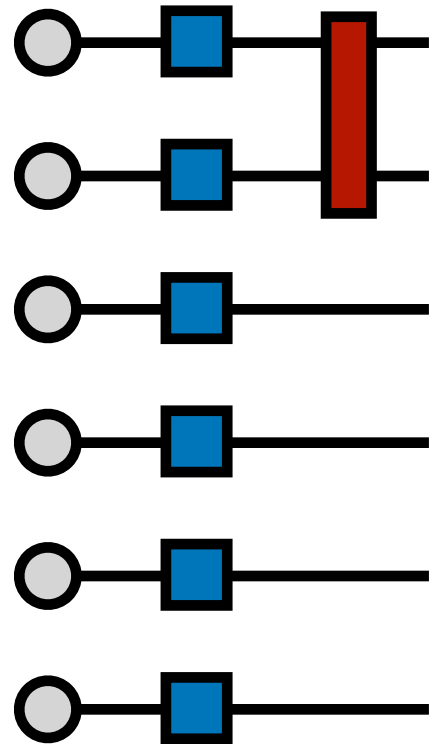
Unentangled state,
massive superposition



$$|\Psi\rangle = \begin{bmatrix} 1 \\ 1 \end{bmatrix}_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_6 / \sqrt{2}^6$$

Quantum Circuits

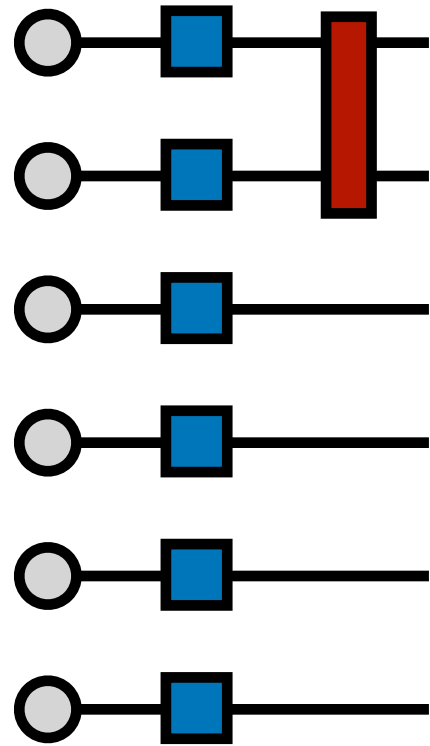
Act with CZ gate on qubits 1 & 2



$$|\Psi\rangle = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}_{1,2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_6 / \sqrt{2}^6$$

Quantum Circuits

Act with CZ gate on qubits 1 & 2

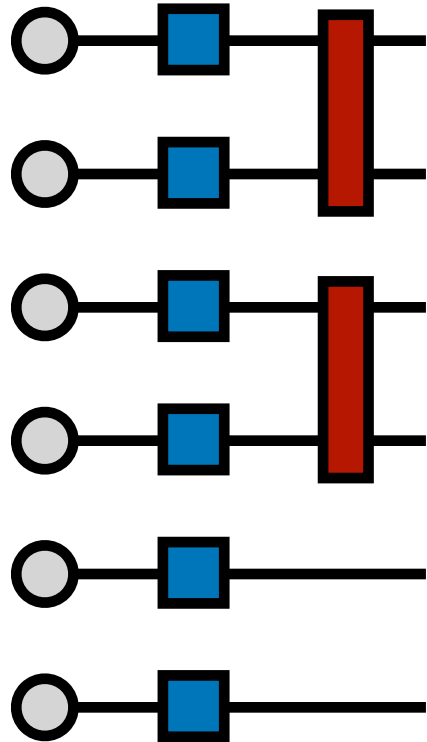


Entanglement
between qubits 1 & 2

$$|\Psi\rangle = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}_{1,2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_6 / \sqrt{2}^6$$

Quantum Circuits

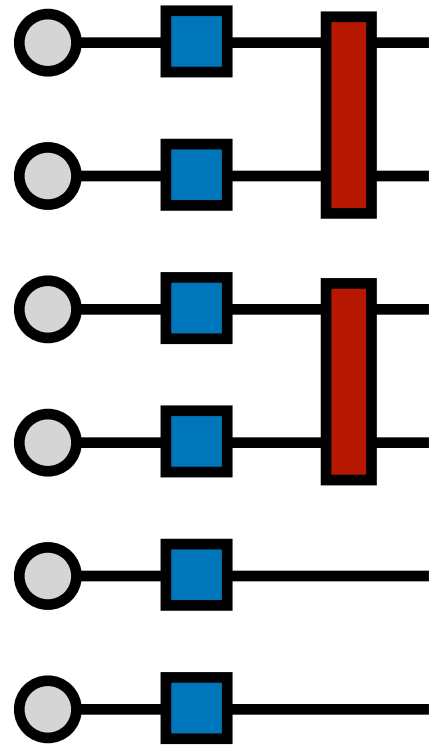
Act with CZ gate on qubits 3 & 4



$$|\Psi\rangle = \left[\begin{array}{cc} 1 & 1 \\ 1 & -1 \end{array} \right]_{1,2} \left[\begin{array}{cc} 1 & 1 \\ 1 & -1 \end{array} \right]_{3,4} \left[\begin{array}{c} 1 \\ 1 \end{array} \right]_5 \left[\begin{array}{c} 1 \\ 1 \end{array} \right]_6 / \sqrt{2}^6$$

Quantum Circuits

Act with CZ gate on qubits 3 & 4

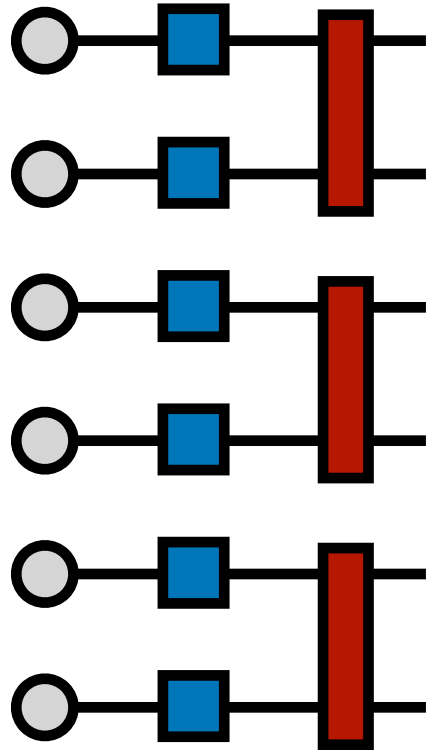


Entanglement
between qubits 3 & 4

$$|\Psi\rangle = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}_{1,2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}_{3,4} \begin{bmatrix} 1 \\ 1 \end{bmatrix}_5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}_6 / \sqrt{2}^6$$

Quantum Circuits

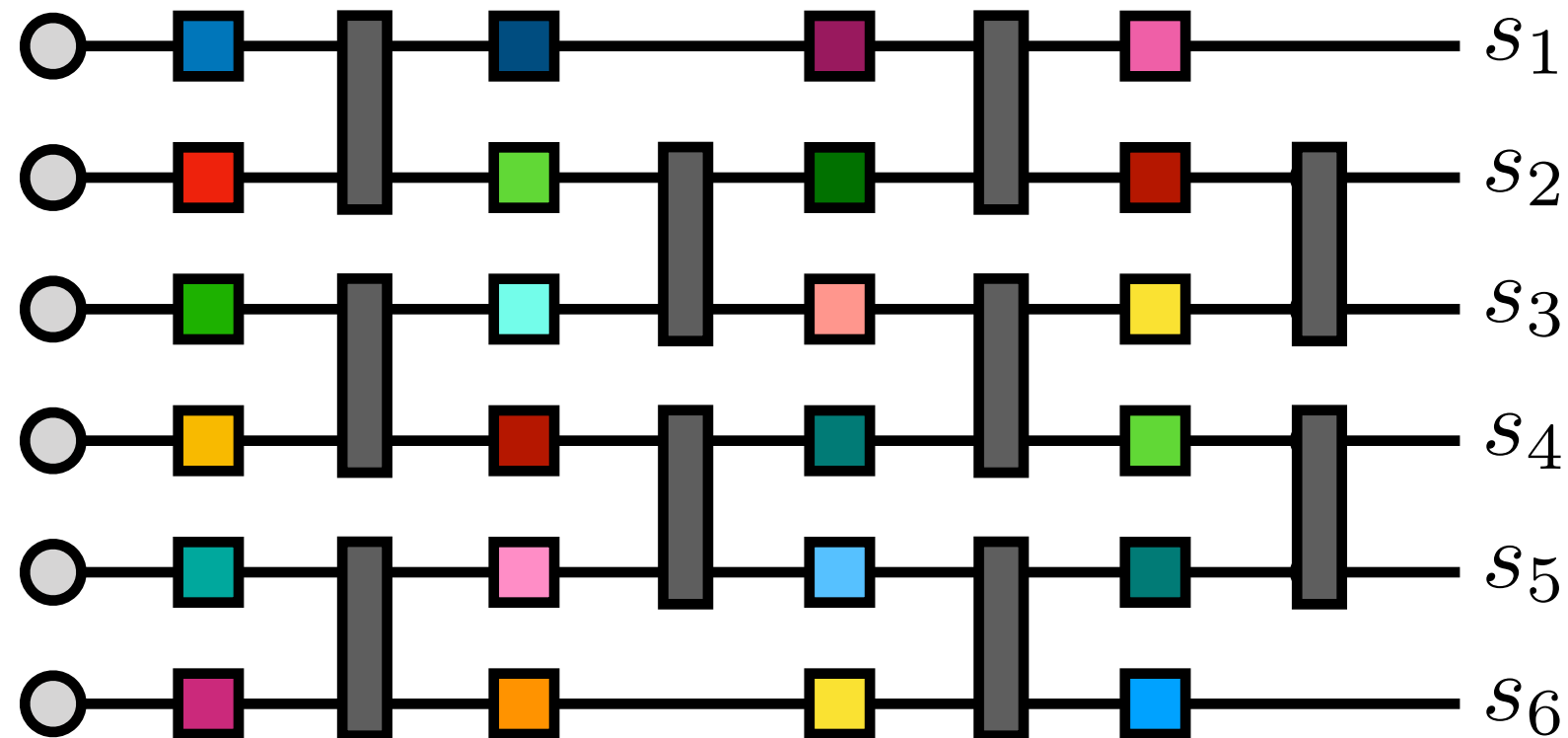
Act with CZ gate on qubits 5 & 6



$$|\Psi\rangle = \left[\begin{array}{cc} 1 & 1 \\ 1 & -1 \end{array} \right]_{1,2} \left[\begin{array}{cc} 1 & 1 \\ 1 & -1 \end{array} \right]_{3,4} \left[\begin{array}{cc} 1 & 1 \\ 1 & -1 \end{array} \right]_{5,6} / \sqrt{2}^6$$

Quantum Circuits

After enough gates, state can get complicated



$$|\Psi\rangle =$$

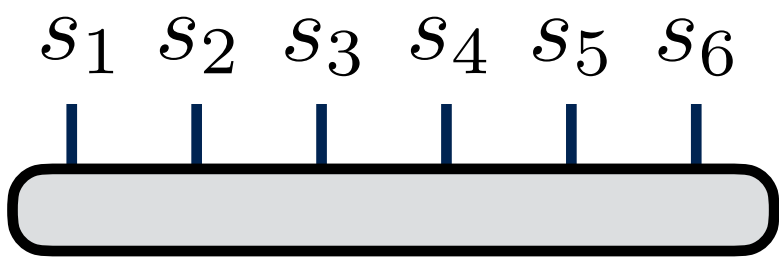
s_1 s_2 s_3 s_4 s_5 s_6



complicated tensor
defined by circuit
(quantum state)

Quantum Circuits

Quantum computing forces you to encounter large tensors

$$|\Psi\rangle = \begin{array}{cccccc} s_1 & s_2 & s_3 & s_4 & s_5 & s_6 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ \text{---} \end{array} \leftarrow \begin{array}{l} \text{complicated tensor} \\ \text{defined by circuit} \\ \text{(quantum state)} \end{array}$$


What if we could work with large tensors on a classical computer?

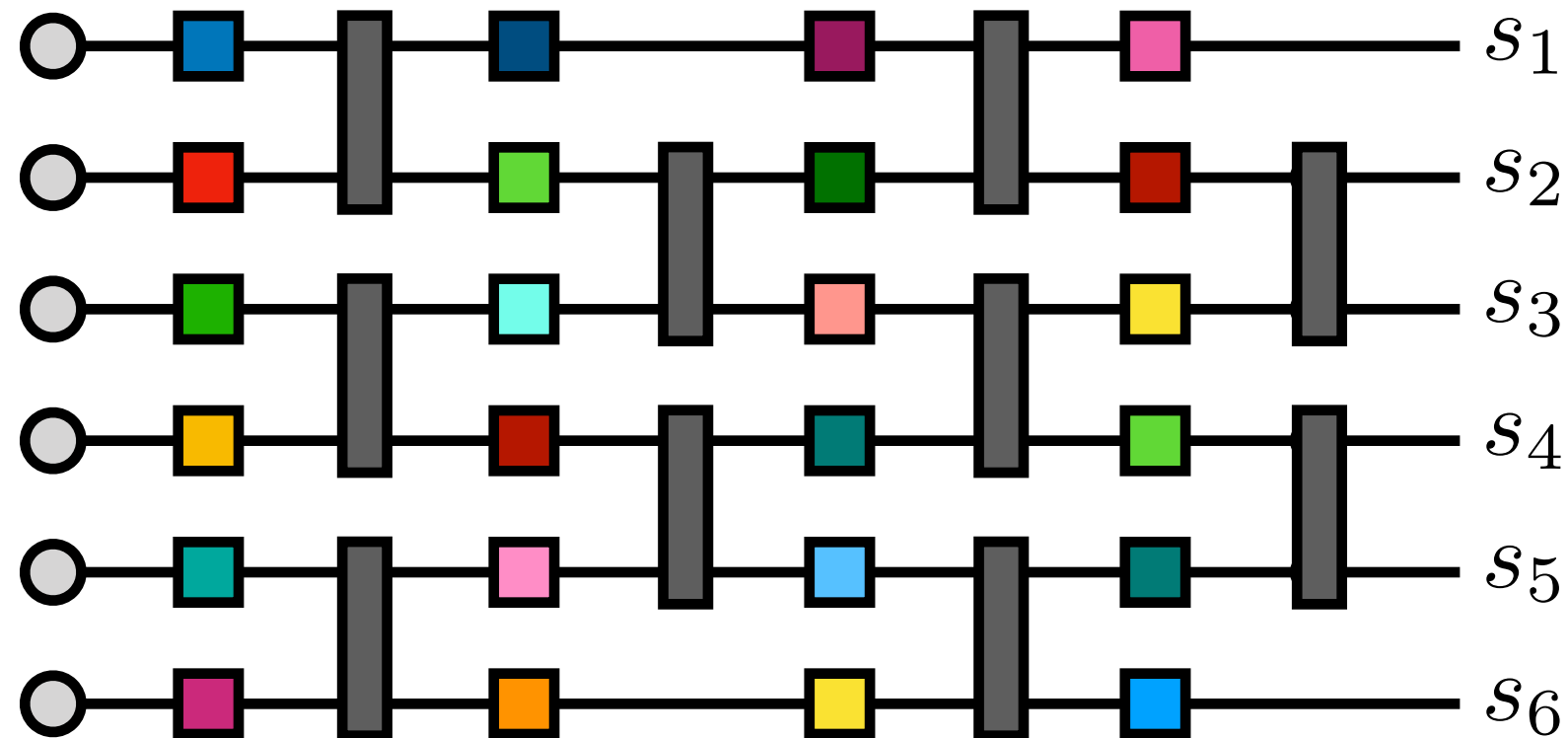
Tensor Network Quantum Simulations

References:

- Zhou, EMS, Waintal, "What Limits the Simulation of Quantum Computers?", [PRX 10, 041038](#) (2020)
- Ayrat et al., "DMRG Algorithm for Simulating Quantum Circuits with a Finite Fidelity", [PRX Quantum 4, 020304](#) (2023)
- Vidal, "Efficient Classical Simulation of Slightly Entangled Quantum Computations", [Phys. Rev. Lett. 91, 147902](#) (2003)
- Markov, Shi, "Simulating Quantum Computation by Contracting Tensor Networks", [SIAM J. Computing, 38, 963-981](#) (2001)

Quantum Circuits

After enough gates, state can get complicated



$$|\Psi\rangle =$$

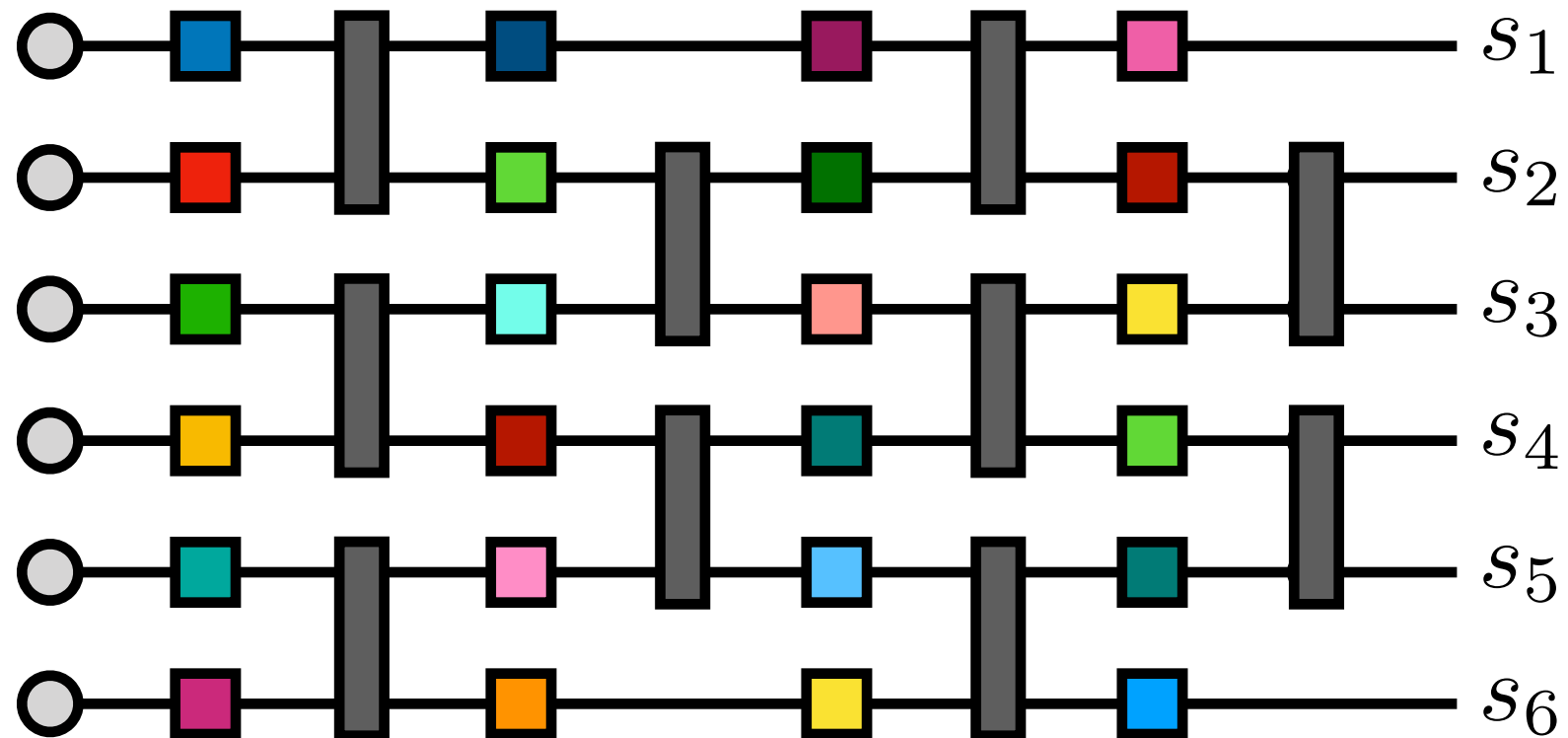
s_1 s_2 s_3 s_4 s_5 s_6



complicated tensor
defined by circuit
(quantum state)

Quantum Circuits

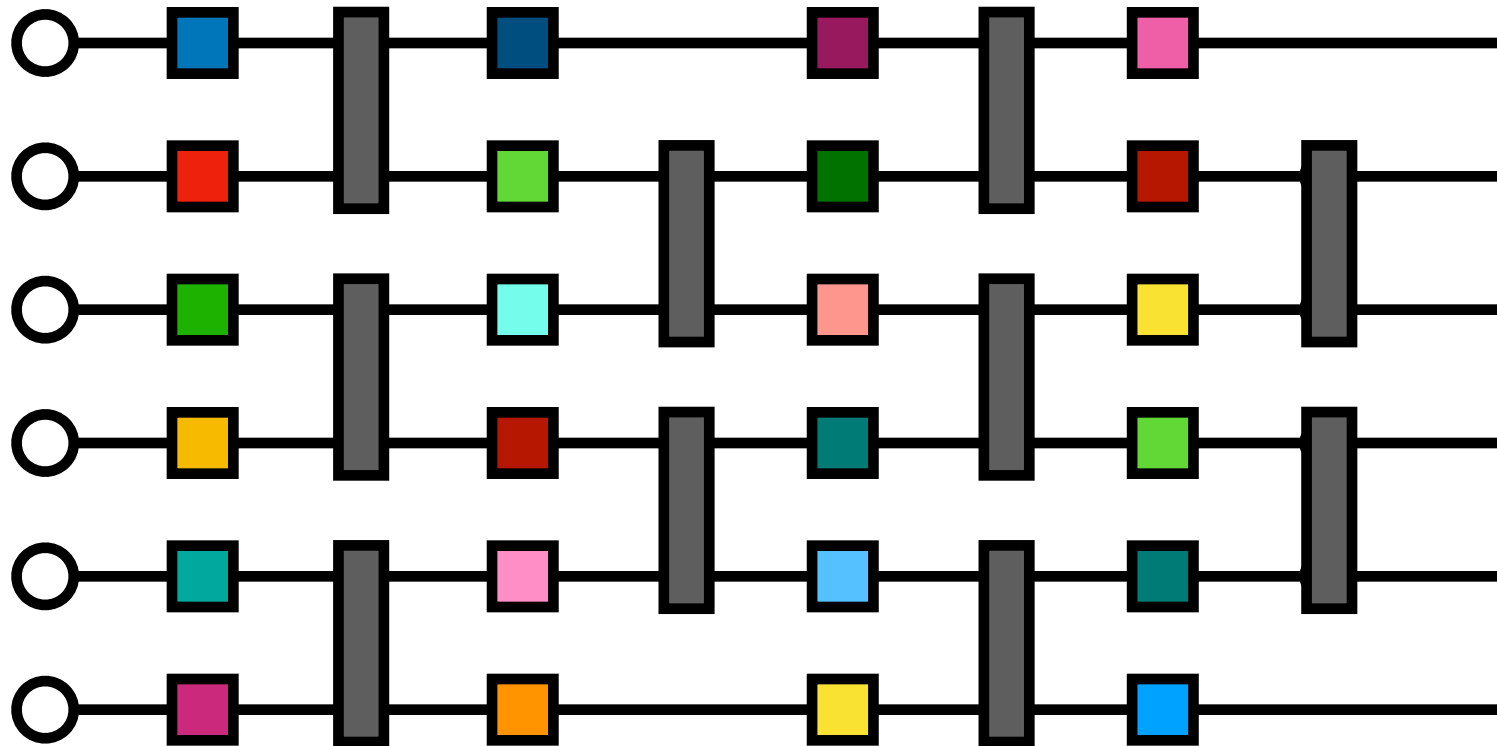
For structured circuits, final state *might* be MPS



$$|\Psi\rangle = \begin{array}{c} s_1 \quad s_2 \quad s_3 \quad s_4 \quad s_5 \quad s_6 \\ \text{[Diagram of a long gray bar with six vertical lines extending upwards from the top, labeled } s_1 \text{ through } s_6 \text{]} \end{array} \approx \begin{array}{c} s_1 \quad s_2 \quad s_3 \quad s_4 \quad s_5 \quad s_6 \\ \text{[Diagram of six blue circles connected in a horizontal chain, with vertical lines extending upwards from each circle, labeled } s_1 \text{ through } s_6 \text{]} \end{array}$$

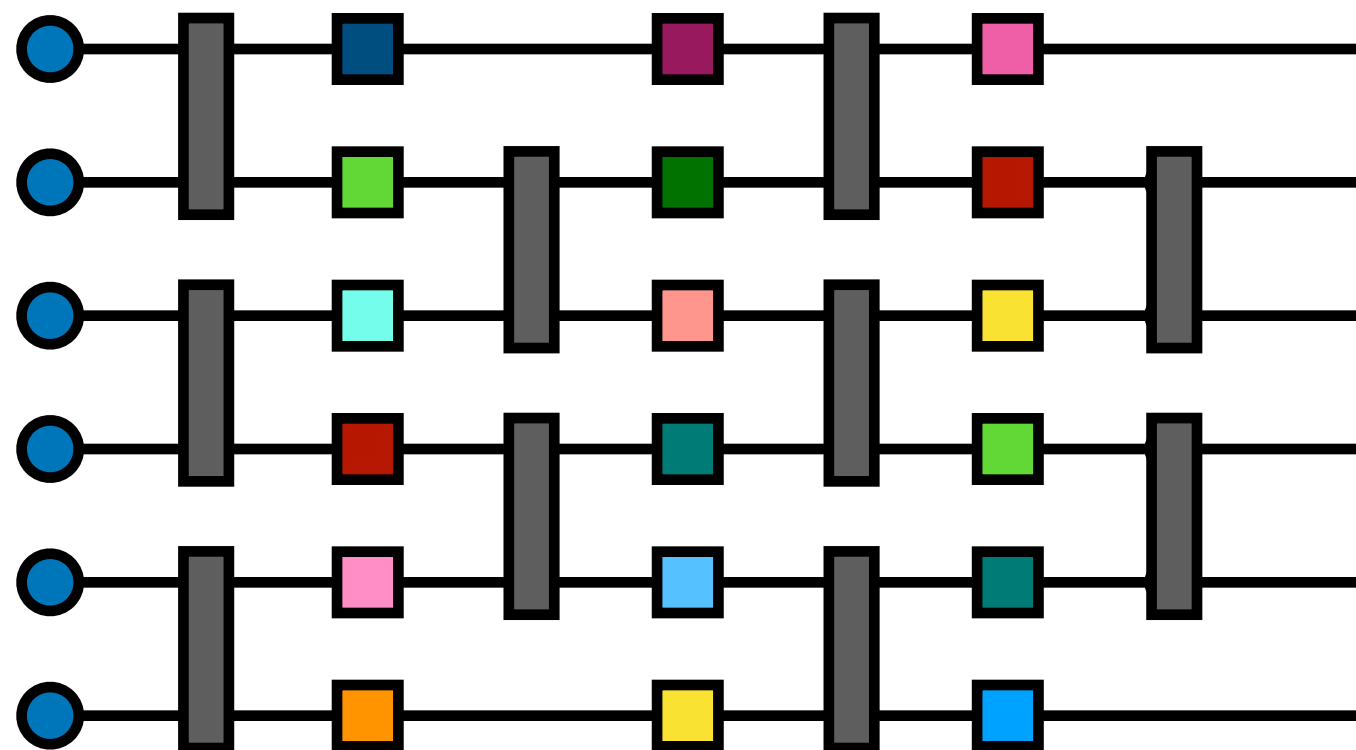
TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm



TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

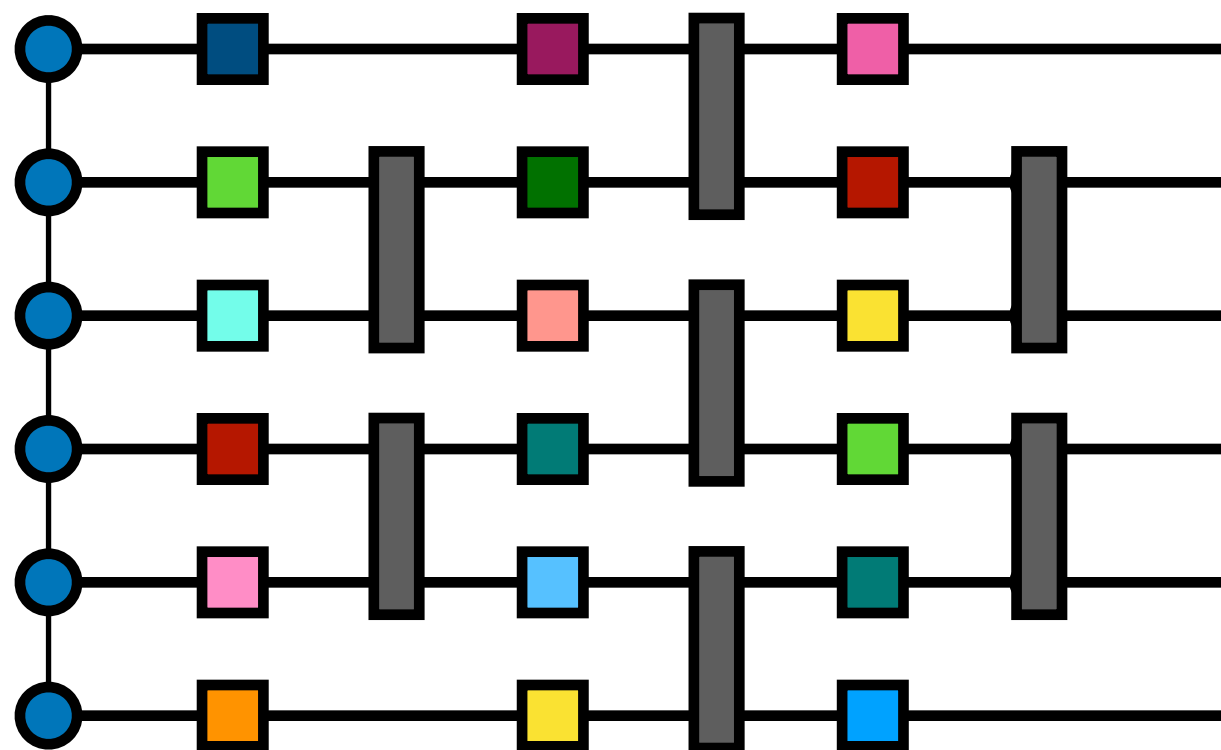


Fidelity



TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

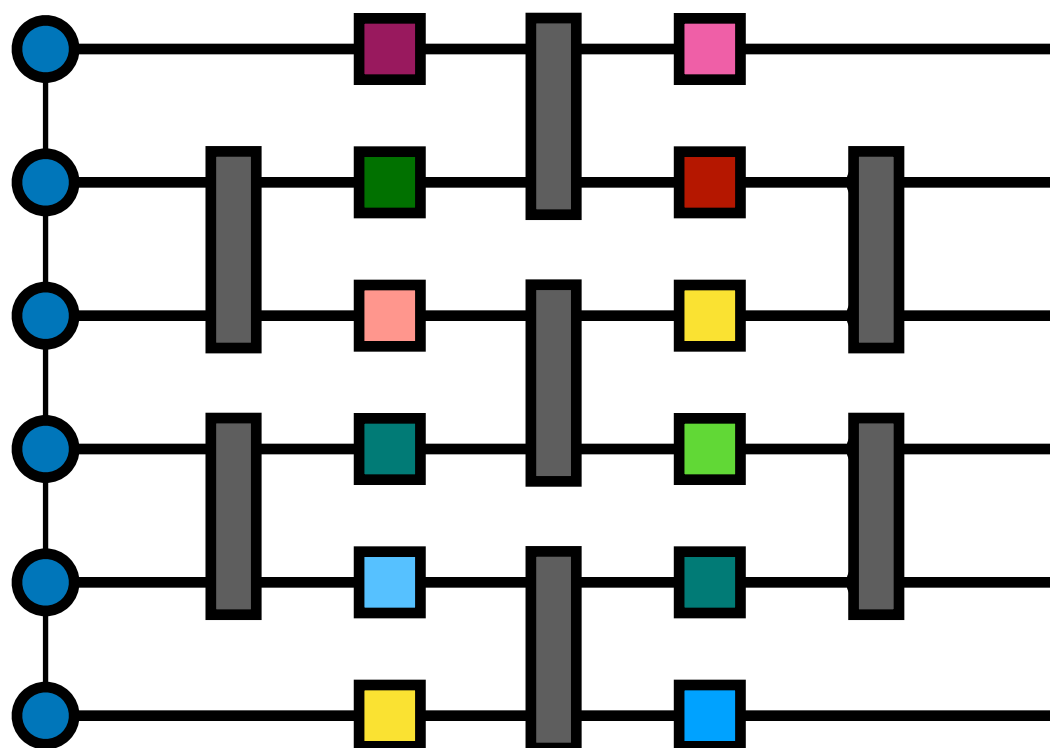


Fidelity



TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

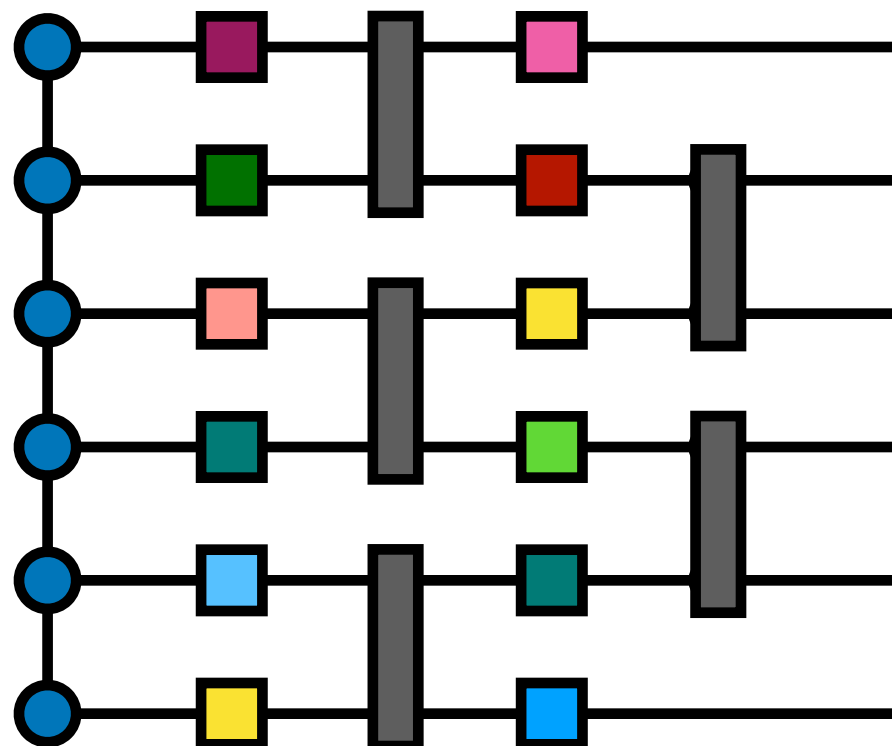


Fidelity



TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

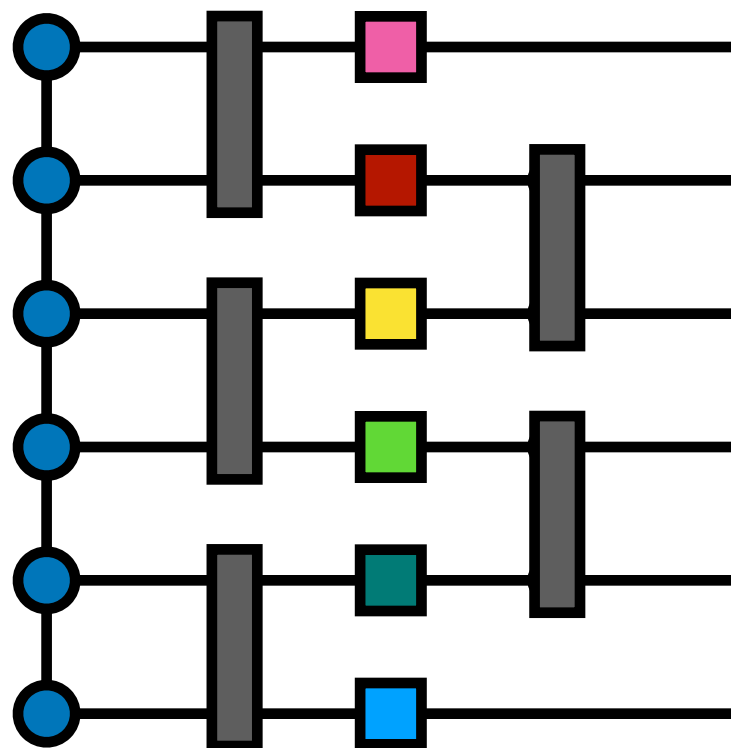


Fidelity



TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

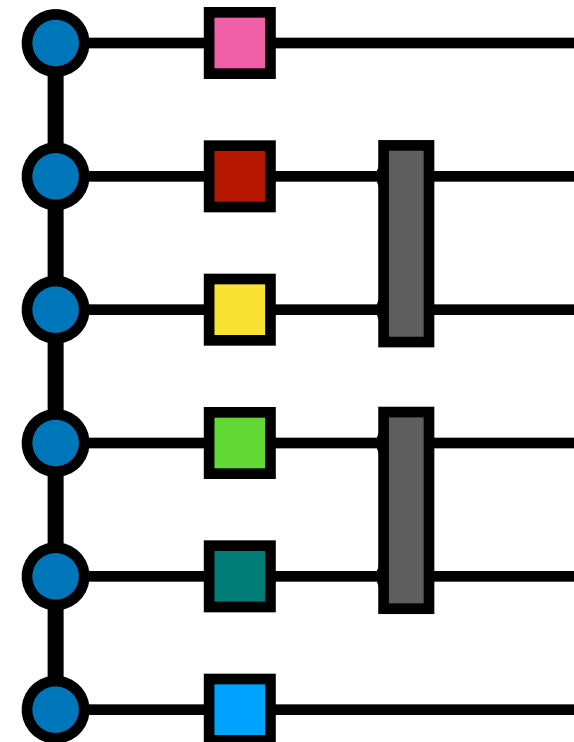


Fidelity

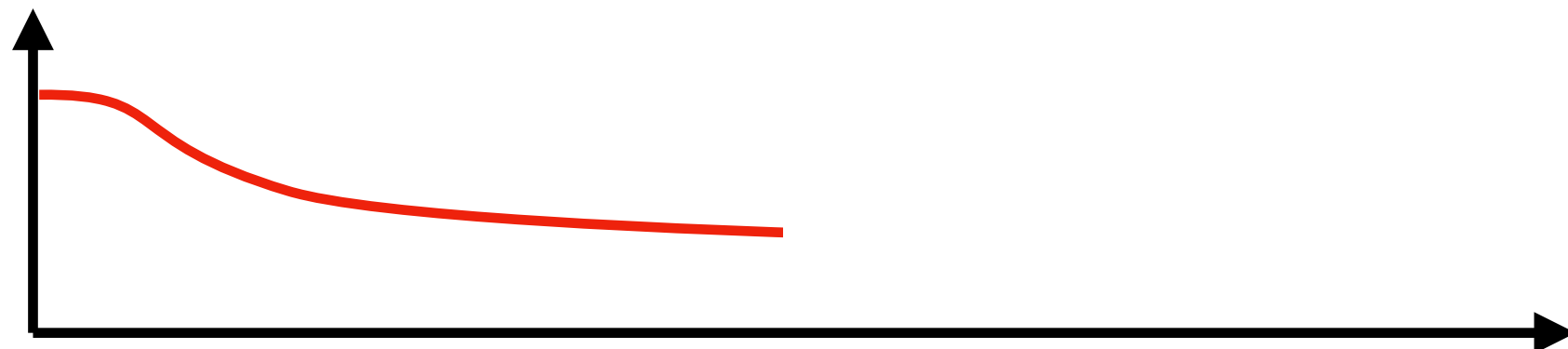


TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

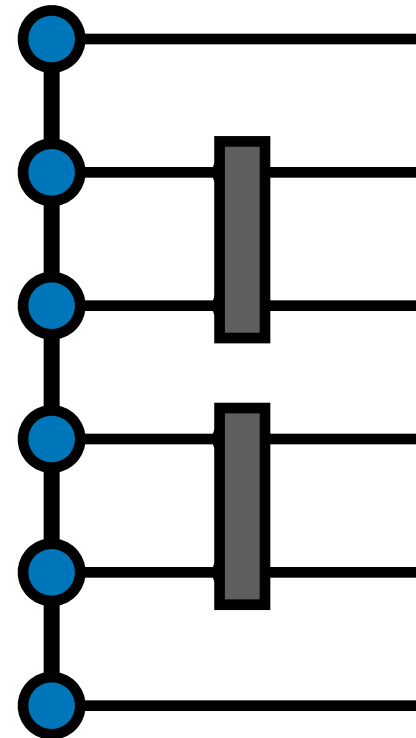


Fidelity

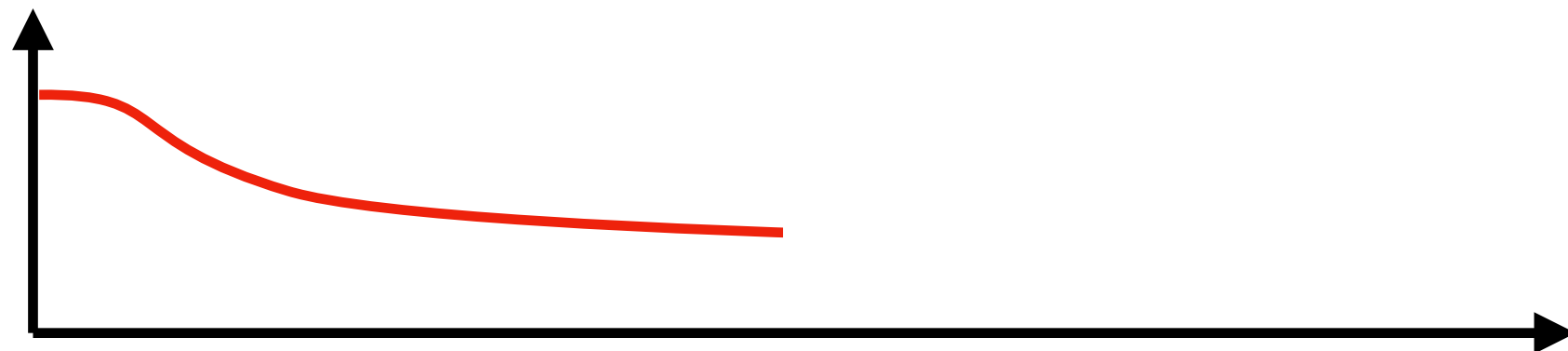


TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm



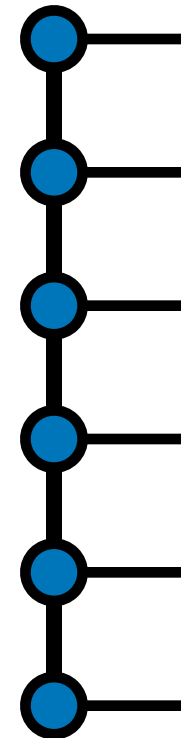
Fidelity



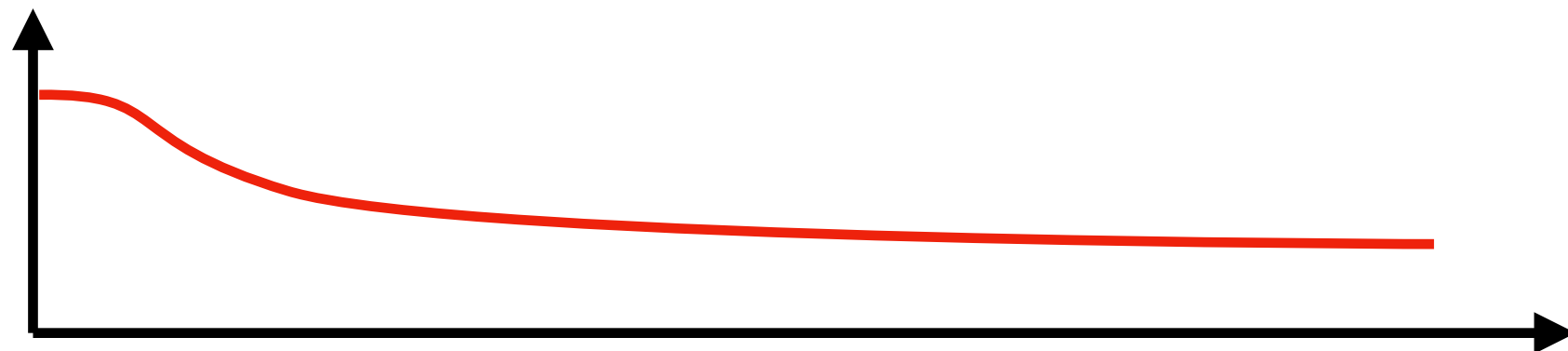
TEBD Algorithm

Simulation of 1D quantum circuit with
TEBD algorithm

obtain entire state afterward



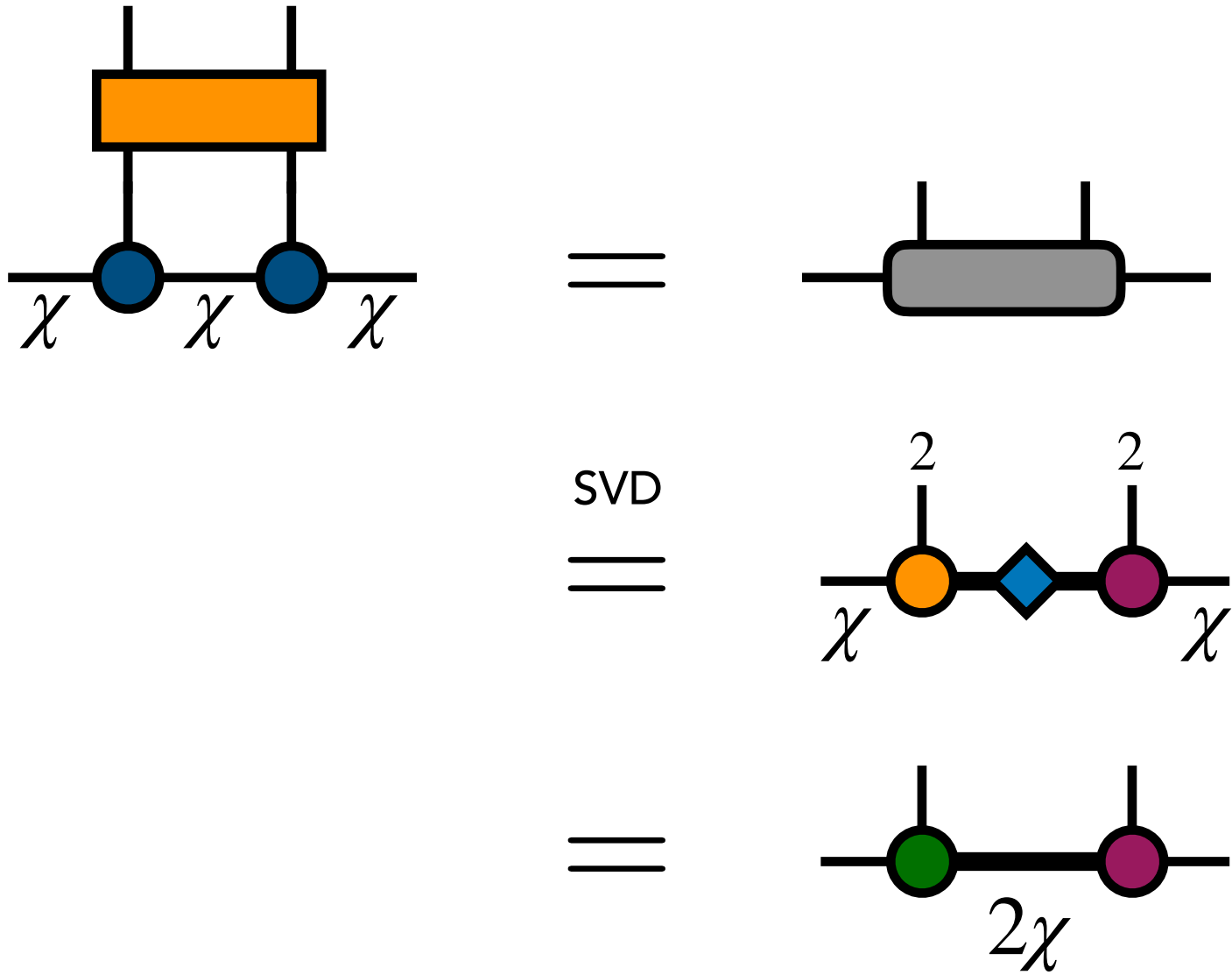
Fidelity



Tensor Network Simulations

When can tensor network simulations succeed?

Worst case, bond dimension χ doubles every layer



Tensor Network Simulations

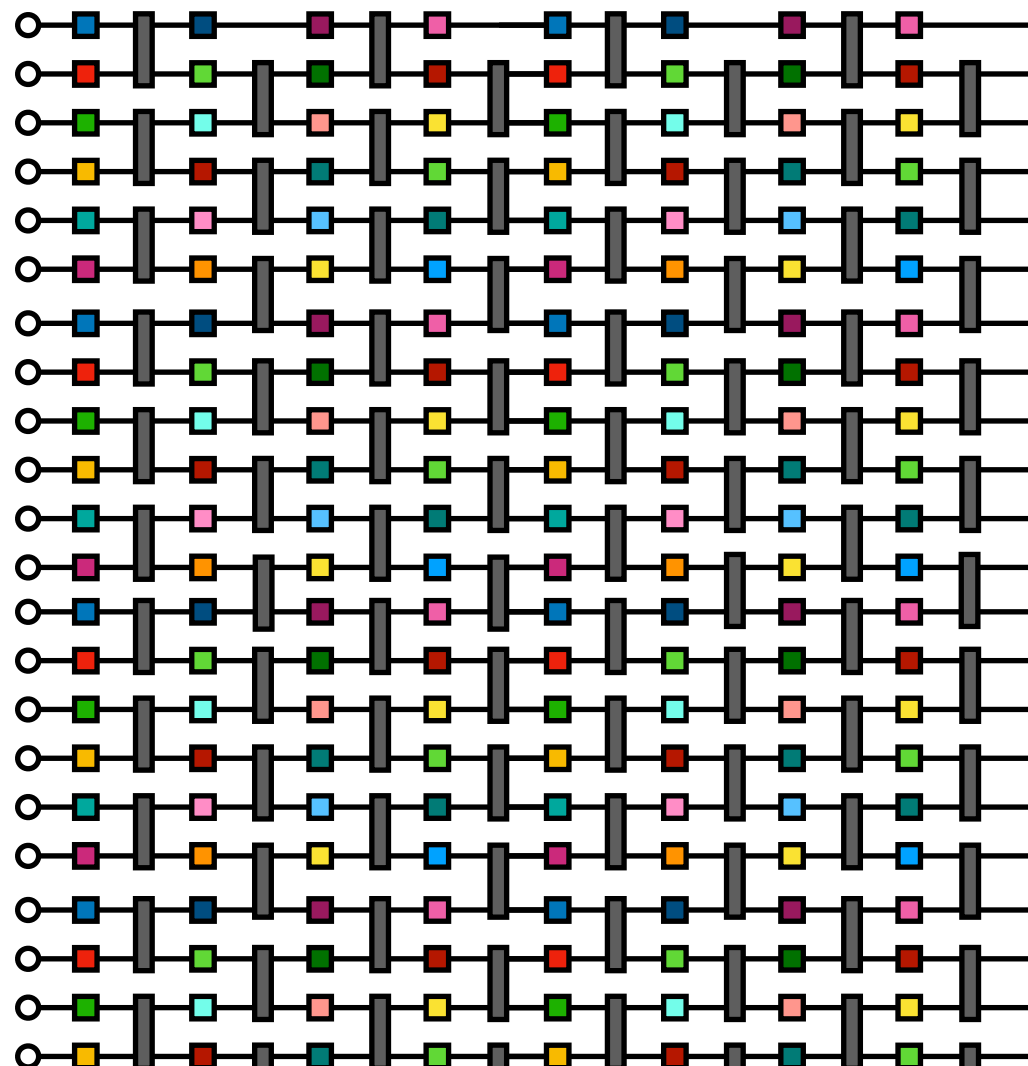
Worst case, bond dimension χ doubles every layer

Exponential growth... but

automatically means ~ 10 layers is always possible in 1D
($2^{10} = 1024$)

Holds even for thousands of qubits

An easy
simulation:

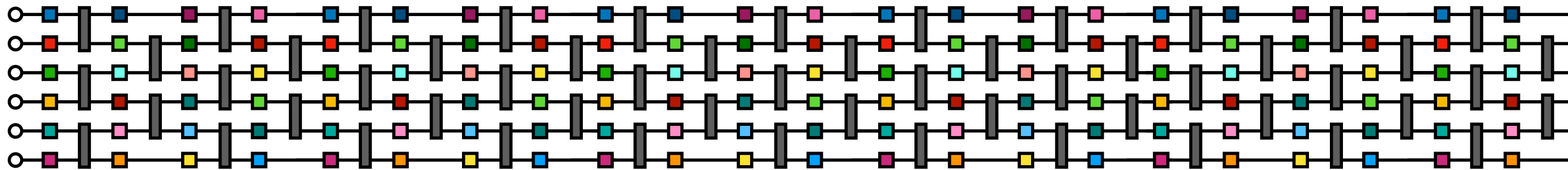


Tensor Network Simulations

Worst case, bond dimension χ doubles every layer

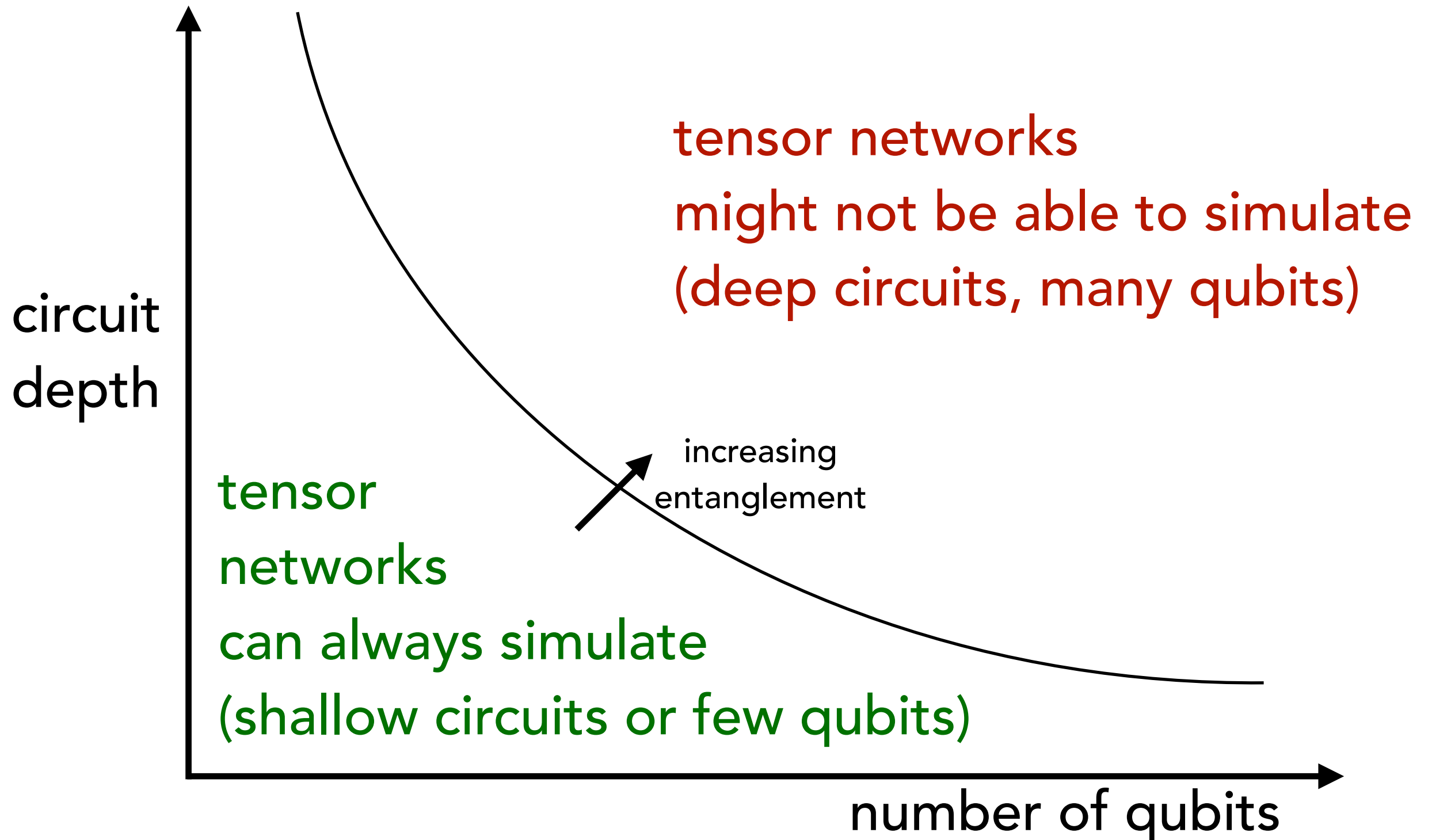
If **number of qubits small** MPS can also do any number of layers

(20 qubits gives $\max \chi = 2^{10}$)



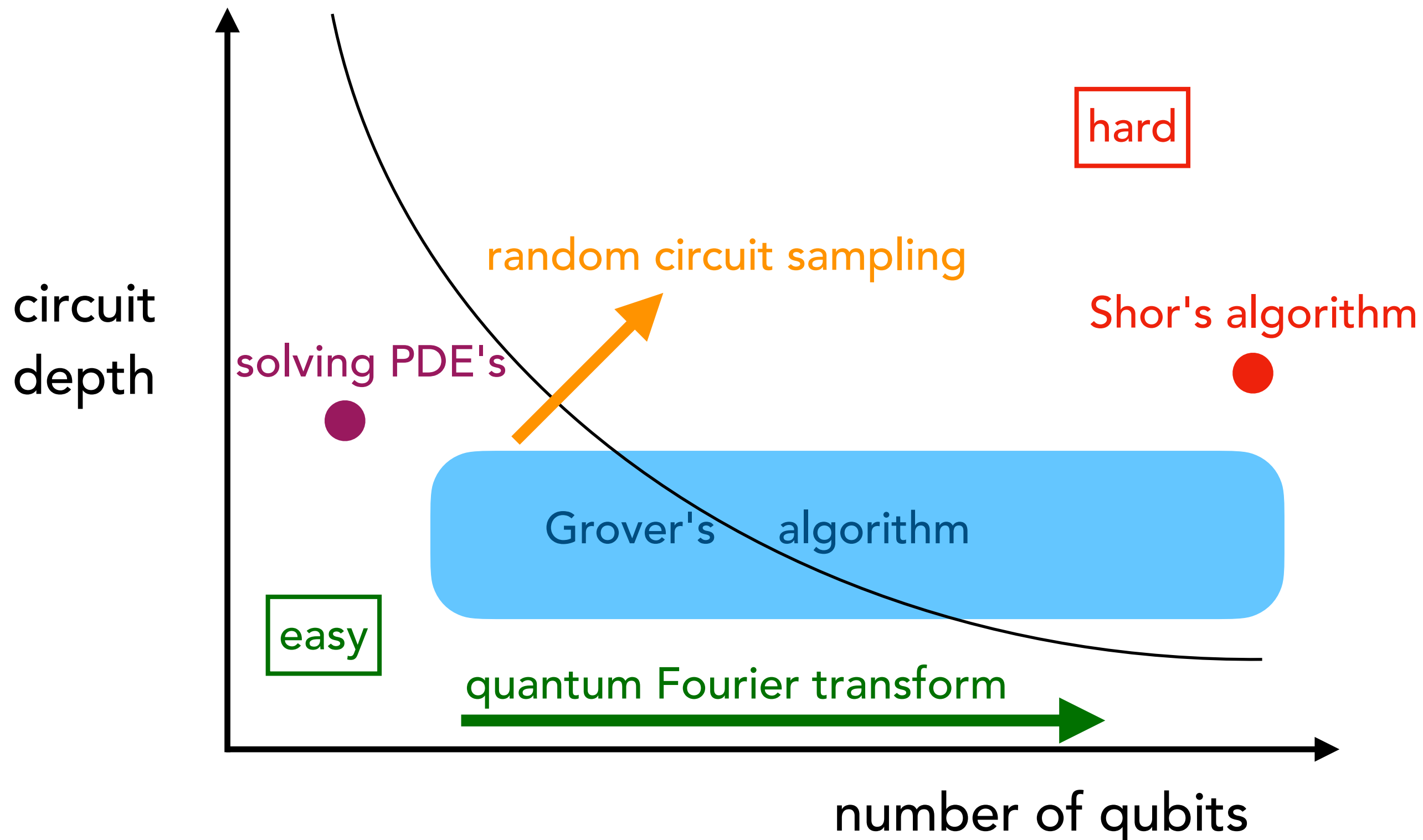
Tensor Network Simulations

Putting this together*



Tensor Network Simulations

Emerging picture of what is easy vs. hard for tensors



Known capabilities of quantum computer vs quantum-inspired tensor network

	Quantum Computer	Tensor Network
unitary operations	yes	yes
sampling / measurement	yes	yes
non-unitary operations	no	yes
"forced" measurement	no	yes
use unnormalized states	no	yes
moderate entanglement	yes	yes
high entanglement	yes, but how high?	no